

CSS

Cascading Style Sheets

CSS

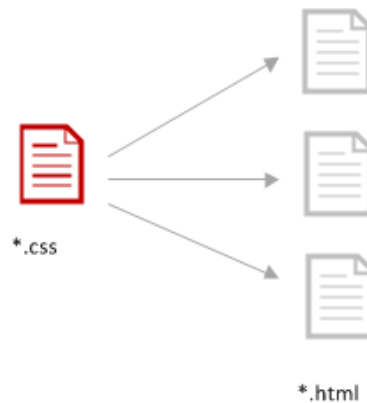
- Incremento da qualidade e consistência gráfica das aplicações
- Separação do **design e do conteúdo**:
 - **HTML** define a estrutura (conteúdo)
 - **CSS** define a formatação/design
 - Layouts
 - Posicionamentos
 - Cores
 - Texto
 - Formulários
 - Tabelas
 - Listas
 - ...

*



■ Vantagens

- Possibilidade de aplicação imediata a vários documentos HTML
 - Garante que diversos documentos HTML são submetidos às mesmas regras de formatação.
- Muito maior flexibilidade
 - Alterações centralizadas
 - Evita a repetição de formatações
- Maior correção
 - Melhora a portabilidade
 - Reduz a possibilidade de erros
 - Maior uniformidade
 - Melhora a consistência gráfica



Sintaxe CSS

- Composta por:
 - Seletor
 - Declaração /conjunto de declarações
 - Propriedade
 - Valor

seletor { propriedade:valor;

- Cada regra seleciona um elemento/conjunto de elementos e declara toda a formatação associada
- A sintaxe das CSS é **case sensitive**

■ Seletor

- Elemento crucial na definição de uma regra CSS
 - A compreensão do funcionamento dos seletores é absolutamente essencial para permitir uma correta implementação de CSS
- Permite identificar os elementos HTML a que a regra CSS se aplica



```
body{ background-color:#FFF;}
```

■ Declaração

- Uma atribuição de um valor a uma propriedade. Uma regra pode conter várias declarações (*declaration block*).
 - Propriedade
 - Atributo da folha de estilo ao qual deve ser atribuído um valor (ex: *color*, ...)

```
body{ background-color:#FFF;}
```

- Valor

- Especificação concreta da variável (ex: *arial*, *#0000FF*)

```
body{ background-color:#FFF;}
```

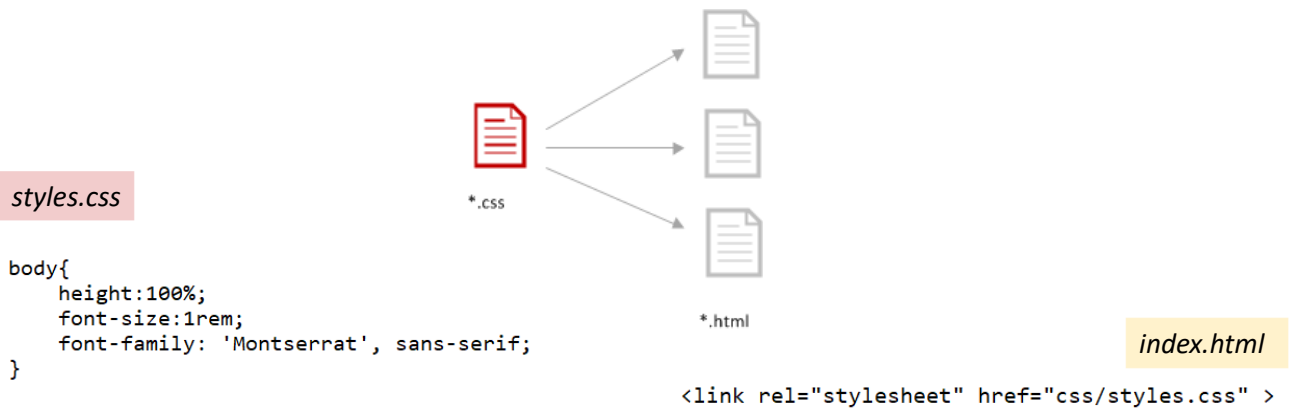
Sintaxe	Exemplo
selector { propriedade: valor}	body {background-color: #FFFFFF}
<i>bloco de declarações ;</i>	
selector { propriedade: valor; propriedade: valor; propriedade: valor; }	p { text-align:center; color:red; font-family: arial;} }
<i>seleção múltipla ,</i>	
selector1, selector2, selector3 { propriedade: valor;}	h1, h2, h3{ color:red;}

Ligação CSS / HTML

- Como aplicar estilos (regras CSS) aos documentos HTML?

- **External Style Sheet**

- Ficheiro externo de extensão *.css que contém as regras CSS
 - Pode ser aplicado a diversos documentos HTML
 - Forma mais poderosa e flexível de incorporar CSS



- **External Style Sheet (*.css)**

- Um ficheiro CSS (*.css) é exclusivamente constituído por regras CSS
 - O código CSS não contém tags

seletor{propriedade:valor;}

```
/* style1.css */

p { font-weight:bold;
    color:#F00;}
```

■ `<link ... />`

- Atributo **href** indica a localização do ficheiro *.css . Atributo obrigatório!
- Atributo **rel** define a relação do documento HTML com o ficheiro externo, neste caso é *stylesheet*. Atributo obrigatório!
- Atributo **type** é opcional na última versão do HTML5 (legado de versões anteriores)

```
<!DOCTYPE HTML>
<html>
<head>
<meta charset="utf-8" />
<title> External Style Sheet</title>

<link href="style1.css" rel="stylesheet" type="text/css"/>

</head>

<body>
</body>
</html>
```

```
/* style1.css */
```

```
p { font-weight:bold;
    color:#F00;}
```

■ **External Style Sheet**

- através de **@import** e da tag **<style>**

```
<!DOCTYPE HTML>
<html>
<head>
  <style>
    @import url("style1.css");
    ...

  </style>
  <meta charset="utf-8" />
  <title> External Style Sheet</title>
</head>

<body>
</body>
</html>
```

```
/* style1.css */
```

```
p { font-weight:bold;
    color:#F00;}
```

- as duas formas de estabelecer a ligação a um ficheiro externo são suportadas pela maioria dos browsers originando resultados semelhantes, no entanto:
 - A tag **<link>** é a forma mais eficaz, aquela que assegura melhor performance, para efetuar a ligação de uma HTML a um CSS externo

■ *Embedded Style Sheet*

- Necesita de ser definido em todos os documentos *.html
 - Menos flexível e eficaz do que a ligação a um ficheiro externo
 - Aumenta o peso dos ficheiros
 - Uma alteração na formatação implica uma alteração em cada um dos ficheiros
 - em MPA pode originar inconsistência na formatação entre diferentes *.html



■ `<style> ... </style>`

- definido do `<head>` do *.html
 - Atributo `type="text/css"` é opcional (obrigatório nas versões anteriores ao HTML5).

Como é óbvio **não podem** ser definidas tags HTML nesta janela, trata-se de código CSS

```
<!DOCTYPE HTML>
<html>
<head>
  <style>
    p { font-weight:bold;
        color:#F00;}
  </style>
  <meta charset="utf-8" />
  <title> Embedded Style Sheet</title>
</head>

<body>
</body>
</html>
```

Janela de
código CSS

■ *Inline Style*

- aplicar o estilo localmente através do atributo **style**
 - É a forma **menos flexível** e **mais propensa** a erros para formatar um elemento HTML
 - Se efetuado de forma estática, favorece a inconsistência na formatação dos conteúdos
 - Implica tantas alterações quantas as definições efetuadas

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
</head>
<body>
  <h1 style="color:orange">Inline Style: A evitar!</h1>
</body>
</html>
```

Inline Style: A evitar!

Seletores CSS

Seletor CSS

- Permite identificar os elementos HTML aos quais a regra CSS se aplica

seletor { propriedade:valor;}

- **Diferentes tipos de seletores** de forma a permitir a flexibilidade necessária para selecionar qualquer dos elementos/conjunto de elementos definidos no HTML
- Os seletores CSS são cruciais para a aplicação da tecnologia CSS, mas também são usados pelo JavaScript / Frameworks JS (ex: React) para aceder a determinado elemento HTML.

Seletores CSS **não se esgotam** na tecnologia CSS!

- Devem ser definidos tendo sempre em consideração os conflitos de formatação:
 - Muito frequentes sempre que o código CSS ganha alguma escala.

Seletor CSS

- Principais tipos de seletores:

Elemento

Contexto

id

class

Atributo

- Principais tipos de seletores:

Elemento

Seletor CSS

- Seletor de **Elemento**

Elemento

- Referência direta ao elemento HTML que se pretende formatar
 - Vantagem: simplicidade
 - Desvantagem: pouco específicos

```
<style>  
p{border:1px solid blue;}  
h2{color:blue}  
</style>
```

- Elementos Agrupados

- Referência direta aos elementos HTML que se pretendem formatar, os quais devem ser separados por **vírgula (,)**

```
<style>  
h1,h2{ color: #900;}  
</style>
```

Seletor CSS

- Seletor Universal (*)
 - Permite selecionar todos os elementos
 - A utilização deste seletor deve ser cuidadosamente ponderada é aplicado a **todos os elementos**
 - podem ser originadas algumas formatações/efeitos não previstos

```
<style>  
  * {color:orange}  
</style>  
</head>  
  
<body>  
  <p>p first-child</p>  
  <ul>  
    <li>first option</li>  
    <li>second option</li>  
  </ul>  
  <p>p last-child</p>  
</body>
```

p first-child

- first option
- second option

p last-child

Seletor CSS

- Principais tipos de seletores:

Contexto

Seletor CSS

Seletor de Contexto

Descendentes

- A formatação dos elementos depende do contexto
- Elementos separados por **espaço em branco**

Contexto

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>

  <style>
    li em {color:orange;}
  </style>
</head>
<body>
  <ul>
    <li><em>1ª opção</em></li>
    <li><em>2ª opção</em></li>
    <li><em>3ª opção</em></li>
  </ul>
  <p>1º <em>parágrafo</em></p>
  <p>2º <em>parágrafo</em></p>
</body>
</html>
```

- 1ª opção
- 2ª opção
- 3ª opção

1º parágrafo

2º parágrafo

Podem ser criados vários níveis de dependências

```
<style>
  ul li em {color:orange;}
</style>
```

Seletor CSS

Seletores de Contexto

Casos particulares de seletores de contexto

- Descendentes diretos (**filhos**) - *child selector* (>)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>

  <style>
    span {color:gray;}
    p > span {color:orange;font-size:1.2em}
  </style>
</head>
<body>
  <p><span>1º <span>parágrafo</span></span></p>
  <p><span>2º <span>parágrafo</span></span></p>
</body>
</html>
```

1º parágrafo

2º parágrafo

Seletor CSS

■ Seletores de **Contexto**

■ Elementos Adjacentes (**Adjacent Sibling Selector**) (+)

- Adjacente: elemento imediatamente seguinte ao elemento que define o contexto

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    → h1 + p {color:orange;font-size:1.2em}
  </style>
</head>
<body>
  <h1 id="p1"> Referência</h1>
  <p><span>1º </span>parágrafo</span></p>
  <p><span>2º </span>parágrafo</span></p>
</body>
</html>
```

Referência

1º parágrafo

2º parágrafo

Seletor CSS

■ Seletores de **Contexto**

■ *element1 ~ element2*

- seleciona todos os *element2* que são precedidos por um *element1*
 - Os elementos devem ter o mesmo pai
 - O *element2* não tem de ser imediatamente precedido pelo *element1*

```
<!DOCTYPE html>
<html lang="">
<head>
  <meta charset="utf-8">
  <title></title>

  <style>
    → h1 ~ p {color:orange}
  </style>
</head>
<body>
  <p> 1º parágrafo </p>
  <h1 id="heading1"> Referência </h1>
  <p> 2º parágrafo </p>
  <p> 3º parágrafo </p>
</body>
</html>
```

1º parágrafo

Referência

2º parágrafo

3º parágrafo

- Principais tipos de seletores:

id

Seletor CSS

- Selectores de ID (#)

- Atribuir um estilo a uma única ocorrência de um elemento HTML

id

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    p {color:gray;}
    #p1 {color:orange;font-size:1.2em}
  </style>
</head>
<body>
  <p id="p1"> Parágrafo com id=p1</p>
  <p><span>1º</span> <span>parágrafo</span></span></p>
  <p><span>2º</span> <span>parágrafo</span></span></p>
</body>
</html>
```

Parágrafo com id=p1

1º parágrafo

2º parágrafo

- Os id's são únicos e como tal dispensam a especificação do elemento
 - seletores equivalentes
 - Especificidade diferente

#p1 p#p1

Seletor CSS

- Seletor de ID (#)
 - Podem ser utilizados como parte de um seletor de contexto

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li {color:gray;}
    #lista1 li {color:orange;}
  </style>
</head>
<body>
  <ul id="lista1">
    <li>1ª opção / lista id="lista1" </li>
    <li>2ª opção / lista id="lista1" </li>
  </ul>
  <ul>
    <li>1ª opção / lista sem id </li>
    <li>2ª opção / lista sem id </li>
  </ul>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"
- 1ª opção / lista sem id
- 2ª opção / lista sem id

Seletor CSS

- Principais tipos de seletores:

class

Seletor CSS

■ Seletor de *class* .

- Ao contrário do atributo ID:
 - o atributo *class* pode ser partilhada por múltiplos elementos
- Um elemento pode ter definida mais de uma *class*
 - Sem limite, separadas por espaço, sendo indiferente a ordem pela qual são definidas.

class

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li {color:gray;}
    .opc {color:orange;}
  </style>
</head>
<body>
  <ul id="lista1">
    <li class="opc">1ª opção / lista id="lista1" </li>
    <li class="opc">2ª opção / lista id="lista1" </li>
  </ul>
  <ul>
    <li class="opc">1ª opção / lista sem id </li>
    <li>2ª opção / lista sem id </li>
  </ul>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"
- 1ª opção / lista sem id
- 2ª opção / lista sem id

Seletor CSS

■ Seletor de *class* .

- *class* aplicada/definida com base num elemento específico

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li {color:gray;}
    p.opc {color:orange;}
  </style>
</head>
<body>
  <ul id="lista1">
    <li class="opc">1ª opção / lista id="lista1" </li>
    <li class="opc">2ª opção / lista id="lista1" </li>
  </ul>
  <ul>
    <li class="opc">1ª opção / lista sem id </li>
    <li>2ª opção / lista sem id </li>
  </ul>
  <p class="opc">1º Parágrafo</p>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"
- 1ª opção / lista sem id
- 2ª opção / lista sem id

1º Parágrafo

Seletor CSS

- Seletor de *class*.
 - Podem ser utilizados para criar um seletor contextual
 - São formatados os elementos integrados num outro elemento cujo atributo *class* foi definido com um valor específico.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li {color:gray;}
    .lst .opc {color:orange;}
  </style>
</head>
<body>
  <ul id="lista1">
    <li class="opc">1ª opção / lista id="lista1" </li>
    <li class="opc">2ª opção / lista id="lista1" </li>
  </ul>
  <ol class="lst">
    <li class="opc">1ª opção / lista sem id </li>
    <li>2ª opção / lista sem id </li>
  </ol>
  <p class="opc">1º Parágrafo</p>
</body>
</html>
```

- 1ª opção / lista id="lista1"
- 2ª opção / lista id="lista1"

1. 1ª opção / lista sem id
2. 2ª opção / lista sem id

1º Parágrafo

Seletor CSS

- **Importante!**
 - Especificação do elemento e seletor **id**

h1#d1{color:gray}

Formatação aplicada ao elemento **h1** cujo **id** é **d1**

≠

h1 #d1

- Especificação do elemento e seletor **class**

h2.c1{color:green}

Formatação aplicada ao elemento **h2** cuja **class** é **c1**

≠

h2 .c1

O espaço em branco permite implementar seletores totalmente diferentes

Seletor CSS

```
...  
<style>  
  h2.c1{color:orange}  
</style>  
  
</head>  
  
<body>  
  <div id="div1" class="c1">  
    <h1 id="d1">heading 1</h1>  
    <h2 class="c1">heading 2</h2>  
    <p class="c1">paragraph1</p>  
  </div>  
  <h2 class="c1">Other heading 2</h2>  
...
```

```
...  
<style>  
  .c1 h2{color:orange}  
</style>  
  
</head>  
  
<body>  
  <div id="div1" class="c1">  
    <h1 id="d1">heading 1</h1>  
    <h2 class="c1">heading 2</h2>  
    <p class="c1">paragraph1</p>  
  </div>  
  <h2 class="c1">Other heading 2</h2>  
...
```

Formatação aplicada aos elementos h2
cuja **class** é **c1**

heading 1
heading 2
paragraph1
Other heading 2

≠

A ordem dos elementos / espaço
em branco é determinante para
uma correta implementação de um
selector.

Formatação aplicada
aos elementos h2
definidos **no**
contexto de um
outro elemento cuja
class é **c1**

heading 1
heading 2
paragraph1
Other heading 2

Seletor CSS

- Principais tipos de seletores:

Atributo

Seletor CSS

■ Seletor de Atributo

■ [attribute]

Atributo

- Caso o elemento não seja especificado são afetados todos os elementos que possuem o atributo definido

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    [id] {color:orange;}
  </style>
</head>
<body>
  <p id="primeiro">1º Parágrafo</p>
  <p> 2º Parágrafo</p>
  <p id="terceiro">3º Parágrafo</p>
  <p> 4º Parágrafo</p>

  <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo
2º Parágrafo
3º Parágrafo
4º Parágrafo
• primeira opção

Seletor CSS

■ Attribute Selectors

- Referenciam um elemento através dos seus atributos ou dos valores que assumem

■ *element [attribute]*

- referencia os elementos especificados que têm declarado um atributo específico.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    p[id] {color:orange;}
  </style>
</head>
<body>
  <p id="primeiro">1º Parágrafo</p>
  <p> 2º Parágrafo</p>
  <p id="terceiro">3º Parágrafo</p>
  <p> 4º Parágrafo</p>

  <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo
2º Parágrafo
3º Parágrafo
4º Parágrafo
• primeira opção

Seletor CSS

■ ***element* [attribute="value"] / [atribute="value"]**

- referencia os elementos *element* cujo atributo assume um valor *específico*

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    [id="lista">{color:orange;}
  </style>
</head>
<body>
  <p id="primeiro">1º Parágrafo</p>
  <p> 2º Parágrafo</p>
  <p id="terceiro">3º Parágrafo</p>
  <p> 4º Parágrafo</p>

  <ul id="lista"><li>primeira opção</li></ul>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

4º Parágrafo

- primeira opção

- Existem algumas variantes de seletor de atributo (menos utilizados) que selecionam os elementos a formatar com base em correspondências parciais dos valores dos atributos.

Seletor CSS

■ ***Attribute Selectors***

■ ***element* [attribute *= " ..."] / [atribute *= " ..."]**

- referencia os elementos cujos valores do atributo especificado contêm uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    p[id*="paragrafo"] {color:orange;}
  </style>
</head>
<body>
  <p id="paragrafo-primeiro">1º Parágrafo</p>
  <p id="segundo paragrafo">2º Parágrafo</p>
  <p id="paragrafo terceiro">3º Parágrafo</p>
  <p id="quarto">4º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

4º Parágrafo

Seletor CSS

■ Attribute Selectors

■ **element** [attribute ^= "..."] / [attribute ^= "..."]

- referencia os elementos cujo valor do atributo **se inicia** com uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    → p[id^="paragrafo"] {color:orange;}
  </style>
</head>
<body>
  <p id="paragrafo-primeiro">1º Parágrafo</p>
  <p id="segundo paragrafo">2º Parágrafo</p>
  <p id="paragrafo terceiro">3º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

Seletor CSS

■ Attribute Selectors

■ **element** [attribute \$= "..."] / [attribute \$= "..."]

- referencia os elementos cujo valor do atributo **termina com** uma dada **expressão** (palavra isolada ou inserida em outra palavra)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    → p[id$="paragrafo"] {color:orange;}
  </style>
</head>
<body>
  <p id="paragrafo-primeiro">1º Parágrafo</p>
  <p id="segundo paragrafo">2º Parágrafo</p>
  <p id="paragrafo terceiro">3º Parágrafo</p>
</body>
</html>
```

1º Parágrafo

2º Parágrafo

3º Parágrafo

- Lista completa: http://www.w3schools.com/cssref/css_selectors.asp

```
<body>  
  <p class="cl1"> Parágrafo Exemplo <span> Elemento Inline </span></p>  
</body>
```

<style>

p {color:red;}

Parágrafo Exemplo Elemento Inline

span {color:blue;}

Parágrafo Exemplo Elemento Inline

p span {color:green;}

Parágrafo Exemplo Elemento Inline

p span {color:green;}

Parágrafo Exemplo Elemento Inline

p .cl1 {color:orange;}

Parágrafo Exemplo Elemento Inline

p.cl1 {color:orange;}

Parágrafo Exemplo Elemento Inline

</style>

Pseudo-Class Selectors

Pseudo-Class Selector

- Baseados no **estado** do elemento
 - *:link*
 - link não visitado
 - *:visited*
 - link anteriormente visitado
 - *:hover*
 - o cursor do rato encontra-se sobre o elemento
 - *:active*
 - o elemento está a ser selecionado
 - *:focus*
 - elemento selecionado e pronto para receber valores
 - *element:first-child*
 - elemento que é o primeiro filho do seu pai (**exceção!**)

Pseudo-Class Selector

■ Pseudo-Class selector (exemplo)

■ *:hover*

- Sobreposição do cursor do rato

Pseudo-Class: estado link

Pseudo-Class: **estado link**

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    a:hover {color:white;background-color:#FF6600;font-size:20px;
  </style>
</head>
<body>
  <p>Pseudo-Class: <a href="http://www.isec.pt">estado link</a></p>
</body>
</html>
```

Pseudo-Class Selector

■ Pseudo-Class selector (exemplo)

■ *li:first-child*

- seleciona o elemento que é o primeiro filho do seu pai

```
<!DOCTYPE html>

<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li:first-child {color:orange;font-size:20px}
  </style>
</head>
<body>
  <ul>
    <li>Primeiro filho de ul</li>
    <li>Segundo filho de ul</li>
    <li>Último filho de ul</li>
  </ul>
  <p> primeiro parágrafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parágrafo

Seletor *sempre aplicado* no elemento filho

Pseudo-Class Selector

■ *li:last-child*

- seleciona o elemento que é o último filho do seu pai

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li:last-child {color:orange;font-size:20px}
  </style>
</head>
<body>
  <ul>
    <li>Primeiro filho de ul</li>
    <li>Segundo filho de ul</li>
    <li>Último filho de ul</li>
  </ul>
  <p> primeiro parágrafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parágrafo

Pseudo-Class Selector

▪ `:first-child` ; `:last-child`

```
<!DOCTYPE html>
<html lang="">
<head>
  <meta charset="utf-8">
  <title></title>

  <style>
    div:first-child{color:orange}
    div:last-child{color:lightblue}
  </style>
</head>
<body>
  <div> Primeiro elemento div</div>
  <div> Ultimo elemento div</div>
</body>
</html>
```

Primeiro elemento div

Ultimo elemento div

Alteração da
estrutura
mantendo o
CSS

```
<body>
  <p>primeiro filho</p>
  <div> Primeiro elemento div</div>
  <div> Ultimo elemento div</div>
  <p>ultimo filho</p>
</body>
```

?

primeiro filho

Primeiro elemento div

Ultimo elemento div

ultimo filho

A formatação não é aplicada, uma vez que a alteração da estrutura originou **que os elementos <div> deixassem de ser o primeiro e o ultimo filho do seu pai** (neste caso o element <body>)

Seletor CSS

▪ `li:nth-child(n)`

- seleciona o elemento que é o enésimo filho

```
<!DOCTYPE html>

<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    li:nth-child(2) {color:orange;font-size:20px}
  </style>
</head>
<body>
  <ul>
    <li>Primeiro filho de ul</li>
    <li>Segundo filho de ul</li>
    <li>Último filho de ul</li>
  </ul>
  <p> primeiro parágrafo </p>
</body>
</html>
```

- Primeiro filho de ul
- Segundo filho de ul
- Último filho de ul

primeiro parágrafo

Pseudo-Class Selector

- `:first-of-type` ; `:last-of-type`; `:nth-of-type(n)`

```
<style>
  div:first-of-type{color:orange}
  div:last-of-type{color:lightblue}
</style>

</head>
<body>
  <p>primeiro filho</p>
  <div>primeiro elemento</div>
  <div>ultimo elemento</div>
  <p>ultimo filho</p>
```

primeiro filho

primeiro elemento

ultimo elemento

ultimo filho

- Apesar de na maioria das situações produzirem resultados idênticos, os seletores:

- `:first-of-type` e `:first-child`
- `:last-of-type` e `:last-child`
- `:nth-child(n)` e `:nth-of-type(n)`

não são equivalentes!

Seletor CSS

- Formulários
 - *Pseudo Class Selectors* **específicos** para formulários

<i>seletor</i>	<i>Exemplo</i>	<i>Observações</i>
<code>:checked</code>	<code>input:checked</code>	seleciona todos os input checked
<code>:disabled</code>	<code>input:disabled</code>	seleciona todos os input que estão inativos
<code>:enabled</code>	<code>input:enable</code>	seleciona todos os input que estão ativos
<code>:focus</code>	<code>input:focus</code>	seleciona todos os input que “ganham” <i>focus</i>
<code>:invalid</code>	<code>input:invalid</code>	seleciona todos os input inválidos
<code>:optional</code>	<code>input:optional</code>	seleciona todos os input não obrigatórios
<code>:required</code>	<code>input:required</code>	seleciona todos os input obrigatórios

- *Pseudo Class Selectors* específicos para formulários

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    input:disabled { background-color:lightcoral}
    input:focus {background-color:lightgoldenrodyellow}
    input:invalid {background-color:lightgray}
    input:required {background-color:lightgreen}
  </style>
</head>
<body>
  <form>
    <input type="text" size="30" placeholder="First Name" disabled/> <br/><br/>
    <input type="text" size="30" placeholder="Last Name" autofocus/><br/><br/>
    <input type="email" size="30" placeholder="Email"/><br/><br/>
    <input type="password" size="30" placeholder="Password" required/><br/><br/>
    <input type="submit" value="sign up" />
  </form>
</body>
</html>
```

First Name

Last Name

teste

Password

sign up

- Lista completa de *pseudo class selectors*: http://www.w3schools.com/cssref/css_selectors.asp

Pseudo-Elements Selectors

Seletor CSS

- Referem-se a elementos fictícios (não correspondem a elementos HTML), os quais são baseados na estrutura do documento.
- Utilizam uma notação “::” diferente da pseudo-classe “:”
 - ::first-line**
 - ::first-letter**
 - ::before**
 - Com base na propriedade **content** permite inserir conteúdo antes de um elemento
 - ::after**
 - Com base na propriedade **content** permite inserir conteúdo depois de um elemento
 - ::selection**
 - Parte de um elemento que é seleccionada pelo utilizador.

Seletor CSS

- ::before ::after + propriedade content**

```
** ANTES ** Pseudo elements (::before;::after) ** DEPOIS **
```

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    p {color:gray;}
    span {color:orange;}
    p::before {content:"** ANTES **"}
    p::after {content:"** DEPOIS **"}
  </style>
</head>
<body>
  <p><span>Pseudo elements (::before;::after)</span></p>
</body>
</html>
```

■ **::selection**

- permite formatar o conteúdo que está a ser selecionado pelo utilizador

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>

  <style>
    ::selection {background-color:orange;color:white; }
  </style>
</head>
<body>
  <p><span> Formatação diferente para uma seleção de texto </span></p>
</body>
</html>
```

Formatação diferente para uma seleção de texto

Conflitos de Formatação

CSS - Conflitos de Formatação

- Tipos de Conflito
 - Seletores iguais
 - Ordem pela qual são definidos
 - Seletores Diferentes
 - Especificidade
 - Herança

CSS - Conflitos de Formatação

- Seletores iguais
 - Ordem pela qual são definidos
 - *last one listed wins*: **prevalece a ultima definição** para **um determinado seletor**

```
<!DOCTYPE HTML>
<html>
<head>
  <meta charset="utf-8" />
  <title> External Style Sheet</title>
  <style>
    h1{color:#F00;} /*red*/
    h1{color:#00F;} /*blue*/
  </style>
</head>

<body>
  <h1> Resolver Conflitos de Formatação </h1>
</body>
</html>
```

?

Resolver Conflitos de Formatação

CSS - Conflitos de Formatação

■ Seletores iguais

■ Ordem pela qual são definidos

- Um ficheiro externo foi incorporado **depois** da definição de um *embedded style*,

```
<style> ... </style>
<link ...>
```

em caso de conflito **para o mesmo seletor**, prevalece a formatação estabelecida no **ficheiro externo**.

- Um ficheiro externo foi incorporado **antes** da definição de um *embedded style*,

```
<link ...>
<style> ... </style>
```

em caso de conflito **para o mesmo seletor**, prevalece a formatação estabelecida no ***embedded style***.

CSS - Conflitos de Formatação

■ Conflitos

■ Seletores Iguais: Prioridade ?

- Critério: “Ordem pela qual são definidos”

1. Regra assinalada como **!important** [mais prioritário]

```
<style>
  h1{color:#F00 !important;}
</style>
```

!important

Prevalece sobre todas as formatações
É a **exceção** à regra da “ordem pela qual são definidos”

2. *Inline style* (atributo **style** na *opening tag*)

3. *Embedded Style Sheet* (**<style>...</style>**)

4. *External Style Sheet* (**<link .../>; @import**)

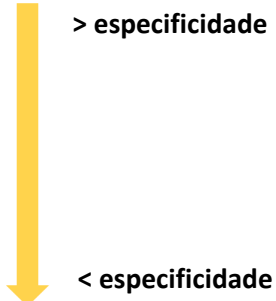
5. Definições por defeito do *browser* [menos prioritário]

Considera que o *embedded style* é declarado **após** a ligação ao ficheiro externo; caso contrário prevaleceria a declaração no ficheiro externo.

CSS - Conflitos de Formatação

■ Especificidade

- Os seletores **mais específicos tem mais peso** na resolução de conflitos de formatação:

1. ID Selectors (mais específico / mais prioritário)
 2. Class Selectors; Attribute Selectors
 3. Contextual Selectors (apenas com elementos)
 4. Individual Element Selectors
- 

- Entre seletores com **a mesma especificidade prevalece** a ultima declaração.

CSS - Conflitos de Formatação

■ Especificidade

- O browser **atribui pesos específicos** de acordo com o tipo de seletor
- Como calcular?
 - **d**: seletores de elemento e pseudo-elemento (**1 ponto**)
 - **c**: seletores de class, atributo e *pseudo-class* (**10 pontos**)
 - **b**: seletor de ID (**100 pontos**)
 - **a**: *inline style* (**1000 pontos**)

```
2 <style>  
  body h1{color:red;}  
1 h1{color:blue;}  
  </style>  
  </head>  
  
  <body>  
    <h1> Resolução de conflitos de formatação </h1>
```

Resolução de conflitos de formatação

CSS - Conflitos de Formatação

■ Conflitos de Formatação - Prioridades na aplicação de estilos

■ Especificidade

- Como calcular?

a	b	c	d

```
<style>
13  body div.especifica h1{color:darkolivegreen}
2   body h1{color:red}
1   h1 {color:blue}
</style>
</head>
<body>
  <div class="especifica">
    <h1 id="maisEspecifico">Resolução Conflitos de Formatação </h1>
  </div>
```

Resolução Conflitos de Formatação

CSS - Conflitos de Formatação

■ Conflitos de Formatação - Prioridades na aplicação de estilos

■ Especificidade

a	b	c	d

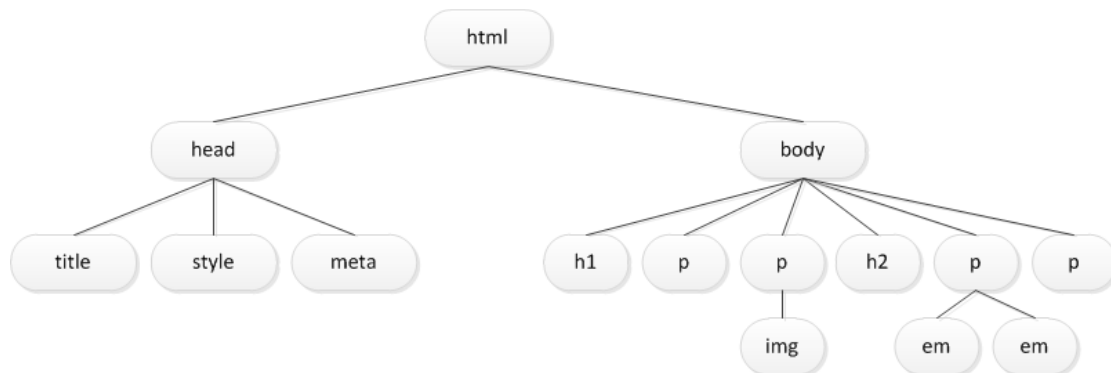
```
<style>
100 → #maisEspecifico{color:darkorange}
      body div.especifica h1{color:darkolivegreen}
      body h1{color:red}
      h1 {color:blue}
</style>
</head>
<body>
  <div class="especifica">
    <h1 id="maisEspecifico">Resolução Conflitos de Formatação </h1>
  </div>
```

Resolução Conflitos de Formatação

CSS - Conflitos de Formatação

■ Herança

- Documentos HTML tem uma estrutura hierárquica implícita



- body head filhos de html
- h1 p em h2 img descendentes de body
- ...

CSS - Conflitos de Formatação

■ Herança

- **Algumas** propriedades de **alguns** elementos HTML são herdadas
 - Geralmente as propriedades relacionadas com o estilo do texto são herdadas

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    p {color:gray;}
  </style>
</head>
<body>
  <p> Conceito Herança <span> formatação herdada de p </span></p>
</body>
</html>
```

Conceito Herança formatação herdada de p

O elemento é formatado por herança uma vez que não foi formatado diretamente.
Na realidade, apenas foi formatado o elemento pai <p>.

■ Herança

- Só é aplicada **caso não seja definido** o estilo do elemento.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    .p1 {color:gray;}
  </style>
</head>
<body>
  <p class="p1"> Conceito Herança <span> formatação herdada de p </span></p>
</body>
</html>
```

Conceito Herança formatação herdada de p

- Caso o elemento seja diretamente formatado, a herança não se aplica: **prevalece sempre a formatação direta do elemento**

Conceito Herança formatação herdada de p

```
<style>
  .p1 {color:gray;}
  span {color:orange;}
</style>
```

- Este funcionamento aplica-se sempre, mesmo que o seletor aplicado ao elemento tenha menor especificidade do que a especificidade do seletor aplicado ao elemento pai

Exercício

Exame Época Normal 21/22

Exercicio (Exame Época Normal 21/22)

```
<body>
  <div>
    <p> Client <span>Side<span>Web Technologies </span></span></p>
    <ul>
      <li class="item">HTML</li>
      <li>CSS</li>
      <li>Java<span>Script</span></li>
    </ul>
  </div>
</body>
```

Exercicio (Exame Época Normal 21/22)

```
<body>
  <div>
    <p> Client <span>Side<span>Web Technologies </span></span></p>
    <ul>
      <li class="item">HTML</li>
      <li>CSS</li>
      <li>Java<span>Script</span></li>
    </ul>
  </div>
</body>
```

Unidades CSS

Unidades CSS

■ Unidades Relativas

■ São baseadas no tamanho de outro elemento/referência

- Não pode existir um espaço em branco entre o valor e a unidade
- Se o valor for 0 a unidade pode ser omitida



em

Relativa ao font-size
definido para a fonte
default: 16px

rem

Relativa ao font-size
definido para a fonte do
root element <html>
default: 16px

%

Relativa às
dimensões do
elemento pai

vmin

Relativa a 1% da
menor dimensão do
viewport (mobile)

vmax

Relativa a 1% da
maior dimensão do
viewport (mobile)

vw

Relativa a 1% da
largura do viewport
(mobile)

vh

Relativa a 1% da
altura do viewport
(mobile)

■ CSS pixel

- É uma unidade abstrata, tudo o que é definido em **px** em CSS baseia-se no CSS pixel

- exemplo: {font-size:16px;}

*The **px** unit is defined **to be small but visible, and such that a horizontal 1px wide line can be displayed with sharp edges**. What is sharp, small and visible depends on the device and the way it is used: do you hold it close to your eyes, like a mobile phone, at arms length, like a computer monitor, or somewhere in between, like an e-book reader?*

- O CSS pixel pode ou não coincidir com os pixels físicos (dpi)

- **Device pixel ratio**: informação de que o browser necessita para determinar quantos pixels físicos são utilizados para desenhar um único CSS pixel.

Modelo	Resolução Física (px)	CSS pixels	Device Pixel
iPhone 14 Pro	1179 x 2556	393 x 852	3
iPhone 14 Pro Max	1290 x 2796	430 x 932	3



<http://blog.popupdesign.com.br/desenvolvimento-responsivo-e-viewport/>

Box Model

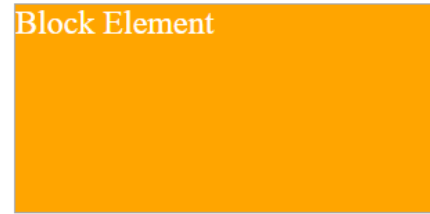
Propriedades

Box Model

- Os elementos HTML (*inline/block*) são interpretados pelo *browser* como estando contidos em caixas rectangulares, às quais podem ser aplicadas propriedades.

```
div{
  background-color: orange;
  color:white;
  font-size: 1.5em;
  width:300px;
  height:150px;
  border:darkgray 1px solid;
}
</style>
</head>
<body>
  <div> Block Element </div>
```

Block Element



Box Model

- content area*
 - área reservada ao conteúdo
- padding area*
 - área definida entre a área de conteúdo e o border (**opcional**)
- border*
 - linha que envolve a content area e a padding área (**opcional**)
- margin*
 - área adicionada no exterior do border (**opcional**)



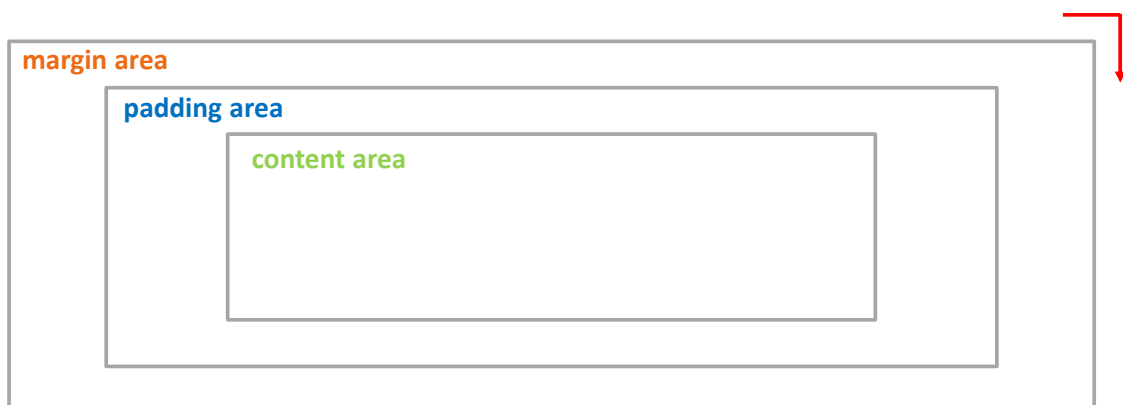
■ Propriedades

Propriedade	Exemplo	Observações
padding <i>padding-top</i> <i>padding-right</i> <i>padding-left</i> <i>padding-bottom</i>	<code>p {padding:2em;}</code>	Permite definir a <i>padding area</i> . Valores: <i>length measurement</i> , <i>percentage</i> , <i>auto</i> , <i>inherit</i> .
margin <i>margin-top</i> <i>margin-right</i> <i>margin-left</i> <i>margin-bottom</i>	<code>p {margin:2em;}</code>	Permite definir a <i>margin area</i> . Valores: <i>length measurement</i> , <i>percentage</i> , <i>auto</i> , <i>inherit</i> . Nota importante: a inserção de valores negativos conduz geralmente à sobreposição dos elementos.

Box Model

■ padding

- `seletor{padding:5px;}` 1 valor (top/bottom/right/left)
- `seletor{padding:5px 10px;}` 2 valores (top/bottom right/left)
- `seletor{padding:5px 10px 15px 5px;}` 4 valores (top right bottom left)

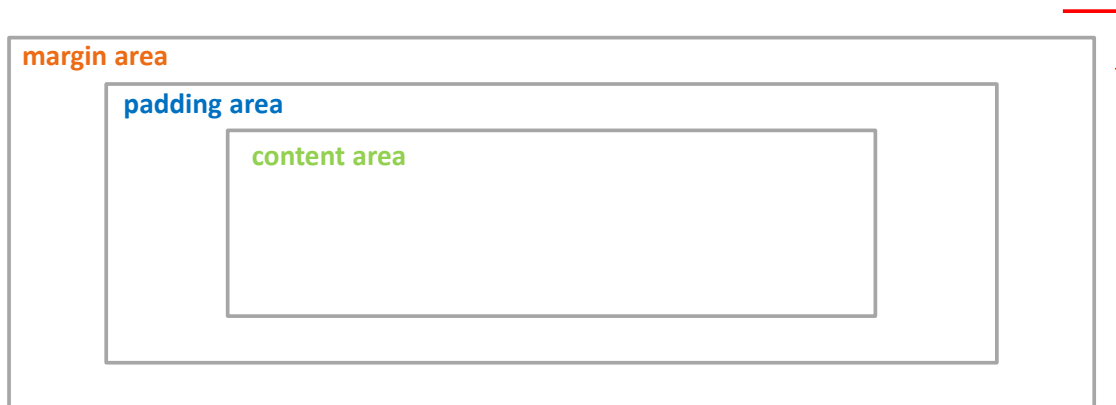


- *padding-top; padding-right; padding-bottom; padding-left*

Box Model

■ margin

- `seletor{margin:5px;}` 1 valor (top/bottom/right/left)
- `seletor{margin:5px 10px;}` 2 valores (top/bottom right/left)
- `seletor{margin:5px 10px 15px 5px;}` 4 valores (top right bottom left)



- `margin-top;` `margin-right;` `margin-bottom;` `margin-left`

Box Model

■ *content area*

- *width; height*
 - definem as dimensões da *content area*

```
div{
    background-color: orange;
    color:white;
    font-size: 1.5em;
    width:300px;
    height:150px;
    border:darkgray 1px solid;
}
</style>
</head>
<body>
    <div> Block Element </div>
```

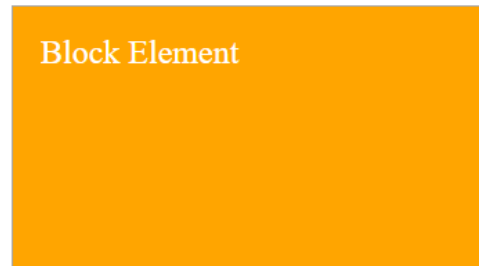
Block Element



Box Model

- padding area
 - padding: top right bottom left;*
 - as dimensões globais resultam do somatório da área de conteúdo e de espaçamento interno

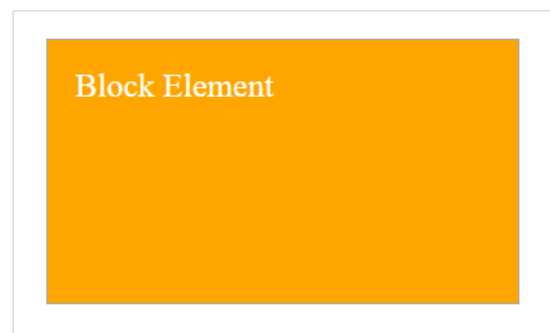
```
div{
  background-color: orange;
  color:white;
  font-size: 1.5em;
  width:300px;
  height:150px;
  padding:20px;
  border:darkgray 1px solid;
}
</style>
</head>
<body>
  <div> Block Element </div>
```



Box Model

- margin area
 - margin: top right bottom left;*
 - estabelece as dimensões da *margin area* (área livre em redor do elemento)

```
div{
  background-color: orange;
  color:white;
  font-size: 1.5em;
  width:300px;
  height:150px;
  padding:20px;
  margin:20px;
  border:darkgray 1px solid;
}
</style>
</head>
<body>
  <div> Block Element </div>
```



box model

■ Dimensões da área visível (**box-sizing**)

■ content-box

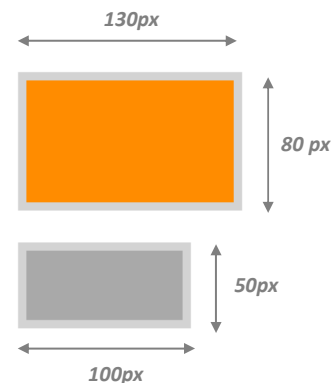
- Valor por default, os valores de width/height referem-se à área de conteúdo

■ border-box

- content area + padding area + border

```
#d1{width:100px;
    height:50px;
    background-color:darkorange;
    border: 5px lightgray solid;
    padding:10px; }
```

```
#d2{width:100px;
    height:50px;
    background-color:darkgray;
    border: 5px lightgray solid;
    padding:10px;
    box-sizing:border-box}
```



Box Model

■ border

Propriedade	Exemplo	Observações
border-style <i>border-top-style</i> <i>border-right-style</i> <i>border-left-style</i> <i>border-bottom-style</i>	<i>p {border-style: solid;}</i>	Permite escolher o estilo do border. Valores: <i>none*</i> , <i>dotted</i> , <i>dashed</i> , <i>solid</i> , <i>double</i> , <i>inset</i> (sombra interior), <i>outset</i> (sombra exterior) ...
border-width <i>border-top-width</i> <i>border-right-width</i> <i>border-left-width</i> <i>border-bottom-width</i>	<i>p {border-style: solid;</i> <i>border-width: 4px;}</i>	Permite definir a largura do border. Valores: <i>length units</i> , <i>thin</i> , <i>medium*</i> , <i>thick</i> , <i>inherit</i>
border-color <i>border-top-color</i> <i>border-right-color</i> <i>border-left-color</i> <i>border-bottom-color</i>	<i>p {border-style: solid;</i> <i>border-width: 4px;</i> <i>border-color: red;}</i>	Definição da cor do border. Valores: <i>color name/RGB value</i> , <i>transparent</i> , <i>inherit</i>
border <i>border-top</i> <i>border-right</i> <i>border-left</i> <i>border-bottom</i>	<i>p{border: 4px solid red;}</i>	Propriedade agregada (width, style, color) Mais frequentemente utilizada.

Box Model

■ overflow

Propriedade	Exemplo	Observações
overflow	<code>p{overflow:scroll;}</code>	Valores: <i>visible, hidden, scroll, ...</i>

```
p{width:200px;
  height:100px;
  border: solid 3px lightsalmon;
}
</style>
</head>
<body>
```

```
<p> Propriedade Width permite definir a largura de um elemento.
Propriedade Width permite definir a largura de um elemento
Propriedade Width permite definir a largura de um elemento
Propriedade Width permite definir a largura de um elemento
Propriedade Width permite definir a largura de um elemento
</p>
```

Propriedade Width permite definir a largura de um elemento. Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento

CSS – Box Model

```
p{width:200px;
  height:100px;
  border: solid 3px lightsalmon;
  overflow:hidden
}
```

Propriedade Width permite definir a largura de um elemento. Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento

Propriedade Width permite definir a largura de um elemento. Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento Propriedade Width permite definir a largura de um elemento

```
p{width:200px;
  height:100px;
  border: solid 3px lightsalmon;
  overflow:scroll
}
```

Propriedade Width permite definir a largura de um elemento. Propriedade Width permite definir a largura de um elemento

■ Cantos arredondados

- `border-radius: valor;`
 - O mesmo valor aplica-se a todos os cantos
- `border-radius: valor1 valor2;`
 - `valor1`: canto superior direito/inferior esquerdo
 - `valor2`: canto inferior direito/superior esquerdo
- `border-radius: valor1 valor2 valor3 valor4;`
 - `valor1` canto superior esquerdo (sentido horário)
- Propriedades individuais:
 - `border-top-left-radius`
 - `border-top-right-radius`
 - `border-bottom-right-radius`
 - `border-bottom-left-radius`

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p{background-color:darkorange;
      color:white;
      width:200px;
      border:darkorange 2px solid;
      border-radius:4px;}
  </style>
</head>
<body>
  <p>BORDER RADIUS</p>
</body>
</html>
```

BORDER RADIUS

■ Efeito Sombra (box)

- `{box-shadow: h-shadow v-shadow blur (opt.) spread (opt.) color (opt.) inset (opt.);}`

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #d1 {width:100px;height:50px;
      background-color:darkorange;
      color:white;
      border:5px lightgray solid;
      padding:10px;
      box-shadow:lightgray 10px 10px 5px;}
  </style>
</head>
<body>
  <div id="d1">Box Shadow</div><br/>
</body>
</html>
```

Box Shadow

Propriedades

display

Block-level element

- A sua posição original é definida pelo fluxo do HTML
- Provoca uma quebra de linha
- Expande-se naturalmente para ocupar o seu *container*
 - A largura do elemento pode ser controlada (*width*)
 - Permite a definição de *margins* e/ou *padding*
- Se não for especificada a altura (*height*) o elemento expande-se para englobar os seus elementos filho
- Exemplos:
 - `<p>`, `<h1>`, `<div>`, `<form>`, `<ul>`, `<li>`, ...



<https://dev.to/akshayjaagirdhar/differences-between-html-inline-and-blocks-elements-43d7>

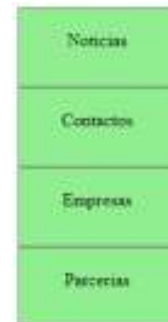
*{display:**}*

- seletor{display: **block**}
- Define o elemento como um **block element**
- Útil para a implementação de menus verticais

```
<a href="">Noticias</a>  
<a href="">Contactos</a>  
<a href="">Empresas</a>  
<a href="">Parcerias</a>
```

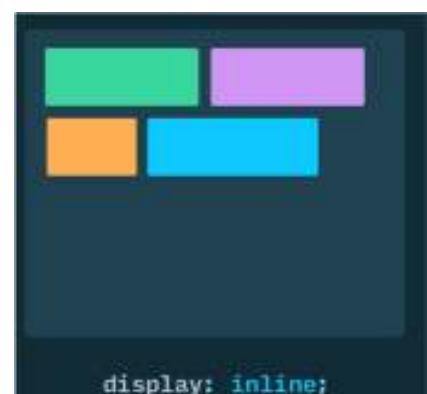
[Noticias](#) [Contactos](#) [Empresa](#) [Parcerias](#)

```
a{display:block;  
width:100px;  
height:30px;  
text-decoration:none;  
color:black;  
text-align:center;  
padding:20px;  
border:gray solid 1px;  
background-color: lightgreen;}
```



Inline-level Element

- Aplica-se a conteúdo textual
 - **Não se expande** à totalidade do espaço disponível
 - Container/viewport
- **Ignora** as propriedades:
 - largura e altura (width/height)
 - margens topo/fundo (*top/bottom*)
- Permite a definição de:
 - *left/right margin*
 - *padding*.
 - alinhamento vertical (*vertical-align*)
- Exemplos:
 - `<a>`, `<span>`, ...



{display:***}

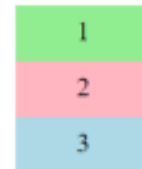
■ seletor{display:inline}

- Define o elemento como um **inline element**
- Muito útil para a implementação de menus horizontais (*horizontal navigation bar*)

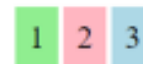
```
div{width:100px;
    height: 30px;
    font-size: 1.5em;
    padding:10px;
    text-align: center;}

#d1{background-color:lightgreen}
#d2{background-color:lightpink}
#d3{background-color:lightblue}
</style>
</head>
<body>

<div id="d1">1</div>
<div id="d2">2</div>
<div id="d3">3</div>
```



div{display:inline}

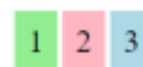


{display:***}

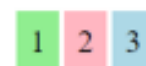
■ seletor{display:inline-block}

- preserva o comportamento inline mas com a possibilidade de definir largura altura do elemento

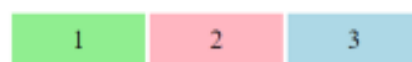
div{display:inline}



```
div{display:inline;
    width:200px;
    height:50px;}
```



```
div{display:inline-block;
    width:200px;
    height:50px;}
```



{display:***}

■ *display*

- propriedade muito importante nomeadamente na conceção de menus de navegação

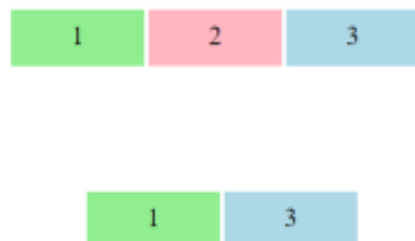
Propriedade	Exemplo	Observações
display	li {display:inline;}	Permite definir a disposição de elementos. Valores: none; inline; block; inline-block; flex; grid;...

{display:***}

■ `seletor{display:none}`

- O elemento não é visualizado **nem tem qualquer** influência no layout

```
div{display:inline-block;  
width:200px;  
height:50px;}  
#d2{display:none}
```



- lista completa dos valores propriedade ***display***

- http://www.w3schools.com/cssref/pr_class_display.asp

Exercício Seletores

Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element  
<span>parágrafo1</span></span></p>  
  
<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>  
  
<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element  
<span>parágrafo1</span></span></p>  
  
<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>  
  
<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

Exercício Seletores

```
<p class="p1" >Parágrafo 1<span class="s1">inline element  
<span>parágrafo1</span></span></p>  
  
<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>  
  
<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

```
<p class="p1" >Parágrafo 1<span class="s1">inline element  
<span>parágrafo1</span></span></p>  
  
<p id="p2">Parágrafo 2<span>inline element parágrafo 2</span></p>  
  
<p>Parágrafo 3<span>inline element parágrafo 3</span> </p>
```

Layouts

Tipos

- É possível agrupar **layouts** da seguinte forma:

fluid layout
flexible layout

As dimensões são estabelecidas de forma a **existir um redimensionamento** dos conteúdos quando as dimensões do viewport são alteradas

Versatilidade!

Flexibilidade!

Mobile!

fixed layout

As dimensões são especificadas em px **sem atender** às dimensões do viewport ou do tamanho do texto

Deprecated!

Layouts

Tipos

Abordagens

■ CSS FlexBox

- Baseia-se num conjunto de propriedades que permitem a disposição flexível dos elementos ao longo de um eixo.

■ CSS Grid Layout

- Baseia-se na criação de um grid container para disposição dos elementos por linha/coluna
- Suportado pelos browsers (sem vendor prefix) a partir de 2017

■ Frameworks CSS

- Forma rápida e poderosa de criação de um layout (exemplo:Bootstrap)
- Aprendizagem de um novo framework

■ *float /clear* (deprecated!)

- Implementação simples. Ligado ao fluxo do HTML o que reduz a sua flexibilidade

CSS Flexbox (Layouts)

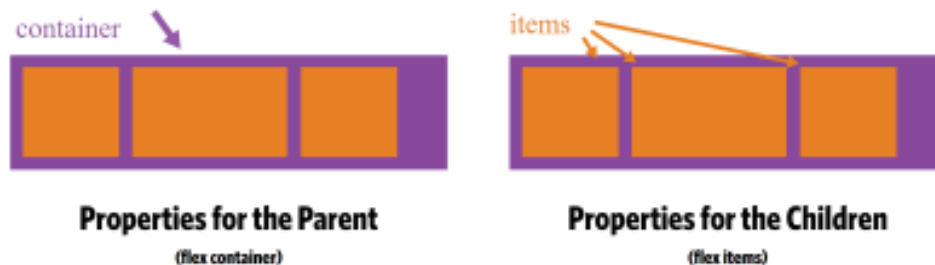
CSS Flexbox

■ flexbox container

- permite agilizar a definição de layouts

According to the specification, the Flexbox model provides for an efficient way to **layout, align, and distribute space among elements within your document** — even when the viewport and the size of your elements is dynamic or unknown.

<https://medium.freecodecamp.org/understanding-flexbox-everything-you-need-to-know-b4013d4dc9af>



■ {display:flex}

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

- cria um *container* flexível com um conjunto de propriedades para o *container* e para os respetivos *items*

CSS Flexbox (layout)

- Pode ser aplicado a diversos elementos
 - ul -> flex container
 - li -> flex item (filhos do flex container)

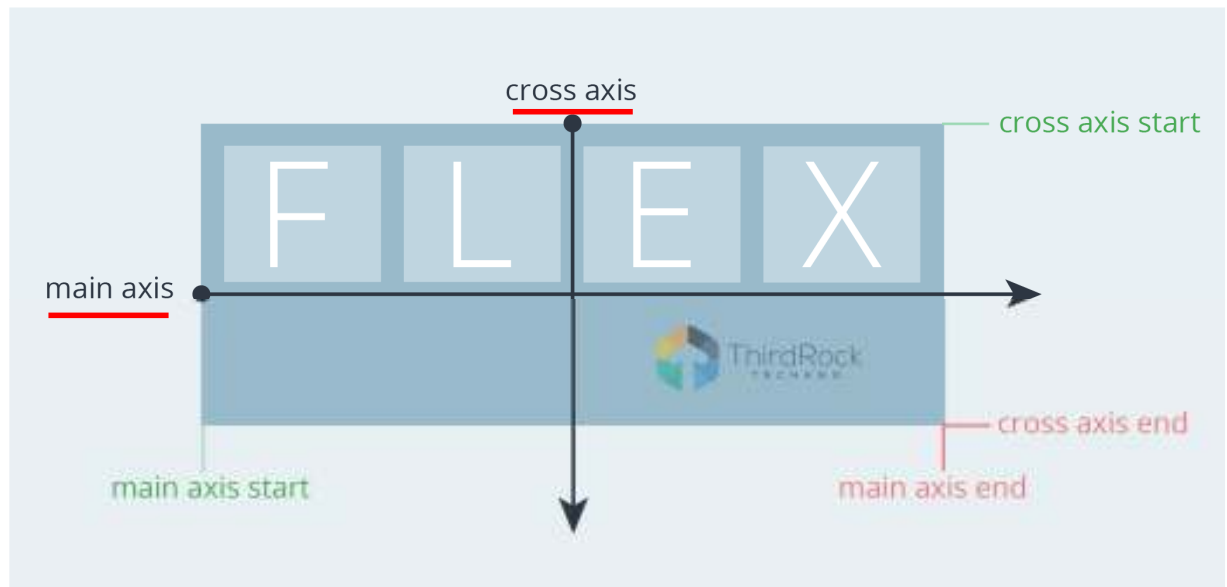
```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
</ul>
```

```
ul{display:flex;}

li {  width: 100px;
      height: 100px;
      background-color:lightblue;
      margin:8px;

      list-style-type:none;
      text-align:center;
      color:white}
```





<https://www.thirdrocktechkno.com/blog/how-to-improve-css-layout-with-flex/>

CSS Flexbox (Layouts)

flex container

flex item

■ Propriedades do **flex container**:

■ **flex-direction**

- Estabelece a direção do eixo principal ao longo do qual os elementos são dispostos

■ **flex-wrap**

- Define se o *container* adapta a sua dimensão ou se cria múltiplas linhas para conter os *flex items*

■ **flex-flow**

- propriedade agregada de *flex-direction* e *flex-wrap* (ex: `flex-flow: row wrap;`)

■ **align-items**

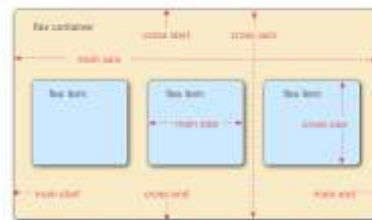
- define o alinhamento dos *items* ao longo do *cross axis*

■ **align-content**

- define o alinhamento dos *items* ao longo do *cross axis* quando existe mais do que uma linha

■ **justify-content**

- define o alinhamento dos *items* ao longo do *main axis*



■ Propriedades dos **flex container**

■ **flex-direction**: row | row-reverse | column | column-reverse



■ **flex-wrap**: nowrap | wrap | wrap-reverse

- O comportamento por defeito é nowrap, o flex container adapta-se aumentando de tamanho para conter todos os flex items
- O valor wrap cria múltiplas linhas

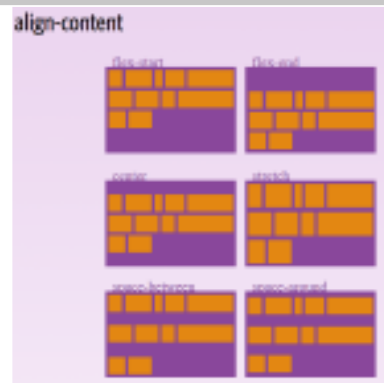


■ **align-items**: flex-start | flex-end | center | space-between | space-around | stretch;



CSS Flexbox (layout)

- **align-content:** flex-start | flex-end | center | space-between | space-around | stretch;
 - **Não se aplica** quando existe apenas uma linha de *items*



- **justify-content:** flex-start | flex-end | center | space-between | space-around | space-evenly;



<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

CSS Flexbox (layout)

Exemplo:

```
<div id="container">
  <div id="d1">Item1 Item1 Item1 Item1 Item1 Item1 Item1 </div>
  <div id="d2">Item2 Item2 Item2 Item2 Item2 Item2 Item2 </div>
  <div id="d3">Item3 Item3 Item3 Item3 Item3 Item3 Item3 </div>
</div>
```

```
#container{
  height:400px;
  width:400px;
  display:flex;
  flex-direction: row;
  align-items:flex-start;}
```

Item1	Item1	Item1	Item2	Item2	Item2	Item3	Item3	Item3
Item1	Item1	Item1	Item2	Item2	Item2	Item3	Item3	Item3
Item1			Item2			Item3		

- Na definição do *container* flexível (**display:flex**):
 - Conteúdos apresentados por linha (**flex-direction:row**)
 - Alinhamento dos *items* seria feito a partir do topo (**align-items:flex-start**)

Os elementos aproveitam a largura disponível (400px), são dispostos com a mesma largura e pela ordem que surgem no HTML

CSS Flexbox (layout)

```
<div id="container">
  <div id="d1">Item1 Item1 Item1 Item1 Item1 Item1 Item1 </div>
  <div id="d2">Item2 Item2 Item2 Item2 Item2 Item2 Item2 </div>
  <div id="d3">Item3 Item3 Item3 Item3 Item3 Item3 Item3 </div>
</div>
```

```
#d1{background-color: orange;}
#d2{background-color: lightgray;}
#d3{background-color: lightblue;}
```

```
#container{
  height:400px;
  width:800px;
  border: 1px gray solid;
  display:flex;
  flex-direction: column;
  align-items:flex-end;
}
```

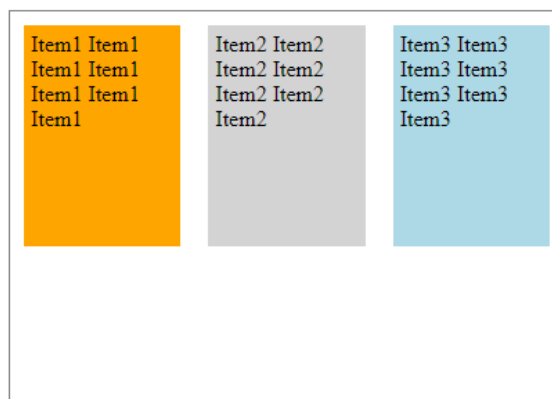
```
#container div{
  height:100px;
  margin:0px;
  padding:0px;
  width: 400px;}
```

- Mesmos elementos (estrutura) mas:
 - Conteúdos apresentados por coluna (***flex-direction:column***)
 - Alinhamento dos *items* a partir do final (***align-items:flex-end***)

CSS Flexbox (layout)

- O flex container permite que sejam aplicadas propriedades aos flex items:
 - Exemplo: neste caso podem ser definidos o *padding*, a *margin* e a *height* de cada flex item

```
#container div{
  height:150px;
  margin:10px;
  padding:5px;
}
```



CSS Flexbox (Layouts)

flex container

flex item

CSS Flexbox (layout)

■ Propriedades dos **flex items**

■ **order**

- permite reordenar os *flex items* num *container*

■ **flex-grow**

- estabelece quanto é que um item pode crescer se existir espaço livre no *flex container*

■ **flex-shrink**

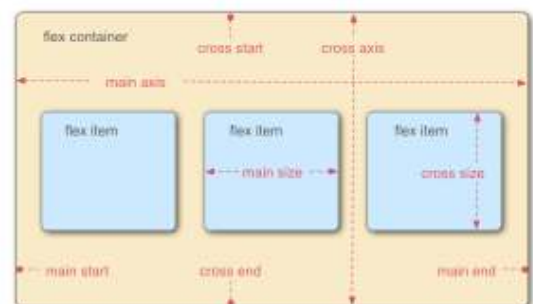
- estabelece quanto é que um item pode reduzir caso não exista espaço no *flex container*

■ **flex-basis**

- define a dimensão inicial de um *flex item*

■ **flex**

- propriedade agregada (*flex*: *flex-grow flex-shrink flex-basis*)



■ Propriedades para os *flex items*:

■ {*order*: <integer>;}

- Por defeito a ordem dos *items* segue a ordem do HTML, sendo que todos os *flex items* possuem o valor 0 (*order*; *default value*).
- É possível controlar a posição de um item controlando o valor desta propriedade
 - um valor negativo coloca o *item* em primeiro lugar
 - um valor positivo posiciona o item depois de todos aqueles com um valor de *order* inferior.

```
<div id="container">
  <div id="d1">Item1...</div>
  <div id="d2">Item2...</div>
  <div id="d3">Item3...</div>
</div>
```

```
#container div{
  height:150px;
  margin:10px;
  padding:5px;
}
```

```
#d2{order: -1;}
```

```
#d2{order: 1;}
```

■ {*flex-grow*: <integer>;}

- Estabelece a capacidade de um item para crescer
- Aceita um valor que define a quantidade de **espaço disponível** que cada um dos *items* vai ocupar.
 - Se todos os *items* possuírem o valor 1 o espaço disponível é dividido de igual forma pelos diversos items
 - Caso contrário o item com o maior valor vai ocupar mais espaço do que os outros *items*.

```
#container{
  height:400px;
  width:800px;
  border: 1px gray solid;
  display:flex;
  flex-direction: row;
  align-items:flex-start;}

#container div{ width:150px;
  height:150px;
  margin:10px;
  padding:5px;}
```

Se largura/altura inicial dos diversos items é diferente, então as dimensões finais também serão diferentes apesar do espaço disponível ser distribuído de igual forma

```
#container div{flex-grow: 1;}
```

CSS Flexbox (layout)

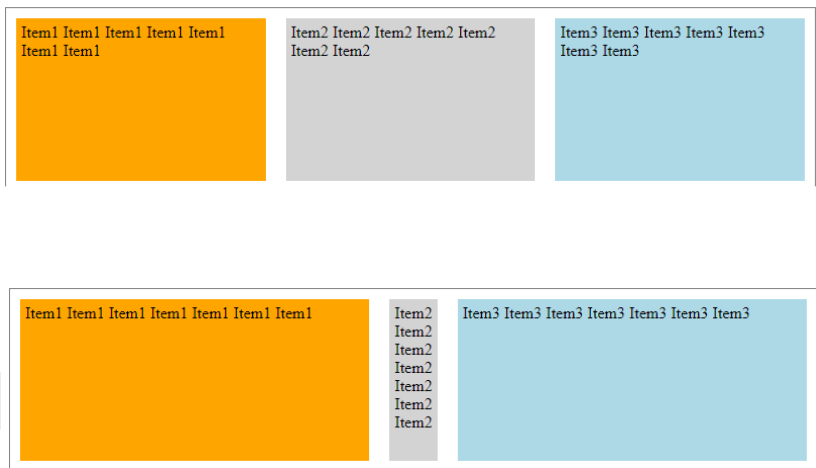
- `{flex-shrink: <integer>;}`
 - Capacidade de um item para reduzir a sua dimensão, o valor inteiro define o grau de redução atribuído a esse elemento.
 - Esta propriedade só se aplica em situação de **overflow** (largura items > largura do container)

```
#container{  
  height:400px;  
  width:800px;  
  border: 1px gray solid;  
  display:flex;  
  flex-direction: row;  
  align-items:flex-start;}
```

```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  flex-grow: 1;  
  flex-shrink: 1;  
  width: 400px;}
```

```
#container #d2{flex-shrink:6;}
```

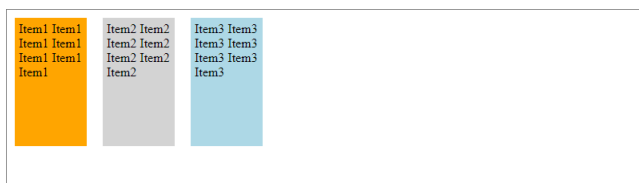
Como a largura está definida e existe uma situação de *overflow*, a dimensão dos *flex-items* irá ajustar-se à largura do *flex container*



CSS Flexbox (layout)

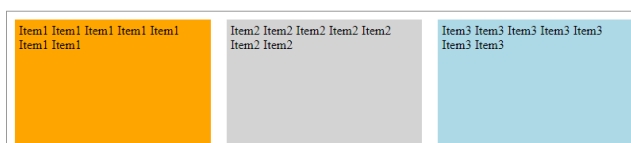
- `{flex-basis: <length> | auto;}`
 - Define a dimensão (% , em, ...) inicial de um flex item.

```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  /*flex-grow: 1;*/  
}  
  
#container div{flex-basis:10%;}
```



- Se *flex-grow* definido o espaço livre é repartido pelos *flex-items*

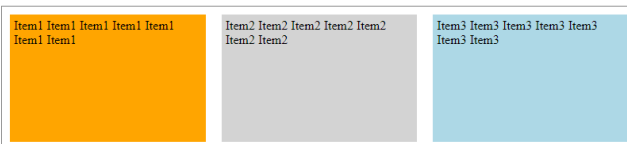
```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  flex-grow: 1;  
}  
  
#container div{flex-basis:10%;}
```



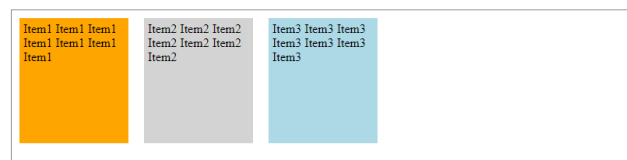
CSS Flexbox (layout)

- {flex-basis: <length> | auto;}
 - Se a largura do elemento não se encontrar definida, o valor **auto** ajusta ao conteúdo de cada um dos flex items
 - Caso a largura do *flex-item* tenha sido definida (*width*), {*flex-basis:auto*} não altera essa definição.

```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  /* flex-grow: 1; */  
  /* width: 130px;*/  
  
#container div{flex-basis:auto;}
```



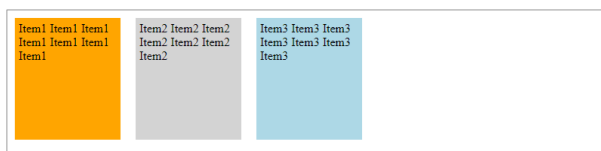
```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  /* flex-grow: 1; */  
  width: 130px;}  
  
#container div{flex-basis:auto;}
```



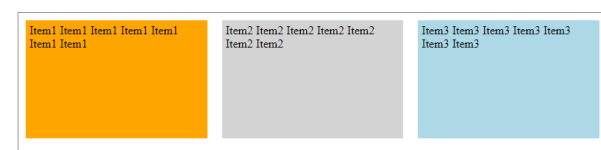
CSS Flexbox (layout)

- {flex: 0 1 auto}
- propriedade agregada para os valores *flex-grow*, *flex-shrink* (opcional) e *flex-basis* (opcional) .
- Os valores por defeito são (0 1 auto).

```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  width: 130px;}  
  
#container div{flex:0 1 auto;}
```

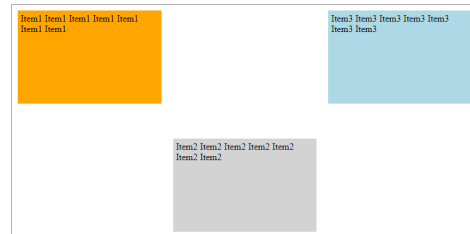


```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  width: 130px;}  
  
#container div{flex:1 1 auto;}
```



- **align-self**: auto || flex-start || flex-end || center || baseline || stretch
 - Permite o alinhamento de um *flex item*, mantendo o alinhamento previamente definido para os restantes *flex items*

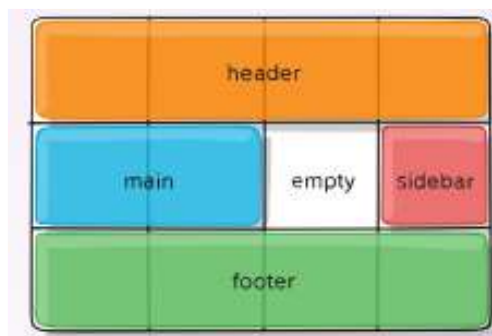
```
#container div{  
  height:150px;  
  margin:10px;  
  padding:5px;  
  width: 130px;}  
  
#container div{flex:1 1 auto;}  
  
#d2{align-self:flex-end}
```



CSS Grid Layout

CSS Grid Layout

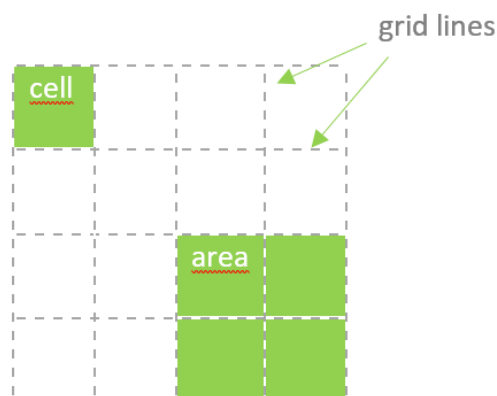
- A criação de um *grid container* é efetuada através de `{display: grid}`
 - Deve ser estabelecido o número de linhas/colunas do grid container
 - Todos os filhos **diretos** de um *grid container* tornam-se *grid items*
 - Associar cada *grid item* a uma área no *grid container*
 - pode ser criada uma grid dentro de uma outra grid e assim criar vários contextos de posicionamento



<https://css-tricks.com/snippets/css/complete-guide-grid/>

CSS Grid Layout

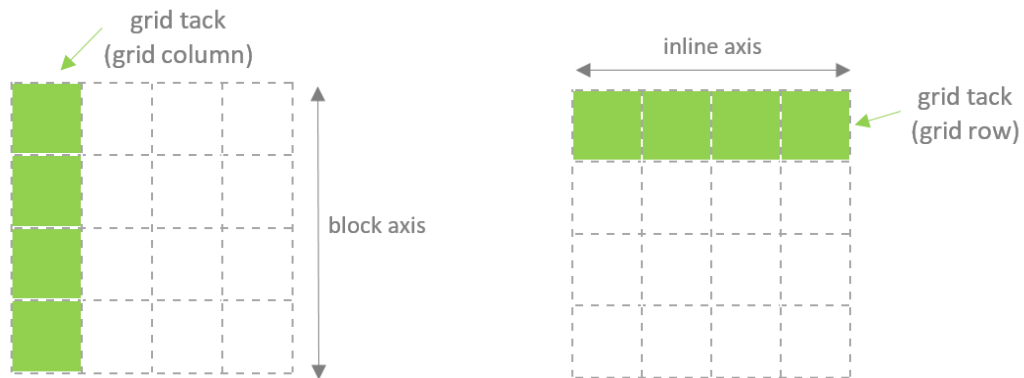
- CSS Grid
 - *line*
 - linhas horizontais e verticais que constituem a *grid*
 - *cell*
 - unidade mais pequena de uma *grid* a qual é delimitada por 4 *grid lines* adjacentes
 - *area*
 - área retangular constituída por uma ou mais *grid cells* adjacentes



■ CSS Grid

■ track

- o espaço entre células adjacentes é um *grid track*, o qual pode ser um *grid column* ou *grid row*
 - *grid column* é vertical (*block axis*)
 - *grid row* é horizontal (*inline axis*)



CSS Grid Layout

Grid Container

display: grid
grid-template-rows
grid-template-columns
grid-template-areas

Definição de um Grid Container

- A declaração de um CSS grid é feito através da propriedade **display:grid**
 - **grid-template-rows**; **grid-template-columns** definem n.º e dimensões de linhas/colunas

```
#layout{display: grid;
      grid-template-rows: 100px 400px 100px;
      grid-template-columns: 200px 500px 200px;}
</style>
</head>
<body>

<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
</div>
```

Definidas 3 linhas e 3 colunas.
O *template* pode ser dividido no número de linhas e colunas definidos pelos valores atribuídos a **grid-template-rows**; **grid-template-columns**

Definição de um Grid Container

- A propriedade **grid-template-areas** permite identificar as várias *grid cells*
 - Importante!
 - Trata-se apenas da identificação das células uma vez que a grid e as respetivas dimensões das grid tracks são definidas através das propriedades **grid-template-rows**, **grid-template-columns**

```
#layout{display: grid;
      grid-template-rows: 100px 400px 100px;
      grid-template-columns: 200px 500px 200px;
      grid-template-areas:
        "header header header"
        "ads main links"
        "footer footer footer"}
</style>
</head>
<body>

<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
  <div id="six">Six</div>
</div>
```

Células são referenciadas por **linha**

header	header	header
ads	main	links
footer	footer	footer

- Existe ainda a propriedade **grid** que agrega as propriedades:
 - grid-template-rows
 - grid-template-columns
 - grid-template-areas
- não é muito utilizada uma vez que origina uma sintaxe mais complexa do que a utilização das propriedades individuais:

```
#layout{display: grid;  
  grid:  
    "header header header" 100px  
    "ads main links"       400px  
    "footer footer footer" 100px  
    / 200px 500px 200px  
}
```

Especificação feita por **linha**
/ dimensão das colunas

CSS Grid Layout

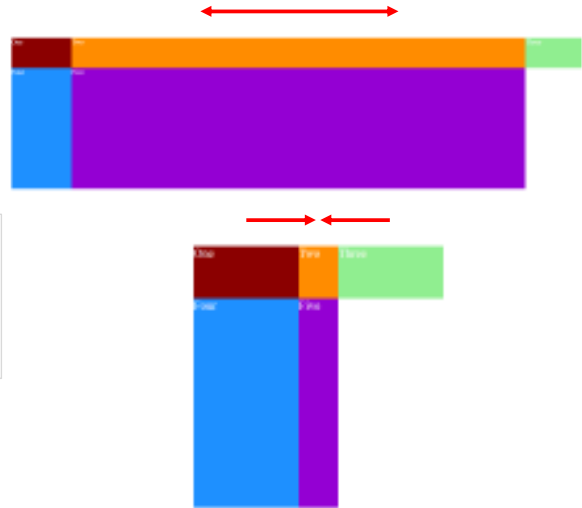
Grid Container

Unidades

Unidades

- Na CSS grid as unidades a utilizar devem assegurar a flexibilidade necessária para um *responsive web design* (diversidade de dispositivos)
 - px (fixed); %
 - fr (*fractional unit*)
 - permite que os grid tracks se ajustem ao espaço disponível, expandam/retraiam de acordo com as variações do *viewport*.

```
#layout{  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 1fr 200px}
```

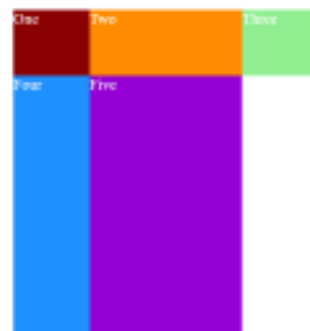


Unidades

- As *fractional units* podem ser usadas para:
 - combinar áreas de dimensão variável com dimensão fixa (exemplo anterior)
 - combinar áreas de dimensão variável

```
#layout{  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 1fr 2fr 1fr}
```

A coluna central terá sempre o dobro da capacidade de ajustamento das colunas dos extremos



- A função `minmax()` permite limitar as dimensões de uma grid track
 - tornar essa *grid track* flexível mas dentro de limites pré-definidos

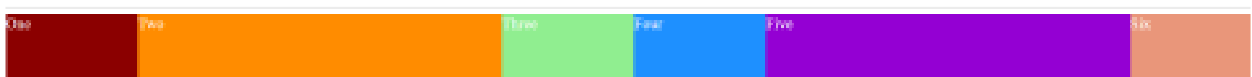
```
#layout{
  display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px minmax(5em,15em) 200px}
```

- As dimensões de uma grid track também podem ser definidas com base nas dimensões do seu próprio conteúdo
 - `min-content`
 - o menor valor para não criar situações de overflow (ex: a palavra mais longa; a imagem mais larga, ...)
 - `max-content`
 - o valor máximo que permita conter o conteúdo sem wrapping.
 - **Atenção!** Não é adequado para parágrafos, o efeito pode ser imprevisível.

- As dimensões da grid podem ser definidas tirando partido de:
 - `repeat (number_of_repetitions, pattern)`

```
#six{background-color:darksalmon}

#layout{display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: repeat(2, 200px 1fr 200px)}
</style>
</head>
<body>
  <div id="layout">
    <div id="one">One</div>
    <div id="two">Two</div>
    <div id="three">Three</div>
    <div id="four">Four</div>
    <div id="five">Five</div>
    <div id="six">Six</div>
  </div>
```



CSS Grid Layout

Grid Container

Unidades

Grid Items

Posicionamento Grid Items

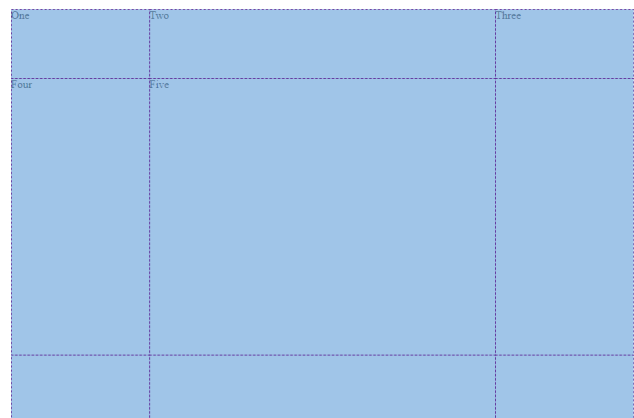
- Cada um dos *grid items* é atribuído de forma automática a cada uma das *grid cells*
 - por defeito essa atribuição é sequencial e feita por linha

```
#layout{display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px 500px 200px;}

div{color:white; font-size: 1.2em}
#one{background-color:darkred}
#two{background-color:darkorange}
#three{background-color:lightgreen}
#four{background-color:dodgerblue}
#five{background-color:darkviolet}

</style>
</head>
<body>

  <div id="layout">
    <div id="one">One</div>
    <div id="two">Two</div>
    <div id="three">Three</div>
    <div id="four">Four</div>
    <div id="five">Five</div>
  </div>
```



CSS Grid Layout

Grid Container

Unidades

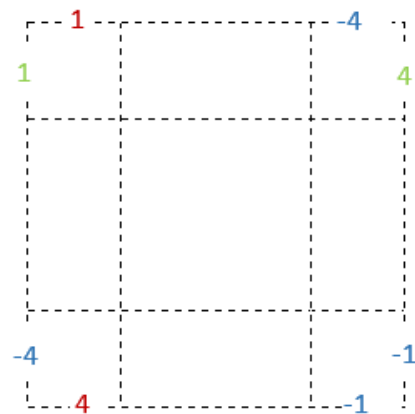
Grid Items : Posicionamento

Grid Items: Posicionamento

■ Identificação das linhas

■ Identificação numérica

- row line (1 – 4)
- column line (1-4)
- Contagem Inversa
 - a contagem inicia-se na ultima linha sempre com o valor -1 e processa-se de forma decrescente (-2, -3, ...)



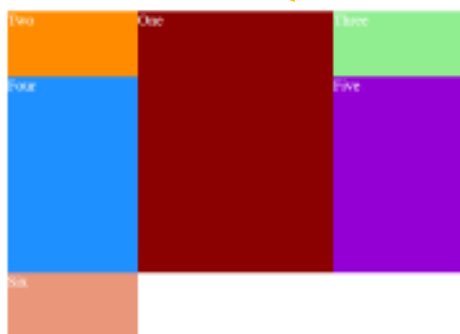
■ Identificação baseada em nomes [...]

```
#layout{display: grid;
  grid-template-rows: [header-start] 100px [content-start] 400px [footer-start] 100px;
  grid-template-columns: [adds] 200px [main] 500px [links] 200px}
```


Grid Items: Posicionamento

- A alteração do posicionamento é efetuado a partir da identificação das linhas tendo por base as propriedades:

- grid-row-start
- grid-row-end
- grid-column-start
- grid-column-end



```
#layout{display: grid;
  grid-template-rows: 100px 300px 100px;
  grid-template-columns: 200px 300px 200px;
  grid-template-areas:
    "header header header"
    "ads main links"
    "footer footer footer"
}

#one{
  grid-row-start: 1;
  grid-row-end:3;
  grid-column-start: 2;
  grid-column-end: 3;
}
```

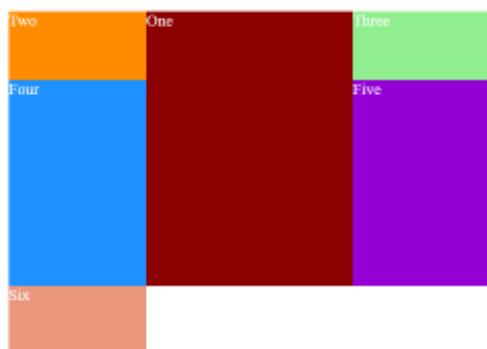
```
</style>
</head>
<body>
  <div id="layout">
    <div id="one">One</div>
```

O item é posicionado, sendo que todos os outros reajustam de acordo com o espaço disponível

O posicionamento é feito pelo número/nome da linha

Grid Items: Posicionamento

- O posicionamento anterior pode ser efetuado de forma mais condensada:
- grid-row: start line / end line
- grid-column: start line / end line



```
#layout{display: grid;
  grid-template-rows: 100px 300px 100px;
  grid-template-columns: 200px 300px 200px;
  grid-template-areas:
    "header header header"
    "ads main links"
    "footer footer footer"
}

#one{
  grid-row: 1 / 3;
  grid-column: 2 / 3;
}
```

```
</style>
</head>
<body>
  <div id="layout">
    <div id="one">One</div>
    <div id="two">Two</div>
    <div id="three">Three</div>
    <div id="four">Four</div>
    <div id="five">Five</div>
    <div id="six">Six</div>
  </div>
```

Grid Items: Posicionamento

- O posicionamento através de grid-area também pode ser feito com base na designação das áreas criadas através de **grid-template-areas**

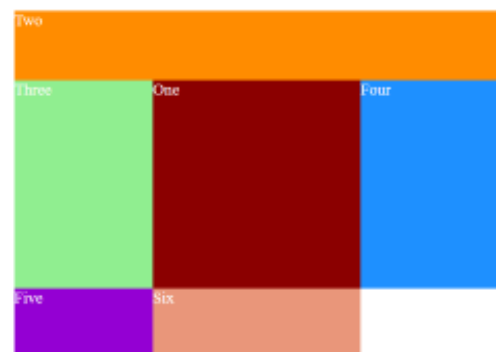
```
#layout{display: grid;
  grid-template-rows: 100px 300px 100px;
  grid-template-columns: 200px 300px 200px;
  grid-template-areas:
    "header header header"
    "ads main links"
    "footer footer footer"
}

#one{
  grid-area: main;
}

#two{
  grid-area: header;
}

</style>
</head>
<body>
  <div id="layout">
    <div id="one">One</div>
    <div id="two">Two</div>
    <div id="three">Three</div>
```

header	header	header
ads	main	links
footer	footer	footer



Grid Items: Posicionamento

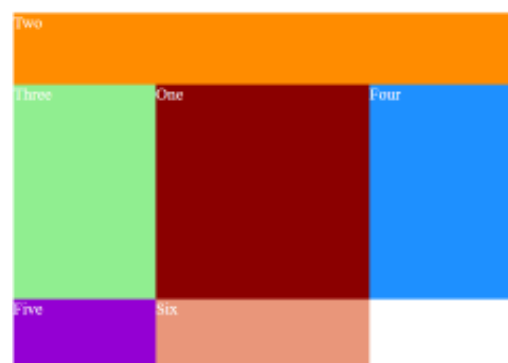
- A propriedade **grid-area** pode também ser aplicada com base em 4 grid lines as quais definem uma área:
 - `grid-area: row-start / column start / row-end / column-end;`

```
#one{
  grid-area: main;
}

#two{
  grid-area: 1 / 1 / 2 / 4;
}

</style>
</head>
<body>
  <div id="layout">
    <div id="one">One</div>
    <div id="two">Two</div>
    <div id="three">Three</div>
```

header	header	header
ads	main	links
footer	footer	footer



Grid Items: Posicionamento

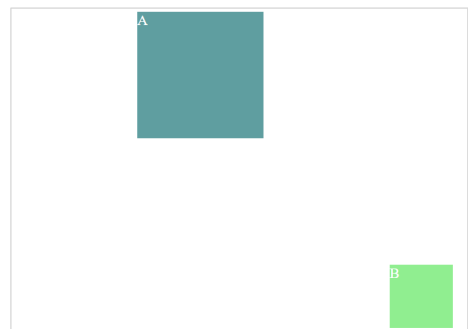
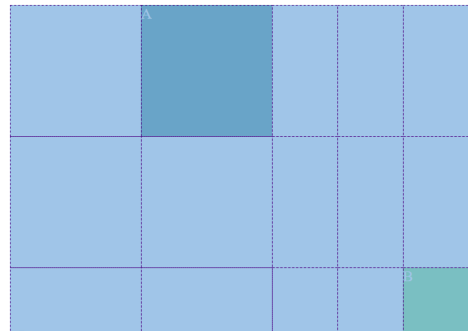
- Se o posicionamento for definido **fora da grid inicial** podem ser geradas automaticamente linhas/colunas para esse posicionamento ser possível
 - grid-auto-rows** e **grid-auto-columns** utilizadas para geração automática de *tracks* e aplicar à grid prédefinida

```
div{color:white; font-size: 1.2em}
#littlegrid{
  display: grid;
  grid-template-columns: 200px 200px;
  grid-template-rows: 200px 200px;
  grid-auto-columns: 100px;
  grid-auto-rows: 100px }

#a{
  grid-row: 1 / 2;
  grid-column: 2 / 3;
  background-color: cadetblue;}

#b{
  grid-row: 3 / 4;
  grid-column: 5 / 6;
  background-color: lightgreen; }

</style>
</head>
<body>
  <div id="littlegrid">
    <div id="a">A</div>
    <div id="b">B</div>
  </div>
```



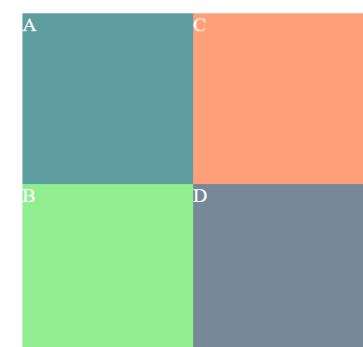
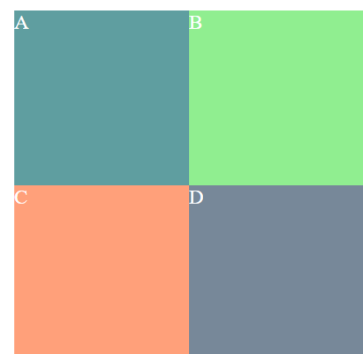
Grid Items: Posicionamento

- A propriedade **grid-auto-flow** permite estabelecer a direção do posicionamento (*row (default); column*)

```
div{color:white; font-size: 1.2em}
#littlegrid{
  display: grid;
  grid-auto-flow: column;
  grid-template-columns: 200px 200px;
  grid-template-rows: 200px 200px;
  grid-auto-columns: 100px;
  grid-auto-rows: 100px }

#a{background-color: cadetblue;}
#b{background-color: lightgreen; }
#c{background-color: lightsalmon}
#d{ background-color: lightslategrey }

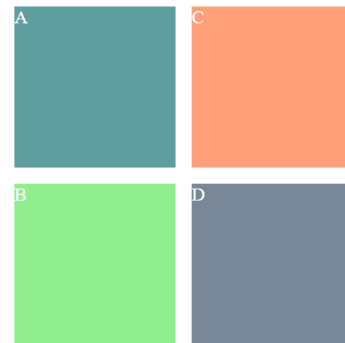
</style>
</head>
<body>
  <div id="littlegrid">
    <div id="a">A</div>
    <div id="b">B</div>
    <div id="c">C</div>
    <div id="d">D</div>
  </div>
```



Grid Items: Posicionamento

- As propriedades **grid-row-gap/grid-column-gap** permitem estabelecer um espaço entre tracks
 - **grid-gap** permite agregar estes dois valores por linha e coluna

```
#littlegrid{
  display: grid;
  grid-auto-flow: column;
  grid-gap: 20px;
  grid-template-columns: 200px 200px;
  grid-template-rows: 200px 200px;
  grid-auto-columns: 100px;
  grid-auto-rows: 100px
}
```



Grid Items: Posicionamento

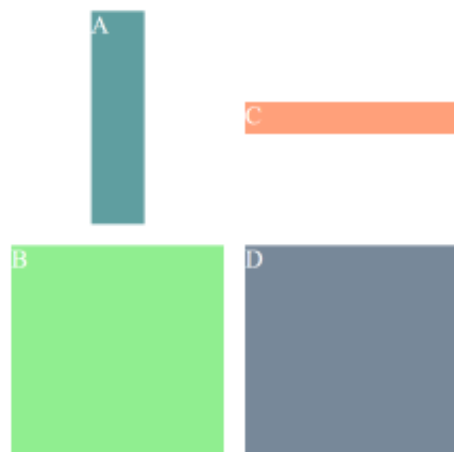
- Quando os itens possuem dimensão inferior à grid que foi criada, podem ser alinhados (posicionados) tendo por base as propriedades:
 - **justify-self** (alinhamento horizontal)
 - **align-self** (alinhamento vertical)

```
#littlegrid{
  display: grid;
  grid-auto-flow: column;
  grid-gap: 20px;
  grid-template-columns: 200px 200px;
  grid-template-rows: 200px 200px;
  grid-auto-columns: 100px;
  grid-auto-rows: 100px }

#a{background-color: cadetblue;
  width:50px;
  justify-self: center;}

#b{background-color:lightgreen; }

#c{background-color:lightsalmon;
  height:30px;
  align-self:center}
```

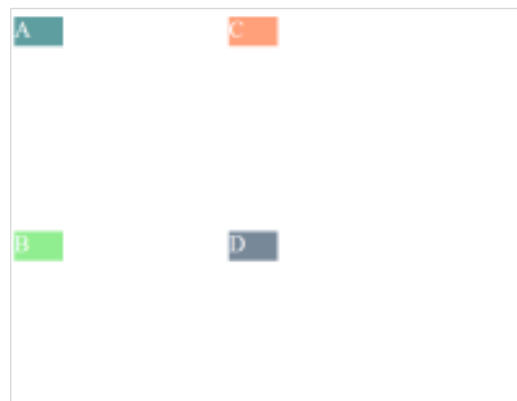


Grid Items: Posicionamento

- A propriedade ***justify-items*** permite o alinhamento horizontal para todos os items

```
div{color:white; font-size: 1.2em;
width:50px;
height:30px}

#littlegrid{
display: grid;
grid-auto-flow: column;
justify-items: start;
grid-gap: 20px;
grid-template-columns: 200px 200px;
grid-template-rows: 200px 200px;}
```



```
#littlegrid{
display: grid;
grid-auto-flow: column;
justify-items: end;
grid-gap: 20px;
...}
```



Grid Items: Posicionamento

- A propriedade ***align-items*** permite o alinhamento vertical para todos os items

```
div{color:white; font-size: 1.2em;
width:50px;
height:30px}

#littlegrid{
display: grid;
grid-auto-flow: column;
align-items: start;
grid-gap: 20px;
grid-template-columns: 200px 200px;
grid-template-rows: 200px 200px}
```



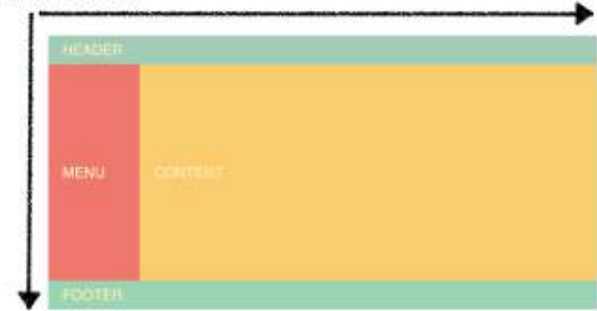
```
#littlegrid{
display: grid;
grid-auto-flow: column;
align-items: end;
...}
```



One dimension



Two dimensions



Flexbox – 1D

Mais flexível
Menos código
Mais fácil de manter

CSS Grid – 2D

Podem ser combinadas!
(ex: disposição de elementos no header/footer)

<https://hackernoon.com/the-ultimate-css-battle-grid-vs-flexbox-d40da0449faf>

Framework CSS

layouts

■ Bootstrapp

- Instalação diretamente a partir de um *Content Delivery Network (CDN)*



<https://9official.com/2017/08/28/content-delivery-network-cdn/>

- Download rápido (possibilidade de *pre-cached files*) assim como controlo de versões

- <https://www.sitepoint.com/7-reasons-to-use-a-cdn/>

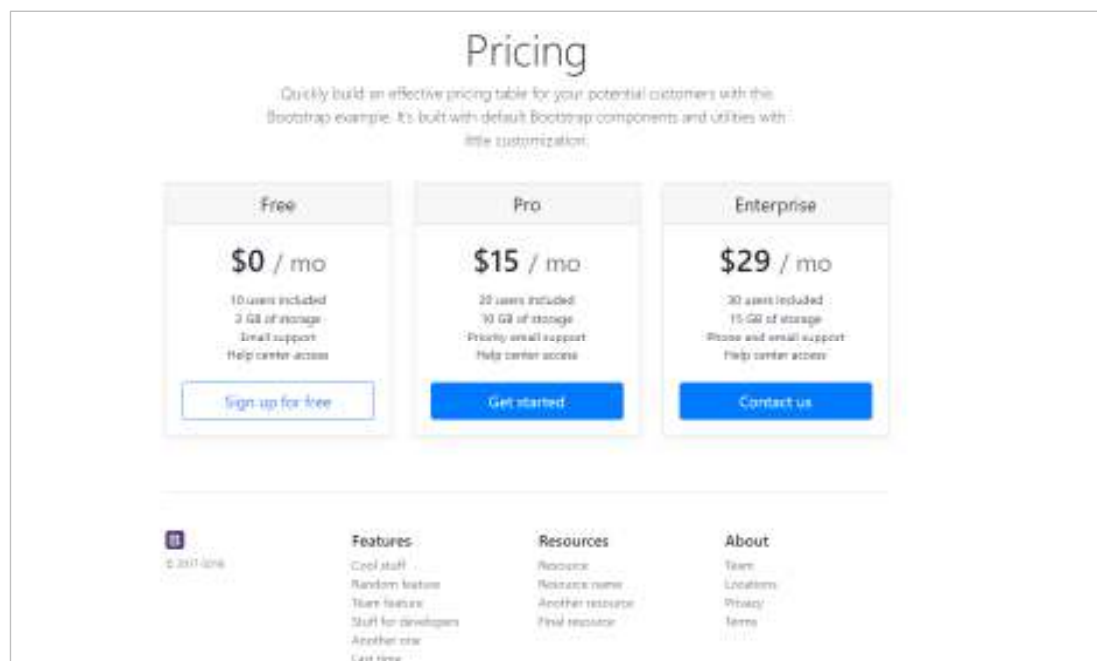
```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js" ></script>
```

■ Bootstrapp

All you need to do is learn the class names and apply them to your html markup. They'll do what they are designed to do.

<https://medium.freecodecamp.org/bootstrap-4-everything-you-need-to-know-c750991f6784>



<https://getbootstrap.com/docs/4.0/examples/>

Propriedades

display

[flexbox]

[CSS Grid]

float

*{float:***}*

■ **float**

- permite que um elemento seja movido o mais possível para a esquerda/direita e que seja envolvido pelo conteúdo que se lhe segue.

- Sem aplicação da propriedade *float*

```
<body>
  <div>FLOAT</div>
  <p>"Lorem ipsum dolor sit amet,
    consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
    Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex
    ea commodo consequat.</p>
</body>
```



"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

{float:***}

■ seletor {float:left}

```
div{
  width:70px;
  height:70px;
  background-color: orange;
  color:white;
  float:left
}
```

FLOAT

"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

■ seletor {float:right}

```
div{
  width:70px;
  height:70px;
  background-color: orange;
  color:white;
  float:right
}
```

"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

FLOAT

{float:***}

- CSS permitem aplicar a propriedade *float* a qualquer elemento (**block ou inline**)
 - **float** aplicado a *inline elements*

```
p{width:300px}
span{
  float:right;
  width:100px;
  margin:10px;
  background-color:lightgreen;
}
```

Exemplo de um elemento com inline com float

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged

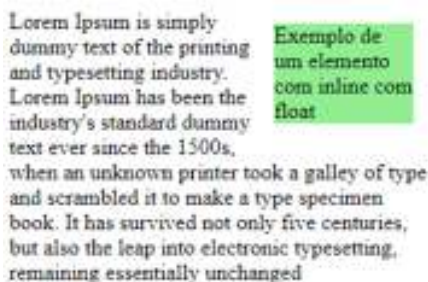
```
<p><span>Exemplo de um elemento com inline com float</span>Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged</p>
```

■ **float** aplicado a *inline elements*

- Um elemento posicionado com *float* não pode ser posicionado antes da sua posição natural no código fonte.

■ **Importante!**

- Todos os **floated elements** comportam-se como **block elements**.
- Sempre que se pretende efetuar o float de **elementos de texto** é necessário especificar a **width do elemento**, caso contrário toda a largura disponível é aproveitada.



■ Posicionamento flutuante

- Atualmente assume uma importância cada vez mais reduzida, mas por vezes ainda é utilizado

Propriedade	Exemplo	Observações
float	.paragrafo {float:left;}	Permite definir elementos com posicionamento flutuante. Valores: <i>left, right, none, inherit</i>
clear	.paragrafo {clear:both;}	Especifica em que lado do elemento não são permitidos elementos com posicionamento flutuante Valores: <i>left, right, both, none, inherit</i>

clear

- Sempre que se pretende interromper o efeito da propriedade float deve ser aplicada a propriedade *clear*

```
div{ width:100px; height:150px; color:white;font-size:1.5em; text-align: center}
#global{width:600px;height:500px}
#d1{background-color:lightgray; float:left}
#d2{background-color: darkorange; float:right}

#footer{ color:darkgray; text-align:left}
</style>
</head>
<body>
  <div id="global">
    <div id="d1">1</div>
    <div id="d2">2</div>
    <p id="footer">Lorem ipsum dolor sit amet,
      consectetur ... </p>
  </div>
```



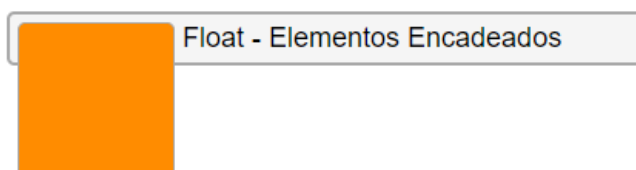
- interromper o efeito *float* (ex: posicionar um elemento de rodapé)

```
#footer{ clear: both; color:darkgray; text-align:left}
</style>
</head>
<body>
  <div id="global">
    <div id="d1">1</div>
    <div id="d2">2</div>
    <p id="footer">Lorem ipsum dolor sit amet,
      consectetur </p>
  </div>
```



clearfix

- *floats* em elementos encadeados
 - Quando a altura do container não é pré-definida:
 - O elemento interior é posicionado sem que o container acompanhe as suas dimensões



Altura não
especificada

```
.wrapper {
  max-width: 600px;
  margin: 40px auto;}

.container {
  border: 2px solid darkgray;
  border-radius: 5px;
  background-color: whitesmoke;
  width: 400px;
  padding: 5px;}

.item {
  height: 100px;
  width: 100px;
  margin-right: 5px;
  background-color:darkorange;
  border: 1px solid darkgray;
  border-radius: 5px;
  float: left;}

</style>
</head>
<body>

  <div class="wrapper">
    <div class="container">
      <div class="item"></div>
      float - Elementos Encadeados
    </div>
```

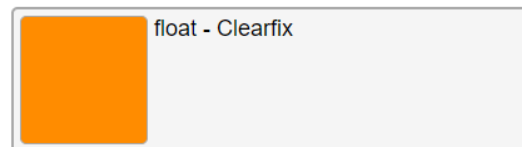
- Para resolver esta situação foi proposta uma solução (*clearfix*):

```

.item {
    height: 100px;
    width: 100px;
    margin-right: 5px;
    background-color:darkorange;
    border: 1px solid darkgray;
    border-radius: 5px;
    float: left;}

.container2::after {
    content: "";
    display: block;
    clear: both;}
</style>
</head>
<body>
    <div class="wrapper">
        <div class="container container2">
            <div class="item"></div>
            float - Clearfix
        </div>

```



Imediatamente após o <div> com class container 2, é adicionado **um elemento vazio**, o qual é definido como **block element** e sobre o qual é aplicada a propriedade **clear**

- Recentemente foi criado um novo valor para a propriedade *display* que origina um efeito semelhante ao *clearfix* anterior.

```

.item {
    height: 100px;
    width: 100px;
    margin-right: 5px;
    background-color:darkorange;
    border: 1px solid darkgray;
    border-radius: 5px;
    float: left;}

.container3 {
    display: flow-root;
}
</style>
</head>
<body>
    <div class="wrapper">
        <div class="container container3">
            <div class="item"></div>
            float - display:flow-root
        </div>

```



fluid Layout (float)

- Deprecated?
 - O CSS FlexBox e CSS GRID apresentam-se como alternativas muito mais flexíveis e vantajosas
 - A propriedade float era utilizada como base para a construção de layouts

```
#wrapper{width:100%;  
    margin:0px auto;  
    font-size: 25px;}  
#header,#footer{width:calc(100% - 4%);  
    height:25px;  
    padding:2%;  
    background-color: lightgray;}  
#nav{float:left;  
    width:20%;  
    height:500px;  
    padding:1%;  
    margin:1%;  
    background-color:orange;}  
#main{float:left;  
    width:72%;  
    height:500px;  
    padding:1%;  
    margin:1%;  
    background-color:lightblue;}  
#footer{clear:both;}
```

```
<div id="wrapper">  
    <div id="header">Header</div>  
    <div id="nav">Nav</div>  
    <div id="main">Main</div>  
    <div id="footer">Footer</div>  
</div>
```



Propriedades

display

position

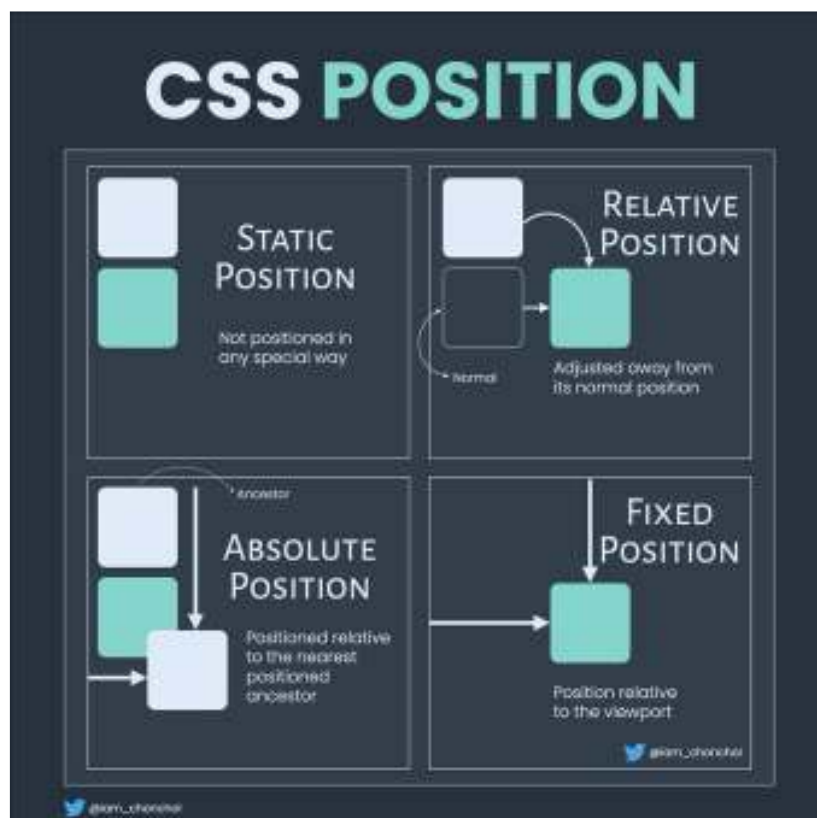
[flexbox]

[CSS Grid]

float

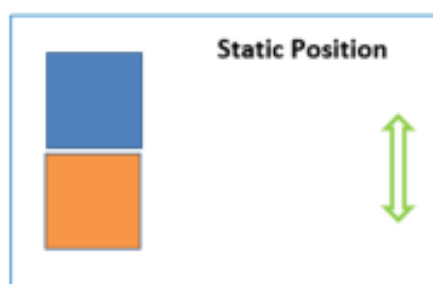
■ Posicionamento

Propriedade	Exemplo	Observações
position	div{position:absolute}	Permite definir o tipo de posicionamento de um elemento. Valores: - static* - relative - absolute - fixed - inherit - sticky



Posicionamento static

- `{position:static}` (default)
 - Os elementos são dispostos pela ordem que são definidos (top->bottom; left->right);
 - *Block elements*
 - Dispostos sequencialmente na vertical (a partir do topo)
 - Ocupam o espaço disponível na janela do browser ou definido pelo respetivo *container*
 - *Inline elements*
 - dispostos alinhados de forma a preencher os *block elements*
 - Não provoca nenhuma alteração ao posicionamento dos elementos



<https://www.csssolid.com/css-position.html>

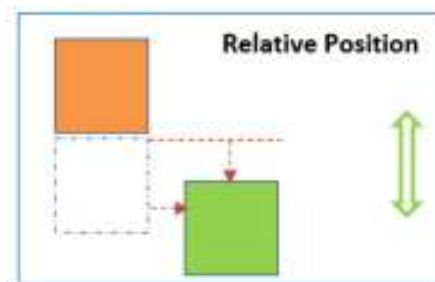
Posicionamentos ≠ static

- Os posicionamentos **relativo; absoluto e fixo** baseiam-se em deslocamentos a partir das referências: *top, left, right, bottom*
 - Nestes casos a propriedade *position* requer a especificação de propriedades de deslocamento:

Propriedade	Exemplo	Observações
top ↓	div{top:30px;left:50px;}	Permite definir os deslocamentos relativamente às referências, as quais dependem do valor especificado em <i>position</i> Valores: <i>length measurement, percentage, auto, inherit</i>
right ←		
bottom ↑		
left →		

Posicionamento Relativo

- Um elemento com `{position:relative}` posiciona-se em relação à sua **posição original** no fluxo HTML:
 - Os elementos com posicionamento relativo conservam o seu espaço original, assim:
 - Afetam o fluxo do HTML
 - Os elementos colocados depois das camadas relative, terão em conta as dimensões dos elementos anteriores
 - Os valores de posicionamento (top, left, ...) não afetam os elementos seguintes.



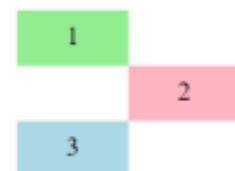
Posicionamento Relativo

- seletor `{position:relative}`
 - block-level elements*

```
<div id="d1">1</div>
<div id="d2">2</div>
<div id="d3">3</div>
```

```
div{width:100px;
height: 30px;
font-size: 1.5em;
padding:10px 0px;
text-align: center;}
#d1{background-color:lightgreen}
#d2{background-color:lightpink;
position:relative;
top:0px;
left:100px;}
#d3{background-color:lightblue}
</style>
```

A referência para os deslocamentos é a posição original do elemento deslocado.



O posicionamento relativo **preserva** o espaço original:

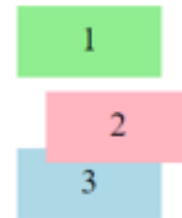
3 é posicionado tendo em conta o espaço originalmente ocupado por **2**

Posicionamento Relativo

- seletor `{position:relative}`
 - *block-level elements*

Sobreposição de elementos

```
#d2{background-color:lightpink;
    position:relative;
    top:10px;
    left:20px;}
#d3{background-color:lightblue}
```



O posicionamento de **3** não considera valores de top/left de **2** o que pode originar sobreposições

Posicionamento Relativo

- seletor `{position:relative}`
 - *inline-level elements*

A referência para os deslocamentos é a posição original do elemento deslocado.

```
<style>
  span{
    position:relative;
    top:50px;
    left:30px;
    background-color: orange;
    color:white;
  }
</style>
</head>
<body>
```

```
  <p>Posicionamento Relativo: <span>SPAN</span>sofreu um deslocamento.</p>
```

Posicionamento Relativo: sofreu um deslocamento.

SPAN

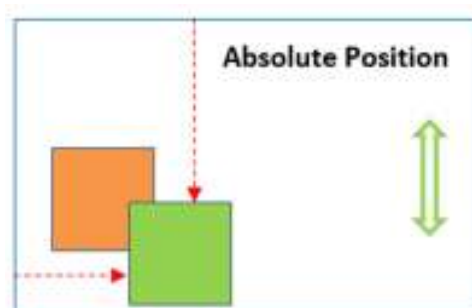
- O posicionamento relativo pode originar a sobreposição de elementos.

```
<style>
  span{
    position:relative;
    left:60px;
    background-color: orange;
    color:white;}
</style>
```

Posicionamento Relativo: sofSPAN deslocamento.

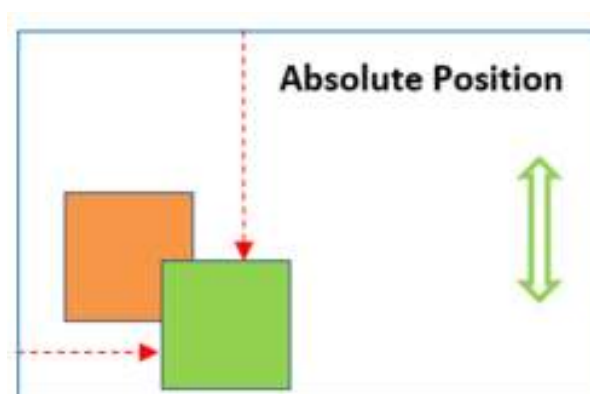
Posicionamento Absoluto

- O posicionamento absoluto é efetuado relativamente:
 - Ao *body* ou ao *container* mais próximo do elemento a deslocar.
 - `container ≠ <body>`
 - Se o elemento a posicionar estiver contido num elemento com **position ≠ static** então:
 - O elemento será posicionado relativamente a esse *container*
 - Caso contrário, é posicionado relativamente ao *body*



Posicionamento Absoluto

- `<body>`
 - Se o elemento a posicionar não está contido em outro elemento com **position ≠ static** então será posicionado relativamente ao `<body>` (diferente do posicionamento relativo ao *viewport*)

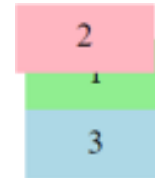


Posicionamento Absoluto

■ {position: **absolute**}

- A posição original do elemento **não é preservada, assim** o espaço original ocupado pelo elemento deslocado é preenchido pelos elementos que se lhe seguem no ficheiro HTML.
 - Criado um novo contexto de posicionamento

```
#d1{background-color:lightgreen}  
#d2{background-color:lightpink;  
  position: absolute;  
  top:0px;  
  left:0px;}  
#d3{background-color:lightblue}  
</style>
```



```
<div id="d1">1</div>  
<div id="d2">2</div>  
<div id="d3">3</div>
```

Neste caso o div 3 ocupa a posição de 2 uma vez que este elemento foi posicionado de forma absoluta

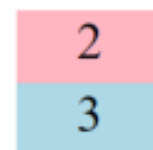
Posicionamento Absoluto

- Referência de posicionamento: container ≠ <body>

```
<div id="ref">  
  <div id="d1">1</div>  
  <div id="d2">2</div>  
  <div id="d3">3</div>  
</div>
```

A referência de posicionamento é #ref uma vez que é container dos div interiores e a sua posição é ≠ static

```
#ref{position: relative;}  
#d1{background-color:lightgreen}  
#d2{background-color:lightpink;  
  position: absolute;  
  top:0px;  
  left:0px;}  
#d3{background-color:lightblue}
```



Posicionamento Absoluto

- {position:**absolute**}
- *inline elements*

```
span{ position:absolute;
      color: darkorange;
      font-size: 1.5em;
      top:50px;
      left:50px}

</style>
</head>
<body>
  <p> Position Absolute: <span> elemento </span> posicionado de forma absoluta </p>
```

Position Absolute: posicionado de forma absoluta
↙
elemento

<body> é a referência para o deslocamento uma vez que o container <p> tem position:static

Posicionamento Absoluto

- {position:**absolute**}
- *inline elements*
 - *container* do elemento deslocado utilizado como referência para os deslocamentos

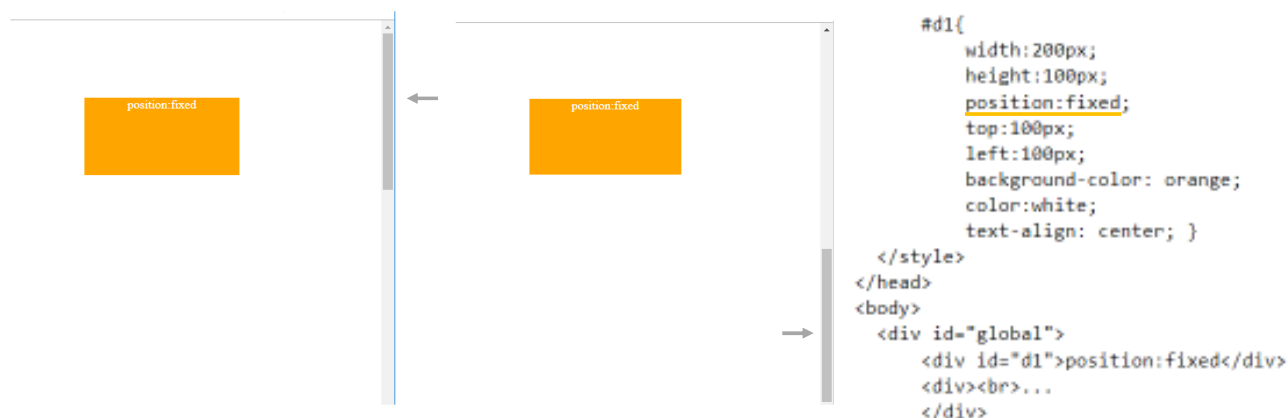
```
span{ position:absolute;
      color: darkorange;
      font-size: 1.5em;
      top:50px;
      left:50px}
p{position:relative}
</style>
</head>
<body>
  <p> Position Absolute: <span> elemento </span> posicionado de forma absoluta </p>
```

Position Absolute: posicionado de forma absoluta
↙
elemento

o posicionamento do *container* <p> foi alterado, o que origina que passe a ser a nova referência para os deslocamentos

Posicionamento fixo

- `{position:fixed}`
 - o posicionamento é efetuado relativamente ao *viewport* (área visível da janela do browser)
 - O posicionamento do elemento relativamente à área visível permanece inalterado mesmo quando é feito um *scroll* da página (ex: menu de navegação, ...)



Posicionamento *sticky*

- `{position:sticky}`
 - o elemento a posicionar, ao contrário do position fixed, acompanha o scroll da página até uma posição limite definida.

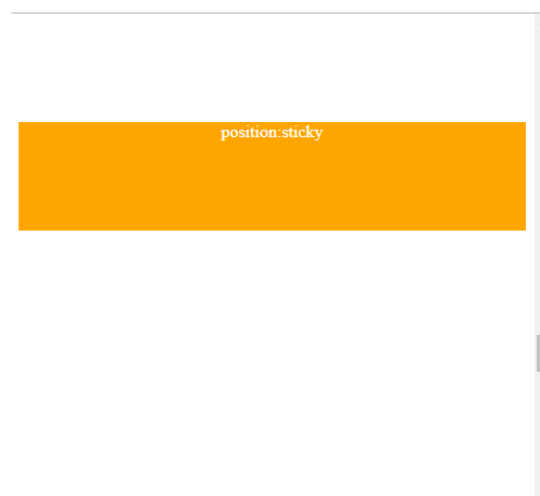
```
#d1{
  height:100px;
  position:sticky; top:100px;
  background-color: orange;
  color:white;
  text-align: center; }
</style>
</head>
<body>
  <div id="global">

    <br>... <br>

    <div id="d1">position:sticky</div>

    <br>...<br>

  </div>
</body>
```



Sobreposição de Elementos

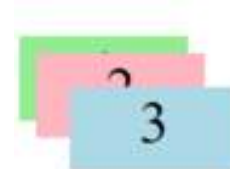
Propriedade	Exemplo	Observações
z-index	<code>div {z-index:5}</code>	Permite definir porque ordem são sobrepostos os elementos. Para valores positivos, quanto maior o valor mais à frente será posicionado o elemento (<i>bring to front</i>), a situação oposta ou a especificação de valores negativos enviar o elemento para trás (<i>send to back</i>)

```
#d1{background-color:lightgreen;  
  z-index:1;}
```

```
#d2{background-color:lightpink;  
  position:absolute;  
  top:10px;  
  left:10px;  
  z-index:2;}
```

```
#d3{background-color:lightblue;  
  position:absolute;  
  top:30px;  
  left:30px;  
  z-index:3;}
```

```
<div id="ref">  
  <div id="d1">1</div>  
  <div id="d2">2</div>  
  <div id="d3">3</div>  
</div>
```



Exemplo

Funcionamento submenu (display/position)

submenu (controlo de funcionamento)

■ submenu.html

```
<h2>Submenu mouseover</h2>

<div class="dropdown">
  <button class="dropbtn">Dropdown</button>
  <div class="dropdown-content">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
    <a href="#">Link 3</a>
  </div>
</div>
```

Submenu mouseover

Dropdown

■ estilos.css

```
.dropbtn {
  background-color: darkorange;
  color: white;
  padding: 16px;
  font-size: 16px;
  border: none;
  cursor: pointer;}
```

Formatação do botão

```
.dropdown {
  position: relative;
}
```

Referência de posicionamento
do submenu

submenu (controlo de funcionamento)

■ submenu.html

```
<h2>Submenu mouseover</h2>

<div class="dropdown">
  <button class="dropbtn">Dropdown</button>
  <div class="dropdown-content">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
    <a href="#">Link 3</a>
  </div>
</div>
```

Submenu mouseover

Dropdown

■ estilos.css

```
.dropdown-content {
  display: none;
  position: absolute;
  background-color: #f9f9f9;
  min-width: 160px;
  z-index: 1;
}
```

Este div fica oculto e posicionado de forma absoluta
(permite ajustar a posição do submenu)

```
.dropdown-content a {
  color: black;
  padding: 12px 16px;
  text-decoration: none;
  display: block;
}
```

Formatação dos links, que são interpretados
como block level elements

```
<h2>Submenu mouseover</h2>

<div class="dropdown">
  <button class="dropbtn">Dropdown</button>
  <div class="dropdown-content">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
    <a href="#">Link 3</a>
  </div>
</div>
```

Submenu mouseover

Dropdown

Submenu mouseover

Dropdown

Link 1

Link 2

Link 3

■ estilos.css : efeito hover

```
.dropdown:hover .dropbtn {
  background-color: lightsalmon;
}

.dropdown:hover .dropdown-content {
  display: block;
}

.dropdown-content a:hover {background-color:lightyellow}
```

Propriedades

display

position

[flexbox]

visibility

[CSS Grid]

float

■ Controlar a visibilidade de um elemento

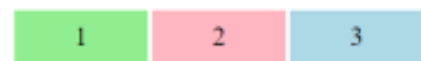
Propriedade	Exemplo	Observações
visibility	li {visibility:hidden;}	Permite definir a visibilidade de um elemento. Valores: hidden, visible,

- seletor{visibility:hidden}
- O elemento não é visualizado mas ao contrário do {display:none} mantém o espaço original

```
div{width:100px;
height: 30px;
font-size: 1.5em;
padding:10px;
text-align: center;}

#d1{background-color:lightgreen}
#d2{background-color:lightpink}
#d3{background-color:lightblue}
</style>
</head>
<body>

<div id="d1">1</div>
<div id="d2">2</div>
<div id="d3">3</div>
```



```
div{display:inline-block;
width:200px;
height:50px;}

#d2{visibility: hidden;}
```



Propriedades

display position
[flexbox] visibility
[CSS Grid] Texto
float

Formatação de texto

■ *font*

Propriedade	Exemplo	Observações
font-family	body{font-family:Arial, sans-serif;}	Podem ser definidas famílias genéricas: serif/sans serif; monospace/proportional; cursive (escrita manual); fantasy Não há limite para o número de fontes definidas.
	body{font-family:Tahoma, Geneva, sans-serif; }	Deve-se começar por fontes específicas e terminar com uma genérica, de forma a garantir a correta interpretação pelo browser.
font-size	h1{font-size: 1.5em;}	Unidades relativas
	h1{font-size: 150%;}	Unidades absolutas
	h1{font-size: x-large;}	keywords: xx-small, small, medium*, large, x-large, ...

Formatação de texto

■ *font*

Propriedade	Exemplo	Observações
font-weight	h1 {font-weight: bold;}	Valores : normal*, bold, bolder, lighter, 100...900, inherit (herda o valor do elemento pai) Forma correcta de colocar elementos a bold, deve substituir o elemento HTML ;
font-style	h1 {font-style: italic;}	Valores : normal*, italic, oblique, inherit
font-variant	h1 {font-variant: small-caps;}	Valores: normal*, small-caps, inherit
font	h1 {font: italic bold inherit 1.5em Arial, sans-serif;}	Propriedade abreviada, onde a ordem dos valores é importante {font: style weight variant size font-family}

* - default value

Formatação de texto

■ *Download* de fontes

■ @font-face rule

- Evita a necessidade da fonte utilizada ter de estar previamente instalada no cliente.
- A fonte é instalada no web server e sempre que necessário é efetuado o seu download.

```
@font-face {  
    font-family: newFont;  
    src: url('myfont.ttf')  
  
div{font-family:newFont;}
```

Formatação de texto

Propriedade	Exemplo	Observações
line-height	p {line-height: 2;} p {line-height: 2em;}	Define o espaçamento entre linhas. Valores: number (factor de escala), length measurement, percentage, normal*, inherit
text-indent	p#1 {text-indent: 2em;} p#1 {text-indent: 20%;}	Define a indentação de um parágrafo. Valores : length measurement (0*), percentage, inherit
text-align	p#1 {text-align: left;}	Alinhamento horizontal do texto. Valores: left*, right, center, justify, inherit
text-decoration	a {text-decoration: none;}	Permite criar sublinhados, linhas sobrepostas ao texto,..
text-transform	h1 {text-transform: none;}	Define o texto em maiúsculas, minúsculas, ... Valores: none*, capitalize (apenas a 1ª letra de cada palavra), lowercase, uppercase, inherit
letter-spacing	p {letter-spacing: 8px;}	Define o espaçamento entre letras. Valores: length measurement, normal*, inherit
word-spacing	p {word-spacing: 8px;}	Define o espaçamento entre palavras. Valores: length measurement, normal*, inherit

Formatação de texto (layout)

■ Múltiplas Colunas

```
p{
  background-color: orange;
  color:white;
  width: 600px;
  column-count: 3;
  font-size: 1.3em;
}
</style>
</head>
<body>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do ....
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud	exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu	fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum
--	--	--

Propriedades

display	position
[flexbox]	visibility
[CSS Grid]	Texto
float	<u>Cor</u>

Cor

■ Propriedades: Formatação (cor)

Propriedade	Exemplo	Observações
color	<code>h1 {color: red;}</code>	Valores: color value (RGB / hexadecimal); color name: 
	<code>h1 {color: #FF0000;}</code>	
	<code>h1 {color: #F00;}</code>	
	<code>h1 {color: rgb(255,0,0);}</code>	
background-color	<code>h1 {background-color: #F00;}</code>	Valores: color value (RGB / hexadecimal); color name

■ Propriedades: Formatação (cor)

■ CSS color names (140)

- `p {color: orange;}`

■ RGB (red, green, blue)

- Decimal [0-255]
 - `p {color: rgb(255,153,0);}`

■ Hexadecimal # RRGGBB [0-9;A-F]

- `p {color: #FF9900}`
- `p {color: #F90}` (notação condensada quando a notação hexadecimal é composta por três pares de valores duplicados [utilizado por defeito em alguns editores de HTML/CSS]

maroon #800000	red #ff0000	orange #ffa500	yellow #ffff00	olive #808000
purple #800080	fuchsia #ff00ff	white #ffffff	lime #00ff00	green #008000
navy #000080	blue #0000ff	aqua #00ffff	teal #008080	
black #000000	silver #c0c0c0	gray #808080		

Conversão decimal – hexadecimal:

*dividir o número original por 16, o 1º dígito é o quociente e o 2º dígito o resto da divisão
 $200 = C8 = (16 \times 12 + 8)$*

■ Existem vários *color palette generator* disponíveis:

- <https://www.materialpalette.com/light-green/purple>
- <https://digitalsynopsis.com/design/website-color-schemes-palettes-combinations/>
- <http://htmlcolorcodes.com/resources/best-color-palette-generators/>



■ *alpha channel*

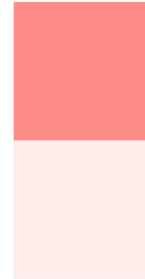
- grau de transparência na definição da cor

■ Exemplo:

- `h1 {color: #FF0000CC;}`

- `rgba(255,0,0,0.8)`

```
.alpha{
  width:100px;height:100px;
  background-color: #ff6f69CC;
}
.alpha2{
  width:100px;height:100px;
  background-color: #ff6f6922;
}
</style>
</head>
<body>
  <div class="alpha"></div>
  <div class="alpha2"></div>
```



Propriedades

display

[flexbox]

[CSS Grid]

float

position

visibility

Texto

Cor

opacity

- A propriedade *opacity* é frequentemente utilizada
 - permite controlar o nível de transparência de um elemento

```
h1{color:darkorange}
#o1{opacity:1;}
#o2{opacity:0.5;}
#o3{opacity:0.25;}
</style>
</head>
<body>
  <h1 id="o1">Opacity</h1>
  <h1 id="o2">Opacity</h1>
  <h1 id="o3">Opacity</h1>
```

Opacity

Opacity

Opacity

Propriedades

display

[flexbox]

[CSS Grid]

float

position

visibility

Texto

Cor

opacity

Imagens de Fundo

Propriedade	Observações
background-image	Imagem de fundo
background-repeat	Controla a repetição da imagem
background-position	Posição da imagem de fundo
background-attachment	Acompanha o deslocamento vertical dos conteúdos

Imagens de fundo

- **{background-size: width height;}**
 - É possível definir várias dimensões para a imagem de fundo, eliminando a dependência das dimensões originais da imagem, admite vários valores:
 - **auto**: default (dimensões originais da imagem)
 - **length / percentage** (relativamente ao *container*)
 - **cover**: escala a imagem de modo a cobrir toda a área visível, algumas partes da imagem podem ficar invisíveis (as proporções da imagem original são mantidas)
 - **contain**: escala a imagem considerando a área visível (toda a imagem é visível). Caso seja necessário a imagem é repetida.

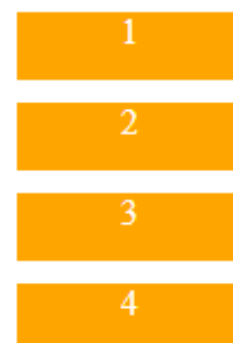
```
<style>
  body {color:white;
        background-image:url(back_Image.jpg);
        background-repeat:no-repeat;
        background-size:600px 100px;}
</style>
</head>
<body>
  <h1>Imagem de Fundo</h1>
```

Exercício *Flexbox*

Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{  background-color: orange;
      color:white;
      width:100px;
      height:30px;
      margin-top: 10px;
      text-align: center;
      list-style: none;
}
```



Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
ul{ display:flex;}
```

```
li{  background-color: orange;
     color:white;
     width:100px;
     height:30px;
     margin-top: 10px;
     text-align: center;
     list-style: none;
}
```

1 2 3 4

Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{background-color: orange;
    color:white;
    width:100px;
    height:30px;
    margin-top: 10px;
    text-align: center;
    list-style: none;

    margin:0 1px;
}

ul{ display:flex;}
```

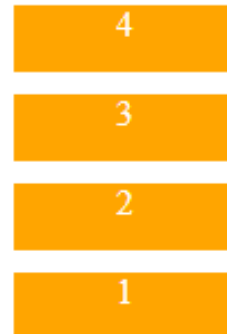
1 2 3 4

Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{background-color: orange;
  color:white;
  width:100px;
  height:30px;
  margin-top: 10px;
  text-align: center;
  list-style: none;
}

ul{ display:flex;
  flex-direction: column-reverse;}
```



Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{background-color: orange;
  color:white;
  width:100px;
  height:30px;
  text-align: center;
  list-style: none;

  margin: 2px;
}

ul{ display:flex;
  flex-direction: row;}
```



Exercício Flexbox

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{background-color: orange;
    color:white;
    width:100px;
    height:30px;
    text-align: center;
    list-style: none;
    margin: 2px;}
```

```
ul{ display:flex;
    flex-direction: row;
    border:1px solid gray;
    width:250px;
    flex-wrap: nowrap;
    justify-content:flex-start;
    padding:0px;}
```



Exercício Flexbox

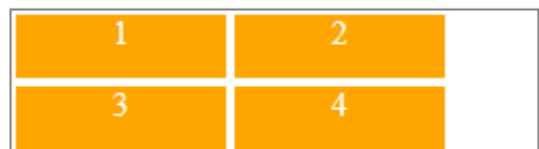
```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```

```
li{background-color: orange;
    color:white;
    width:100px;
    height:30px;
    text-align: center;
    list-style: none;

    margin: 2px;}
```

```
ul{ display:flex;
    flex-direction: row;
    border:1px solid gray;
    width:250px;
    justify-content:flex-start;
    padding:0px;

    flex-wrap: wrap;}
```



Exercício Flexbox

```
li{background-color: orange;
  color:white;
  width:100px;
  height:30px;
  text-align: center;
  list-style: none;
  margin: 2px;

  flex-basis:20px;
}

ul{ display:flex;
  flex-direction: row;
  border:1px solid gray;
  width:250px;
  justify-content:flex-start;
  padding:0px;

  flex-wrap: wrap;}
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```



Exercício Flexbox

```
li{background-color: orange;
  color:white;
  width:100px;
  height:30px;
  text-align: center;
  list-style: none;
  margin: 2px;
  flex-basis:20px;
  flex-grow: 1;}

ul{ display:flex;
  flex-direction: row;
  border:1px solid gray;
  width:250px;
  justify-content:flex-start;
  padding:0px;
  flex-wrap: wrap;}
```

```
<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
  <li>4</li>
</ul>
```



Propriedades

display	position	opacity
[flexbox]	visibility	Imagens de Fundo
[CSS Grid]	Texto	<u>Listas</u>
float	Cor	

listas

Propriedade	Exemplo	Observações
list-style-type	<code>ul {list-style-type:square;}</code>	Permite escolher o símbolo de uma lista. Valores: none, disc*, circle, square, ...
list-style-position	<code>ul {list-style-position: outside;}</code>	Define se o símbolo da lista se encontra no interior ou exterior do background aplicado aos elementos da lista Valores : inside, outside, inherit
list-style-image	<code>ul {list-style-image: url(/image.png);}</code>	Criar novos símbolos de lista (bullets) Valores: url, none*, inherit

* - default value

Propriedades

display	position	opacity
[flexbox]	visibility	Imagens de Fundo
[CSS Grid]	Texto	Listas
float	Cor	<u>Tabelas</u>

Tabelas

■ Propriedades específicas para tabelas

Propriedade	Observações
<i>border-collapse</i>	Separação entre células (borders separados, ou apenas um único border a separar células adjacentes). Valores: separate, collapse, inherit
<i>border-spacing</i>	Espaçamento entre células. Valores: length, inherit
<i>empty-cells</i>	Visibilidade de células vazias Valores: show, hide, inherit


```
<style>
  table{
    border-style:solid;
    border-width:1px;
    border-color:#900;
    border-collapse: separate;
    border-spacing:10px;
    empty-cells:hide;}

  th, td {border-style:solid;
    border-width:1px;
    border-color:#900;
    width:100px;
    text-align:center;}
</style>
```

```
<table>
  <tr><th>(1,1)</th>
    <th>(1,2)</th>
  </tr>
  <tr><td>(2,1)</td>
    <td>(2,2)</td>
  </tr>
  <tr><td>(3,1)</td>
    <td></td>
  </tr>
</table>
```

(1,1)	(1,2)
(2,1)	(2,2)
(3,1)	

Propriedades

display

[flexbox]

[CSS Grid]

float

position

visibility

Texto

Cor

opacity

Imagem Fundo

Listas

Tabelas

Formulários

- Uma formatação correta dos elementos de um formulários é determinante para assegurar a sua usabilidade (objetivo principal)
 - Existem determinadas propriedades que só se aplicam a campos específicos

Elementos	Propriedades
text inputs (text, password, email, search, tel, url)	<i>width, height, background-color, background-image, border, border-radius, margin, padding, box-shadow, color, ...</i>
textarea	<i>line-height, resize</i>
inputs (submit, reset, button)	<i>width, height, border, background-color, margin, padding, box-shadow, ...</i>
select	<i>width, height, color, background-color ...</i>
fieldsets / legends	<i>border, background-color, margin, padding, ...</i>

Propriedades

display	position	opacity	Formulários
[flexbox]	visibility	Imagem Fundo	<u>Transformações 2D/3D</u>
[CSS Grid]	Texto	Listas	
float	Cor	Tabelas	

Transformações 2D

- Propriedade com múltiplos valores: http://www.w3schools.com/cssref/css3_pr_transform.asp

- `seletor{ transform: rotate (deg)}`

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title></title>
  <style>
    div {width:200px;
        height:100px;
        background-color:darkorange;
        color:white;
        position:absolute;left:50px;top:100px;
        text-align:center;
        transform:rotate(30deg);
    }
  </style>
</head>
<body>
  <div>transform:ROTATE(</div>
</body>
</html>
```



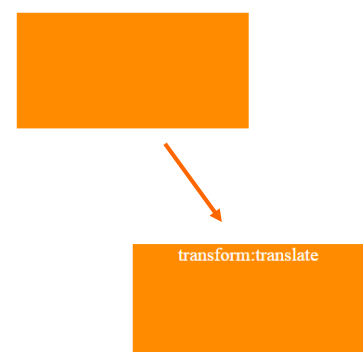
Por defeito, a imagem roda sobre o seu centro.
A propriedade *transform-origin* permite alterar o centro de rotação.

Transformações 2D

- `seletor{ transform: translate(xx,yy)}`

- A referência para o movimento de translação é a posição original do objecto

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div {width:200px;
        height:100px;
        background-color:darkorange;
        color:white;
        position:absolute;left:50px;top:100px;
        text-align:center;}
    #d2 {transform:translate(100px,200px);}
  </style>
</head>
<body>
  <div></div>
  <div id="d2">transform:translate</div>
</body>
</html>
```



Transformações 2D

■ `seletor{ transform: scale(xx,yy)}`

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div {width:60px;
        height:20px;
        background-color:darkorange;
        color:white;
        position:absolute;left:50px;top:100px;
        text-align:center; }
    #d2 {left:150px;
        transform:scale(2,3);}
  </style>
</head>
<body>
  <div></div>
  <div id="d2">scale</div>
</body>
</html>
```



Transformações 3D

■ `seletor{transform: rotate X}; seletor{transform: rotateY}`

```
...
    .dft{ background-color: darkorange;
        color:white;
        font-size: 2em;
        width:50px;
        height:100px;
        margin:10px;
        text-align: center}

    .r3d{ transform: rotateX(180deg);}
  </style>
</head>

<body>

  <div class="dft">1</div>
  <div class=" dft r3d">1</div>
  ...
```



Propriedades

display	position	opacity	Formulários
[flexbox]	visibility	Imagem Fundo	Transformações 2D/3D
[CSS Grid]	Texto	Listas	<u>Transições</u>
float	Cor	Tabelas	

Transições

- Suavização de alterações entre estados diferentes
 - Definir uma transição envolve:
 - Propriedade a alterar (obrigatório!)
 - Duração (obrigatório!)
 - A forma como se processa a aceleração da transição
 - A eventual existência de uma pausa antes de iniciar a transição.

Transições

Propriedade	Exemplo	Observações
transition-property	{transition-property: width;}	Algumas propriedades que podem ser animadas: background-color; background-position; height, width, font-size; font-weight;...
transition-duration	{transition-duration: 1s;}	Os valores podem ser definidos em segundos (s) ou milissegundos (ms)
transition-timing-function	{transition-timing-function: ease;}	ease (início lento, rápido, final lento); linear (a velocidade mantém-se do início ao fim); ease-in (início lento, final rápido); ease-out (início rápido, final lento); ease-in-out (início lento, rápido, final lento).
transition-delay	{transition-delay: 0.3s;}	Os valores podem ser definidos em segundos (s) ou milissegundos (ms)
transition	{transition: width 1s ease-in-out 0.3s;}	{transition: property duration timing-function delay}

Transições

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div {width:100px;
        height:100px;
        background-color:orange;
        transition:width 3s; }
    div:hover {width:300px;}
  </style>
</head>
<body>
  <div></div>
</body>
</html>
```

Transição width Elemento div



Uma vez o rato sobreposto ao elemento div, efetua-se a transição da **width** inicial para a **width** final



duração: 3s

Transição width Elemento div

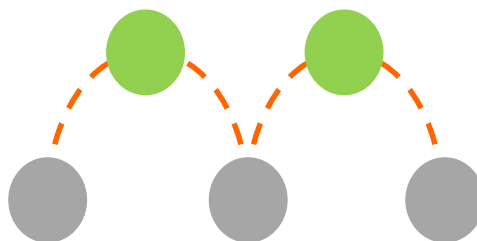


Propriedades

display	position	opacity	Formulários
[flexbox]	visibility	Imagem Fundo	Transformações 2D/3D
[CSS Grid]	Texto	Listas	Transições
float	Cor	Tabelas	<u>Animações</u>

Animação

- Baseada em *key frames* (*keyframe animation*)
 - Transições são animações baseadas em 2 *key frames* (inicial, final)
 - Animações mais complexas requerem a especificação de um maior número de *key frames*



Keyframes

`@keyframes nomeAnimação { ... }`

Parâmetros

▪ Definição de *key frames*

nome da animação (obrigatório)

```
@keyframes quadrados {  
  0% {background-color:red; left:0px; top:0px}  
  25% {background-color:yellow; left:200px; top:0px}  
  50% {background-color:blue; left:200px; top:200px}  
  75% {background-color:green; left:0px; top:200px}  
  100% {background-color:red; left:0px; top:0px}  
}
```

Definição das *key frames* (obrigatório)
(neste exemplo foram definidas 5 *key frames*)

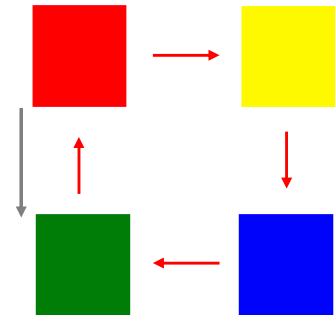
- Definir os parâmetros da animação
 - *animation-name* (**obrigatório!**)
 - *animation-duration*
 - Tempo, definido em segundos, que uma animação demora
 - o valor por defeito é o 0 (sem animação)
 - os valores negativos são tratados como 0
 - *animation-timing-function*
 - **ease** : Valor por defeito. A animação começa lenta, depois rápido, antes de terminar lento
 - **linear** : Velocidade constante ao longo da animação
 - **ease-in** : Animação inicia lenta
 - **ease-out** : Animação termina lenta
 - **ease-in-out** : Animação com um começo e um final lentos
 - **cubic-bezier(x1, y1, x2, y2)** : Controlo mais preciso da velocidade da animação

- *animation-delay*
 - define o tempo de intervalo (definido em s) entre a ocorrência do evento e o início da animação
- *animation-iteration-count*
 - define o número de vezes que a animação é executada
 - Infinite: animação é executada de forma ininterrupta
- *animation-direction*
 - normal
 - valor por defeito. A animação é sempre reproduzida na mesma direção
 - alternate
 - executa as iterações ímpares na direção normal e as iterações pares na direção inversa
- ...

■ Animação

- Parametrização da animação e associação com *key frames* definidas

```
<!DOCTYPE html>
<html>
<head>
  <style>
    div {width:100px;
        height:100px;
        background-color:red;
        position:relative;
        animation:quadrados 5s infinite alternate;}
    @keyframes quadrados {
      0% {background-color:red; left:0px; top:0px}
      25% {background-color:yellow; left:200px; top:0px}
      50% {background-color:blue; left:200px; top:200px}
      75% {background-color:green; left:0px; top:200px}
      100% {background-color:red; left:0px; top:0px}
    }
  </style>
</head>
<body>
  <h2>Animação Quadrados</h2>
  <div></div>
</body>
</html>
```



Exercício

Exercício

- Qual é o output?

```
<style>
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange;}
  #d3 {background-color:lightgreen;}
</style>
```

```
<div>
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```



Exercício

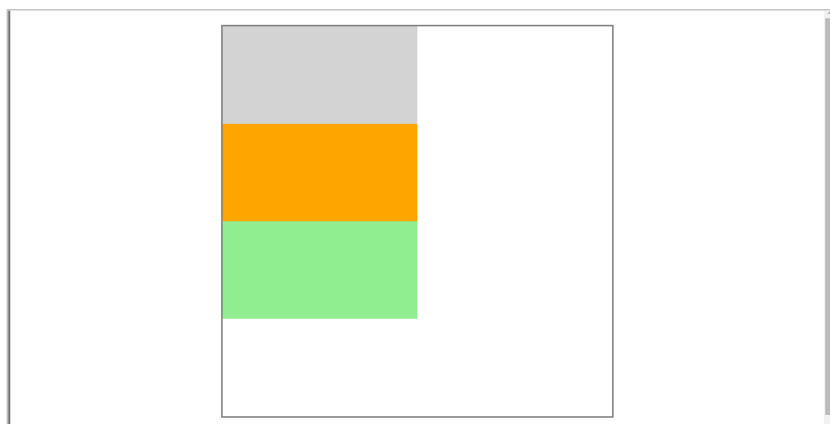
- Qual é o output?

```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```

```
<style>
  #container { width:400px;height:400px;
    margin:20px auto;
    border: 2px gray solid;}

  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange;}
  #d3 {background-color:lightgreen;}
</style>
```



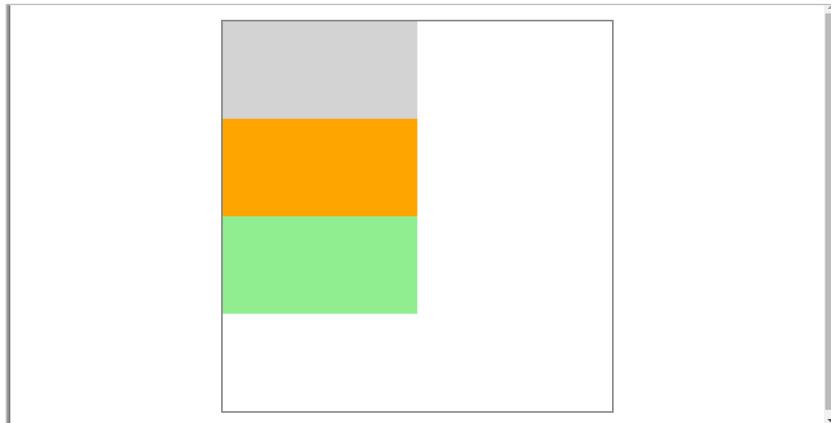
Exercício

- Qual é o output?

```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```

```
<style>
  #container { width:400px;height:400px;
               margin:20px auto;
               border: 2px gray solid;}
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange;   left:200px}
  #d3 {background-color:lightgreen;}
</style>
```



Exercício

- Qual é o output?

```
<style>
  #container { width:400px;height:400px;
               margin:20px auto;
               border: 2px gray solid;}
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange; position: relative; left:200px}
  #d3 {background-color:lightgreen;}
</style>
```

```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```



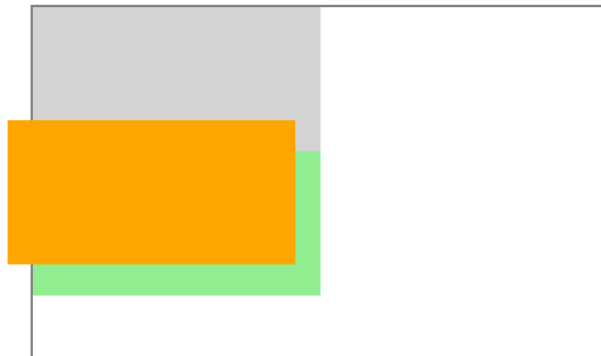
Exercício

Qual é o output?

```
<style>
  #container { width:400px;height:400px;
               margin:20px auto;
               border: 2px gray solid;}
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange; position:absolute; left:200px; top:100px;}
  #d3 {background-color:lightgreen;}
</style>
```

```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```

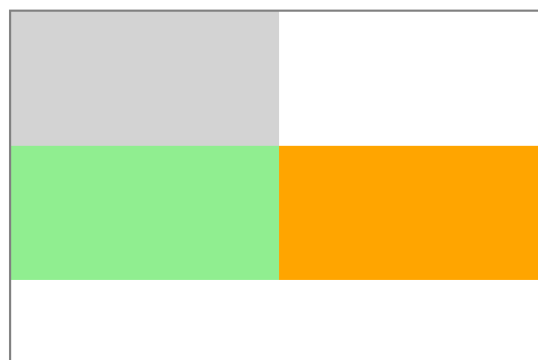


Exercício

Qual é o output?

```
<style>
  #container { width:400px;height:400px;
               margin:20px auto;
               border: 2px gray solid;
               position:relative; }
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange; position:absolute; left:200px; top:100px;}
  #d3 {background-color:lightgreen;}
</style>
```



Exercício

Código?

(posicionamento absoluto)
Bloco verde



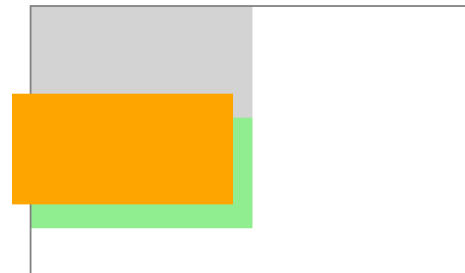
```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```

```
<style>
  #container { width:400px;height:400px;
    margin:20px auto;
    border: 2px gray solid;
    position:relative; }
  #d1, #d2, #d3 {width:200px;height:100px;}

  #d1 {background-color:lightgray;}
  #d2 {background-color:orange; position:absolute; left:200px; top:100px;}
  #d3 {background-color:lightgreen; position:absolute; left:0px; top:200px}
</style>
```

Exercício

```
<div id="container">
  <div id="d1"></div>
  <div id="d2"></div>
  <div id="d3"></div>
</div>
```



```
#d1 {background-color:lightgray; position:relative; z-index:2;}
#d2 {background-color:orange; position:absolute; left:200px; top:100px; z-index:1;}
#d3 {background-color:lightgreen; position:relative; z-index:2;}
```

?



Web Responsive

Web Responsive

- Permite que uma aplicação seja utilizada de forma adequada independentemente do dispositivo que está a ser utilizado para aceder aos conteúdos
 - Todos os dispositivos acedem ao mesmo conteúdo usando o mesmo URL
 - São aplicados estilos diferentes, de acordo com o dispositivo, de forma a redimensionar componentes e otimizar a usabilidade (interação com o utilizador)



Três componentes principais:

■ layout flexível (*flexible grid*)

- As dimensões não podem ser estáticas, têm de ajustar (reduzir/aumentar) de acordo com o espaço disponível.

■ Imagens flexíveis

- Imagens com capacidade de sofrer um efeito de escala (redução da dimensão) de acordo com a alteração do layout.

■ CSS media queries

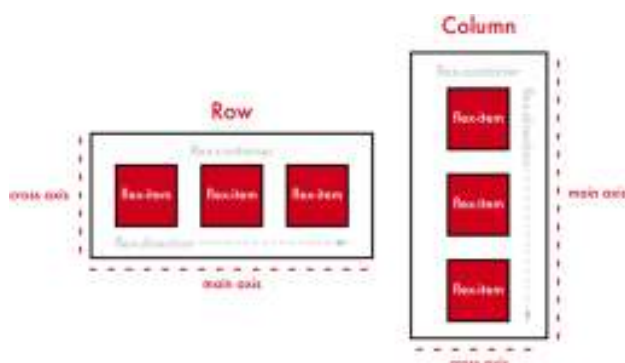
- método que permite a aplicação de estilos de acordo com o dispositivo em que a aplicação vai ser visualizada.

■ Layout flexível

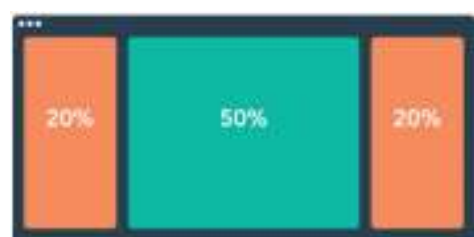


CSS Grid

Flexbox Container



Fluid Layout



■ Imagens/media Flexíveis

- A imagem acompanha a redução do layout sendo que na situação contrária a sua dimensão original é o limite para a sua visualização.

```
img{max-width: 100%;}
```

- A propriedade *max-width* estabelece a largura máxima de um elemento
- Se for definida em %, a largura do elemento é **diretamente indexada às dimensões do respetivo container** até à dimensão original da imagem ($\neq$ width:100%).
 - O elemento ajusta-se automaticamente à variação da largura do respetivo *container*
- Também aplicável a outro tipo de média

```
img, object, video {max-width:100%;}
```

■ CSS media queries

- Permite a aplicação de estilos de acordo com meio usado (*media type*) para visualização dos conteúdos web assim como em função das suas características (*media features*)

■ CSS media queries

■ media types

- screen, print, all, ... <https://www.w3.org/TR/CSS21/media.html#23media-types> *Opcional!*

■ media features

- width; height; device-width; ...

■ CSS media queries

■ media features

Media feature	Observações
width	largura do viewport
height	altura do viewport
orientation	portrait/landscape
aspect-ratio	razão entre a largura e a altura do viewport
...	...

- Algumas propriedades podem ser testadas para valores limite, utilizando para tal prefixos:

■ **min, max**

- min-width; max-width
- min-height; max-height

■ CSS media queries

- Efetuados diretamente na folha de estilos (**a forma mais comum**):

```
@media screen and (min-width:480px){  
  
    /* CSS Rules */  
  
}  
  
@media screen and (min-width:480px) and (orientation:landscape){  
  
    /* CSS Rules */  
  
}
```

- Em alternativa ser incorporados no html, incorporando os *media queries* na tag **<link>** através do atributo **media**

```
<head>  
  <link href="geral.css" rel="stylesheet">  
  <link href="colunas.css" rel="stylesheet" media="screen and (min-width:480px)">  
</head>
```

- Quando se utilizam *@media rules* a **ordem** das regras é muito importante (para seletores iguais prevalecem as ultimas regras a ser definidas):
 - Estratégia:
 - Especificam-se as regras aplicadas por defeito aos elementos (*baseline rules*)
 - Algumas dessas regras vão ser posteriormente substituídas por outras regras inseridas nas *@media* de forma a otimizar o conteúdo para determinados viewports.
 - A definição de *media queries* é baseada do conceito de *mobile-first*, ou seja:
 - Começa-se por definir os layouts para os dispositivos mais pequenos;
 - À medida que o espaço de visualização aumenta, são aplicados novos estilos para novas possibilidades de visualização.

```
@media screen and (min-width:480px){  
    /* CSS Rules */  
}  
  
@media screen and (min-width:768px){  
    /* CSS Rules */  
}
```

- Definição das *@media queries*
 - Escolha dos **breakpoints** (largura estabelecida no *media query* para definir novos estilos)
 - <http://responsivedesign.is/develop/browser-feature-support/media-queries-for-common-device-breakpoints>
 - <http://css-tricks.com/snippets/css/media-queries-for-standard-devices/>

```
/* ----- iPhone X ----- */  
  
/* Portrait and Landscape */  
@media only screen  
    and (min-device-width: 375px)  
    and (max-device-width: 812px)  
    and (-webkit-min-device-pixel-ratio: 3) {  
  
}  
  
/* Portrait */  
@media only screen  
    and (min-device-width: 375px)  
    and (max-device-width: 812px)  
    and (-webkit-min-device-pixel-ratio: 3)  
    and (orientation: portrait) {  
  
}
```

■ CSS Pixel

- Corresponde ao pixel definido (abstração) nas declarações CSS

- width:200px; padding:10px;...

- <https://www.w3.org/TR/CSS2/syndata.html#length-units>

■ pixel (device)

- número de pixels existentes no dispositivo (hardware pixels - resolução)

■ Device Pixel Ratio (DPR)

- A relação que existe entre os pixels (device) e os pixels CSS.

- Exemplo: 1 CSS pixel corresponde a 4 device pixels (DPR = 2)



<http://blog.popupdesign.com.br/desenvolvimento-responsivo-e-viewport/>

■ Device pixel ratio (DPR)

pixel (CSS)
device width / device height

resolution
pixels device

	Apple iPhone 14	390 x 844	1170 x 2532	6.1"	3.0
	Apple iPhone 14 Plus	428 x 926	1284 x 2778	6.7"	3.0
	Apple iPhone 14 Pro	393 x 852	1179 x 2556	6.1"	3.0
	Apple iPhone 14 Pro Max	430 x 932	1290 x 2796	6.7"	3.0
	Apple iPhone 15	393 x 852	1179 x 2556	6.1"	3.0
	Apple iPhone 15 Plus	430 x 932	1290 x 2796	6.7"	3.0
	Apple iPhone 15 Pro	393 x 852	1179 x 2556	6.1"	3.0

<https://yesviz.com/viewport/>

■ Variação da Janela de Visualização (viewport)

■ `<meta name="viewport" ... />`

- A especificação do *viewport* indica ao *browser* que deve ser aplicado um factor de escala à página para esta se ajustar às dimensões do dispositivo (screen)
- Deve ser incluído em todos os documentos html (responsive)

Property	Description
width	The width of the virtual viewport of the device.
device-width	The physical width of the device's screen.
height	The height of the "virtual viewport" of the device.
device-height	The physical height of the device's screen.
initial-scale	The initial zoom when visiting the page. 1.0 does not zoom.
minimum-scale	The minimum amount the visitor can zoom on the page. 1.0 does not zoom.
maximum-scale	The maximum amount the visitor can zoom on the page. 1.0 does not zoom.
user-scalable	Allows the device to zoom in and out. Values are yes or no.

■ O *viewport* necessita de ser declarado apenas uma vez

“A `<meta>` viewport element gives the browser instructions on how to control the page's dimensions and scaling”

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

■ ***content = “width=device-width,”***

- Largura do *viewport* igual à largura do dispositivo

■ ***content = “width=device-width, initial-scale=1”***

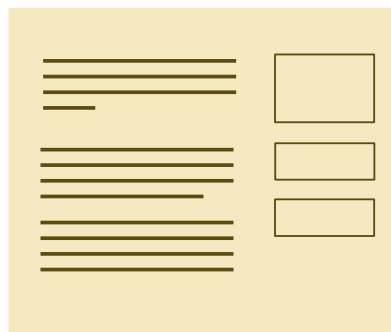
- Garante o nível de zoom inicial quando é feito o download da página
- Evita que seja visualizado apenas uma parte do conteúdo inicial

Web Responsive

Exemplo mobile safari
renderizado: 980px



```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



screen: 320px



renderizado: 320px

Todo o espaço disponível
é aproveitado.

O viewport é definido
como sendo igual à
largura do dispositivo

Permite uma correta
aplicação das CSS media
queries

screen: 320px

Web Responsive

- Sem definição da meta *tag*, considerado por defeito um *viewport* de 980px largura
 - exemplo: iPhone 12 Pro

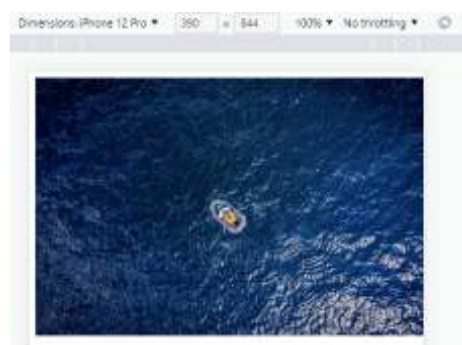


A imagem é renderizada a 980px
e depois ajustada para a
dimensão do dispositivo

Neste caso é desperdiçado cerca
de 50% do espaço disponível

- Com ajuste do *viewport*

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



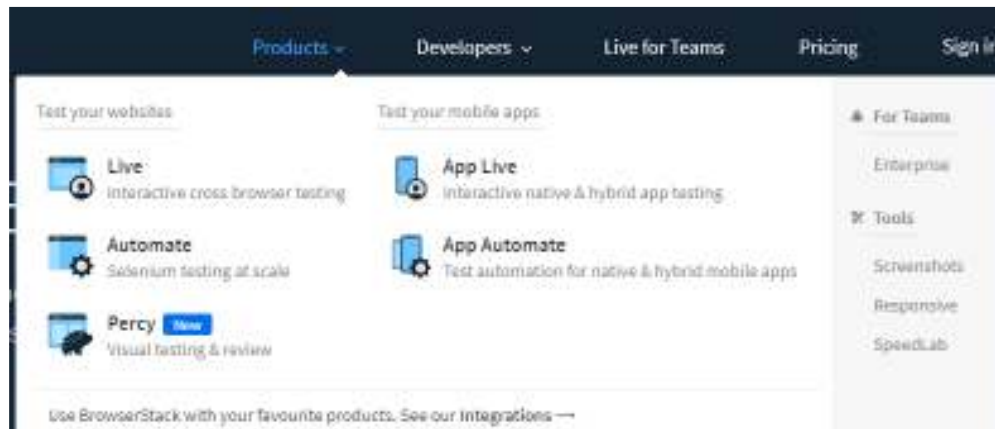
A imagem é renderizada
considerando a largura do
dispositivo

Optimização do espaço disponível

■ Testing

■ Emuladores

- <https://www.browserstack.com/emulators-simulators>



CSS Reset

■ CSS reset

- Aplicação de um conjunto de regras CSS que inviabilize a aplicação de estilos por defeito (browsers) de forma a tornar o ponto de partida tão neutro quanto possível
 - Melhora a portabilidade entre browsers
 - Existem disponíveis vários tipos de CSS reset
 - Um CSS reset deve ser copiado para o início da CSS de forma a garantir que é efetuado de forma correta
 - Tem vindo a ser substituído pelo Normalize.css

```
html, body, div, span, applet, object, iframe,
h1, h2, h3, h4, h5, h6, p, blockquote, pre,
a, abbr, acronym, address, big, cite, code,
del, dfn, em, img, ins, kbd, q, s, samp,
small, strike, strong, sub, sup, tt, var,
b, u, i, center,
dl, dt, dd, ol, ul, li,
fieldset, form, label, legend,
table, caption, tbody, tfoot, thead, tr, th, td,
article, aside, canvas, details, embed,
figure, figcaption, footer, header, hgroup,
menu, nav, output, ruby, section, summary,
time, mark, audio, video {
    margin: 0;
    padding: 0;
    border: 0;
    font-size: 100%;
    font: inherit;
    vertical-align: baseline;
}

/* HTML5 display-role reset for older browsers */
article, aside, details, figcaption, figure,
footer, header, hgroup, menu, nav, section {
    display: block;
}
body {
    line-height: 1;
}
ol, ul {
    list-style: none;
}
blockquote, q {
    quotes: none;
}
blockquote:before, blockquote:after,
q:before, q:after {
    content: '';
    content: none;
}
table {
    border-collapse: collapse;
    border-spacing: 0;
}
```

<http://cssreset.com/>

Tecnologias Web 2023/2024
Simão Paredes sparedes@isec.pt

392

Normalize.css



Normalize.css

A modern, HTML5-ready alternative
to CSS resets

"Normalize.css makes browsers render all elements more consistently and in line with modern standards. It precisely targets only the styles that need normalizing."

```
<link rel="stylesheet" type="text/css" href="path/to/normalize.css">
```

```
/*! normalize.css v8.0.1 | MIT License | github.com/necolas/normalize.css */

/* Document
   ========================================================================= */

/**
 * 1. Correct the line height in all browsers.
 * 2. Prevent adjustments of font size after orientation changes in iOS.
 */

html {
    line-height: 1.15; /* 1 */
    -webkit-text-size-adjust: 100%; /* 2 */
}

/* Sections
   ========================================================================= */

/**
 * Remove the margin in all browsers.
 */

body {
    margin: 0;
}
```

<https://necolas.github.io/normalize.css/8.0.1/normalize.css>

Tecnologias Web 2023/2024
Simão Paredes sparedes@isec.pt

393

Questões Exame (Época Normal 2022/23)

Questões Exame (Época Normal 2022/23)

[0.3 Valores]

- 7. A adaptação do *viewport* aos diversos dispositivos móveis é assegurada pela *tag*:
- A. <responsive>.
 - B. <picture>.
 - C. <meta>. 
 - D. <viewport>.

[0.3 Valores]

- 18. O inline style CSS baseia-se numa formatação:
- A. local aos elementos HTML e como tal não deve ser evitado.
 - B. efetuada num ficheiro externo *.css e como tal deve ser evitado.
 - C. efetuada num ficheiro externo *.css e como tal não deve ser evitado.
 - D. local aos elementos HTML e como tal deve ser evitado. 

[0.3 Valores]

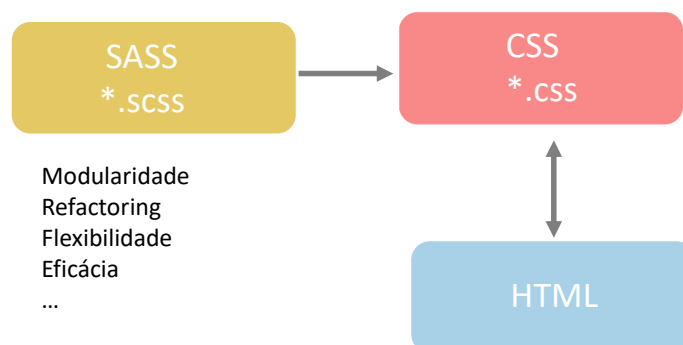
- 29. No contexto das CSS Grid as propriedades grid-row e grid-column destinam-se a:
- A. definir o número de linhas/colunas da grid.
 - B. posicionamento da grid no viewport.
 - C. posicionamentos dos grid items na grid. 
 - D. gerar automaticamente linhas e colunas da grid.

SASS

Syntactically Awesome Style Sheets

Sass

- Pré-processador código CSS
 - facilita a manutenção de código CSS, melhora a organização do código, mais escalável,...
 - Variáveis,
 - Reutilização,
 - ficheiros de extensão *.scss a partir dos quais são criados os ficheiros *.css a ligar ao HTML (HTML **não reconhece** *.scss)



Sass: Características Principais

■ Variáveis

- permite a declaração de variáveis, as quais são precedidas de \$

- Tipos de dados:

- numbers
- strings
- colors
- null
- lists
- maps

```
$margin-values: 1px 2px 3px 4px;
```

```
$map: (  
  key: value,  
  nextkey: nextvalue  
);
```

- As variáveis devem:

- possuir um significado semântico
- seguir um padrão/convenção (ex: prefixos “header-” “footer-” “section-”, ou sufixos “-color”)

Sass: Características Principais

■ Variáveis

- Importantes quando o mesmo valor é utilizado em vários sítios

- Determinantes para a estruturação e manutenção do código, principalmente em projetos de grande dimensão

```
variaveis.scss  
1 $font-stack: Helvetica, sans-serif;  
2 $primary-color: #333;  
3  
4 body {  
5   font: 100% $font-stack;  
6   color: $primary-color;  
7 }
```

variaveis.scss

SASS - Compressed
SASS

```
variaveis.css  
1 body {  
2   font: 100% Helvetica, sans-serif;  
3   color: #333; }  
4
```

variaveis.css

Sass: Características Principais

■ Variáveis

- na realidade as CSS também permitem a declaração de variáveis: **-- variável: valor**
 - No caso das CSS é necessário recorrer à função CSS (var)

```
<style>
  :root {--colorFg: orange}

  h1{color:var(--colorFg)}
  p{color:var(--colorFg)}
</style>
```

Âmbito da variável:
Onde pode ser aplicada

```
<body>
  <h1>Exemplo variáveis: h1</h1>
  <p>Exemplo variáveis: p</p>
```

Exemplo variáveis: h1

Exemplo variáveis: p

Sass: Características Principais

■ Scope das variáveis

- Uma variável declarada num seletor é visível apenas nesse seletor.

```
variaveis.scss

$primaryColor: darkorange;

body {
  $primaryColor: lightgray;
  background-color: $primaryColor;
}

p {
  width: 200px;
  color: white;
  background-color: $primaryColor;
}
```

variaveis.scss

```
variaveis.css

body {
  background-color: lightgray; }

p {
  width: 200px;
  color: white;
  background-color: darkorange; }
```

variaveis.css

```
<body>
  <p>VARIABLES SCOPE</p>
</body>
```

VARIABLES SCOPE

Sass: Características Principais

■ Scope das variáveis

- **!global** permite declarar uma variável global num seletor mas que é visível em todo o código



Sass: Características Principais

■ Nesting

- Permite a criação de uma clara hierarquia visual, declarando diretamente os seletores encaixados nos seletores que definem o contexto
 - Seletores contextuais em que o 1º nível de contexto é dado pelo **nest** que enquadra os outros seletores
 - A extensão desta dependência de seletores resulta em seletores cada vez mais específicos



Sass: Características Principais

■ @import / Partials

- Sass permite importar pequenos blocos de código *.scss (*partials*)
 - *Partial* é um ficheiro *.scss cujo nome inicia com **underscore (_)**.
 - O *underscore (_)* indica que se trata de um *partial* (a ser importado por outro ficheiro *.scss e não convertido diretamente num *.css)
 - Os *Sass partials* são usados com a diretiva @import (só é especificado o nome do *partial*)
 - Torna o código mais modular (reset, variáveis, funções, ...) e consequentemente mais fácil de manter.

_reset.scss

```
1 html,
2 body,
3 ul,
4 ol {
5     margin: 0;
6     padding: 0;
7 }
```

sass_1.scss

```
1 @import 'reset';
2
3 body {
4     font: 100% Helvetica, sans-serif;
5     background-color: #efefef;
6 }
```

sass_1.css

```
html,
body,
ul,
ol {
    margin: 0;
    padding: 0; }

body {
    font: 100% Helvetica, sans-serif;
    background-color: #efefef; }
```

Sass: Características Principais

■ @mixin

- Conjunto de declarações CSS que se pretende reutilizar
- A criação do grupo de declarações CSS é baseado na diretiva @mixin
 - É obrigatório atribuir um nome ao @mixin
 - É possível a passagem de parâmetros ao @mixin
 - O parâmetro deve ser definido com (\$nome_parâmetro)
 - @mixin é usado com base na diretiva @include
 - Os @mixin podem ser agrupados num *partial* de forma a libertar a *.scss principal

sass_1.scss

```
sass_1.scss
@mixin paragraph($radius) {
    background-color: orange;
    color: white;
    width: 200px;
    height: 100px;
    text-align: center;
    padding: 10px;
    border-radius: $radius;
}

p {@include paragraph(10px); }
```

sass_1.css

```
p {
    background-color: orange;
    color: white;
    width: 200px;
    height: 100px;
    text-align: center;
    padding: 10px;
    border-radius: 10px; }
```

sass_1.css

Sass: Características Principais

■ Inheritance (@extend)

- A diretiva **@extend** permite a partilha de declarações CSS entre diferentes seletores
 - Minimiza a repetição de código em diferentes seletores

```
sass_1.scss
.global {
  border: 4px solid darkorange;
  width: 50px;
  height: 30px;
  padding: 10px;
  text-align: center;
  background-color: darkorange;
  color: white;
}

.first {
  @extend .global;
  border-color: lightgray;
}

.second {
  @extend .global;
  border-color: orange;
}

.third {
  @extend .global;
  border-color: darkgray;
}
```

```
sass_1.css
.global, .first, .second, .third {
  border: 4px solid darkorange;
  width: 50px;
  height: 30px;
  padding: 10px;
  text-align: center;
  background-color: darkorange;
  color: white;
}

.first {
  border-color: lightgray;
}

.second {
  border-color: orange;
}

.third {
  border-color: darkgray;
}
```

```
<body>
  <div class="first">1st</div>
  <div class="second">2nd</div>
  <div class="third">3rd</div>
</body>
```

Visual representation of the resulting HTML elements:

1st

2nd

3rd

Tecnologias Web 2023/2024
Simão Paredes sparedes@isec.pt

408

Sass: Características Principais

■ @extend

- cada @extend pode ser aplicado a apenas um seletor

*.SCSS

```
.error{
  color:red;
}

.critical-error{
  @extend .error;
  font-weight:bold;
  border:2px solid red;
}
```

```
.minor-error{ background-color:gray;}

.critical-error{
  @extend .error .minor-error;
  font-weight:bold;
  border:2px solid red;}

```

- múltiplos @extend no mesmo seletor

```
.critical-error{
  @extend .error;
  @extend .minor-error;
  font-weight:bold;
  border:2px solid red;}

```

- encadeamento de @extend

```
.main-error{
  @extend .critical-error;
  font-size: 125%;}

```

409

Sass: Características Principais

■ @extend

- É possível definir um seletor exclusivamente para ser utilizado com *@extend*
 - Iniciam-se por %
 - Não podem ser compilados diretamente para CSS, só são compilados se forem alvo de um

@extend

*.SCSS

```
%error{color:red;}

.critical-error{
  @extend %error;
  font-weight:bold;
  border:2px solid red;}
```

*.CSS

```
.critical-error {
  color: red; }

.critical-error {
  font-weight: bold;
  border: 2px solid red; }
```

- O @extend de um seletor inexistente, provoca um erro e inibe a geração do respetivo *.css
 - **!optional** permite contornar esta limitação

```
.critical-error{
  @extend %error;
  @extend %critical !optional;
  font-weight:bold;
  border:2px solid red;}
```

Sass: Características Principais

■ @mixin vs. @extend

@mixin	@extend
reutilização código	reutilização código
permite passagem de parâmetros	não permite parâmetros
origina menos seletores CSS	Origina mais seletores CSS
repetição de declarações	minimiza repetição de declarações
	limitação no funcionamento com @media
tipicamente mais rápida que o @extend	

<https://tech.bellycard.com/blog/sass-mixins-vs-extends-the-data/>

Sass: Características Principais

■ @media

- Permite a definição de CSS media queries
- Possibilita o *nest* das CSS *media queries*, no entanto no código *.css a definição das *media queries* é sempre efetuada no top level

```
#main {  
  width:$content-width;  
  @media only screen and (max-width: 960px){  
    width:auto;  
    max-width:960px;  
  }  
}
```

*.scss

*.css

```
@media only screen and (max-width: 960px) {  
  #main {  
    width: auto;  
    max-width: 960px; } }  
}
```

Sass: Características Principais

■ @media

- Não é possível efetuar o @extend de seletores definidos fora da media query

```
@media screen{  
  
  .error{color:red;}  
  
  .critical-error{  
    @extend .error;  
    font-weight:bold;  
    border:2px solid red;}  
}
```

Sass: Características Principais

Operadores

Conjunto de operadores matemáticos:

- + , - , * , / , % ...

sass_1.scss

```
.container { width: 100%; }

@mixin global{
  color:white;
  text-align:center;
  height:50px;
}

article[role="main"] {
  float: left;
  width: 600px / 960px * 100%;
  background-color:lightgray;
  @include global;
}

aside[role="complementary"] {
  float: right;
  width: 300px / 960px * 100%;
  background-color:darkorange;
  @include global;
}
```

sass_1.css

```
sass_1.css
@charset "UTF-8";
.container {
  width: 100%; }

article[role="main"] {
  float: left;
  width: 62.5%;
  background-color: lightgray;
  color: white;
  text-align: center;
  height: 50px; }

aside[role="complementary"] {
  float: right;
  width: 31.25%;
  background-color: darkorange;
  color: white;
  text-align: center;
  height: 50px; }
```

```
<body>
  <article role="main">Main</article>
  <aside role="complementary">Aside</aside>
</body>
```

Main

Aside

Sass: Características Principais

Operadores

sass_1.scss

```
$container-width: 100%;

.container {
  width: $container-width;
}

.col-4 {
  width: $container-width / 4;
  height:60px;
  background-color:darkorange;
  color:white;
}
```

operador	obs.
+	adição
-	subtração
/	divisão
*	multiplicação
%	resto divisão inteira
==	igual
!=	diferente

<https://scotch.io/tutorials/getting-started-with-sass>

sass_1.css

```
sass_1.css
@charset "UTF-8";
.container {
  width: 100%; }

.col-4 {
  width: 25%;
  height: 60px;
  background-color: darkorange;
  color: white; }
```

```
<link href="sass_1.css" rel="stylesheet" type="text/css" />
</head>
<body>
  <div class="col-4">Mathematical Operators</div>
</body>
```

Mathematical Operators

Questões Exame (Época Normal 2022/23)

Questões Exame (Época Normal 2022/23)

```
<body>
  <div>
    <p>Electric <span>Vehicles <span>Zero Emissions</span></span></p>
    <ul>
      <li>Fiat 500</li>
      <li class="item">Mercedes EQS</li>
      <li>Volvo<span>XC 40</span></li>
      <li>Peugeot e-2008</li>
    </ul>
  </div>
</body>
```

27. Considerando a estrutura HTML representada na figura 1, indique **justificando** quais os elementos que são afetados pelos seguintes seletores:

A. `[class]{color:green}`

Electric Vehicles Zero Emissions

- Fiat 500
- Mercedes EQS
- VolvoXC 40
- Peugeot e-2008

```
<body>
  <div>
    <p>Electric <span>Vehicles <span>Zero Emissions</span></span></p>
    <ul>
      <li>Fiat 500</li>
      <li class="item">Mercedes EQS</li>
      <li>Volvo<span>XC 40</span></li>
      <li>Peugeot e-2008</li>
    </ul>
  </div>
</body>
```

27. Considerando a estrutura HTML representada na figura 1, indique justificando quais os elementos que são afetados pelos seguintes seletores:

B. `li:nth-child(3) span{color:blue}`

Electric Vehicles Zero Emissions

- Fiat 500
- Mercedes EQS
- VolvoXC 40
- Peugeot e-2008

```
<body>
  <div>
    <p>Electric <span>Vehicles <span>Zero Emissions</span></span></p>
    <ul>
      <li>Fiat 500</li>
      <li class="item">Mercedes EQS</li>
      <li>Volvo<span>XC 40</span></li>
      <li>Peugeot e-2008</li>
    </ul>
  </div>
</body>
```

27. Considerando a estrutura HTML representada na figura 1, indique justificando quais os elementos que são afetados pelos seguintes seletores:

C. `span span{color:orange}`

Electric Vehicles Zero Emissions

- Fiat 500
- Mercedes EQS
- VolvoXC 40
- Peugeot e-2008

```
<body>
  <div>
    <p>Electric <span>Vehicles <span>Zero Emissions</span></span></p>
    <ul>
      <li>Fiat 500</li>
      <li class="item">Mercedes EQS</li>
      <li>Volvo<span>XC 40</span></li>
      <li>Peugeot e-2008</li>
    </ul>
  </div>
</body>
```

27. Considerando a estrutura HTML representada na figura 1, indique justificando quais os elementos que são afetados pelos seguintes seletores:

D. `span > span{color:red}`

Electric Vehicles **Zero Emissions**

- Fiat 500
- Mercedes EQS
- VolvoXC 40
- Peugeot e-2008

Fim!

Sass: Características Principais

■ Funções

■ *Dois tipos de Funções: Built-in functions + Custom Functions*

■ *Built-in Functions:*

■ Conjunto de funções disponibilizadas pelo SASS

<http://sass-lang.com/documentation/Sass/Script/Functions.html>

■ Algumas das funções mais comuns:

■ *darken(\$color, amount)*

- permite escurecer uma cor, tem a grande vantagem de já se conhecer a cor a escurecer

```
a {
  color:$link-color;
  &:hover{
    color:darken($link-color,15%);}
}
```

■ *lighten(\$color, amount)*

- O efeito contrário, torna uma cor mais clara

■ *transparentize(\$color; amount)*

■ *opacify(\$color;amount)*

Sass: Características Principais

■ Custom Funtions

■ *@function*

- Permite múltiplos parâmetros (possibilita especificação dos valores por defeito)
- *@return* para retornar o valor

```
@function col-width($columns:12, $page-width:100%, $gap:1%){
  @return ($page-width - $gap*($columns - 1))/ $columns;
}
```

sass_1.scss

■ Chamada à função

```
#content {
  float:left;
  width:6*col-width(8);
}

#sidebar {
  float:right;
  width:2*col-width(8);
}
```

sass_1.scss

Sass: Características Principais

■ Condições

- @if (...) {...} / @else if (...) {...} / @else {...}



Sass: Características Principais

■ @if

- Podem condicionar de uma forma muito direta toda a formatação (ex.: paleta de cores)



Sass: Características Principais

■ Ciclos

■ @for

- Repetir o processamento um número pré-determinado de vezes
- **#{} (interpolation syntax)**
 - Notação que permite utilizar variáveis em nomes de seletores, propriedades e respetivos valores.

```
@for $i from 1 through 6 {  
  .col-#{ $i } {  
    width: $i * 2em;  
  }  
}
```

*.scss

```
.col-1 {  
  width: 2em; }  
  
.col-2 {  
  width: 4em; }  
  
.col-3 {  
  width: 6em; }  
  
.col-4 {  
  width: 8em; }  
  
.col-5 {  
  width: 10em; }  
  
.col-6 {  
  width: 12em; }
```

*.css

Sass: Características Principais

■ Ciclos

■ @each

- O mais versátil e o mais frequentemente utilizado
 - Percorrer uma lista
 - Percorrer um map

```
$font-sizes: (tiny:8px, small:11px, medium: 13px, large:16px);  
  
@each $name, $size in $font-sizes {  
  .#{$name} {  
    font-size: $size;  
  }  
}
```

*.scss

```
.tiny {  
  font-size: 8px; }  
  
.small {  
  font-size: 11px; }  
  
.medium {  
  font-size: 13px; }  
  
.large {  
  font-size: 16px; }
```

*.css

Sass: Características Principais

■ Ciclos

■ @while

- Exige que a variável que controla o ciclo seja incrementada, caso contrário origina-se um ciclo infinito

```
$j:2;

@while ($j<=8){
  .picture-#{ $j }{
    width:$j * 10%;
  }
  $j:$j+2;
}
```

*.scss

```
.picture-2 {
  width: 20%; }

.picture-4 {
  width: 40%; }

.picture-6 {
  width: 60%; }

.picture-8 {
  width: 80%; }
```

*.css

Sass

■ Apresentadas **apenas** as principais características deste pré-processador

- http://sass-lang.com/documentation/file.SASS_REFERENCE.html#syntax

■ Vantagens

- Organização/Modularidade do código
- Refactoring
- Funcionalidades não disponíveis nas CSS
 - Modularidade, reutilização de código; funções; operações matemáticas; ...
 - ...

■ Desvantagens

- Curva de aprendizagem para implementar as novas features
- ...

Compressão CSS

CSS Minify

Compressão CSS

- CSS minify
 - O objetivo principal é a redução do peso dos ficheiros CSS.
 - Criado apenas um *.css file de forma a minimizar os HTTP Requests
 - Pode atingir uma redução de 20% relativamente ao código original
 - Download mais rápido
 - Torna mais confusa a leitura do código
 - Dificulta cópias ou processos de *reverse engineering*.

Before:

```
.ui-helper-hidden {  
    display: none;  
}  
.ui-helper-hidden-accessible {  
    border: 0;  
    overflow: hidden;  
}
```

After:

```
.ui-helper-hidden{display:none}.ui-helper-hidden-accessible{border:0; overflow: hidden;}
```

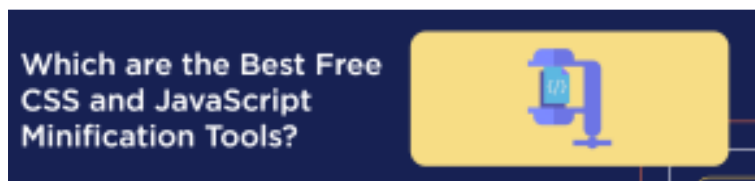
■ CSS minify

■ Remoção:

- Comentários
- Espaços em branco e novas linhas
- Indentações

■ Optimizar (reduzir) a designação dos seletores de ID, class, ...

■ Ferramentas:



<https://wp-rocket.me/blog/best-free-css-and-javascript-minification-tools/>

Bootstrap5

Frameworks CSS

Bootstrap

■ Bootstrap

- É um *framework* dedicado ao *Front-End development*.
 - Bootstrap5 é a nova versão lançada em 2021.
- Um conjunto de características que justificam a sua popularidade na implementação de aplicações Web
 - Responsive
 - Particularmente adaptado para ambientes *mobile*
 - Reduz o tempo de desenvolvimento
 - Assegura consistência
 - Assegura compatibilidade *cross browser*
 - Configurável
 - Possui uma comunidade de utilizadores muito alargada

Bootstrap

■ Instalação

- 2 opções mais vulgarmente utilizadas:
 - Download

Compiled CSS and JS

Download ready-to-use compiled code for **Bootstrap v5.0.2** to easily drop into your project, which includes:

- Compiled and minified CSS bundles (see [CSS files comparison](#))
- Compiled and minified JavaScript plugins (see [JS files comparison](#))

This doesn't include documentation, source files, or any optional JavaScript dependencies like Popper.

Download

- Incorporação via CDN

CDN via jsDelivr

Skip the download with [jsDelivr](#) to deliver cached version of Bootstrap's compiled CSS and JS to your project.



<https://getbootstrap.com/docs/5.0/getting-started/download/>

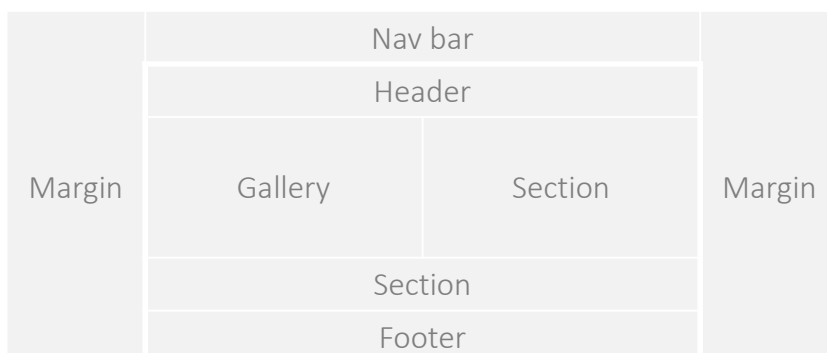
Grid System

Bootstrap5

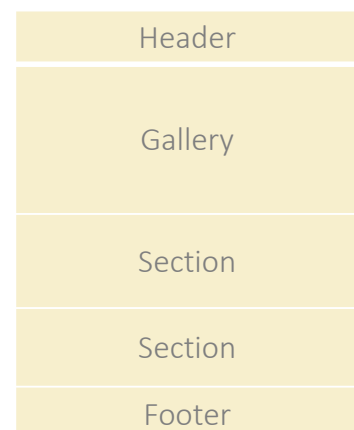
Bootstrap: Grid System

- A construção de layouts é baseado no **Grid System**
 - Permite criar um *responsive layout*
 - Assume duas formas distintas de estruturar a disposição de informação

*Layout 1
(desktop)*



*Layout 2 – 1 column
(tablet / mobile)*



“ ... mobile-first flexbox grid to build layouts of all shapes and sizes thanks to a twelve column system, five default responsive tiers,”

<https://getbootstrap.com/docs/>

Bootstrap: Grid System

- A estrutura base disponibilizada pelo Bootstrap para implementação da Grid System é baseada em 3 conceitos chave:

- Container (C)

- O *wrapper* que permite o alinhamento horizontal do conteúdo e define a forma como a aplicação se adapta a diferentes viewports

- Exemplo: **.container-fluid** permite uma *width:100%* para todos os viewports

- Row (R)

- *Wrapper* de colunas, permite criar grupos de colunas.

- Columns

- Podem ser definidas 12 colunas por linha, às quais pode ser aplicado um agrupamento (span) de colunas (exemplo: **.col-4**).

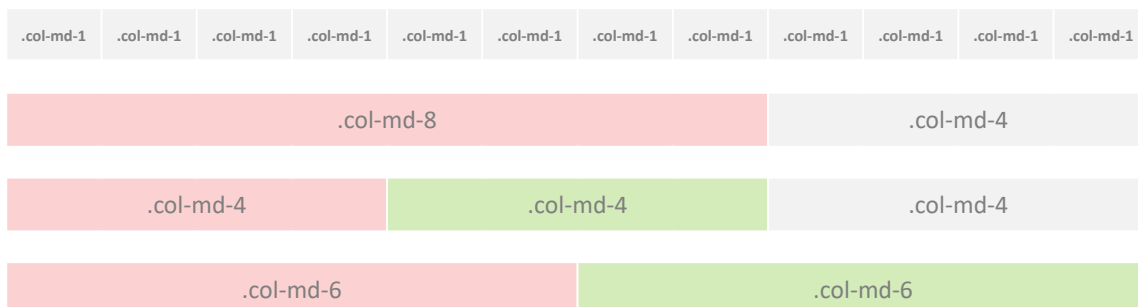
- É um elemento flexível cujo redimensionamento é automático.



Bootstrap: Grid System

- Columns

- 12 colunas base, as quais podem ser sujeitas a um agrupamento horizontal (*spanning*)



- **.col-md-***

- cria uma coluna com um agrupamento de colunas definido por *
 - **md** especifica o breakpoint onde a coluna altera a sua largura
 - md → $screen \geq 768px$ significa que as colunas irão ocupar 100% da largura disponível para ecrãs com largura inferior aos 768px (Layout 2)

Bootstrap: Grid System

■ Breakpoints

■ definidos tendo por base:

- múltiplos de 12
- são representativos de um subconjunto de dispositivos e dimensões de viewport comuns
 - Não são específicos para nenhum dispositivo, mas asseguram que um funcionamento consistente para todos os dispositivos no mercado.
- definem em que classe de dispositivos se inicia a configuração definida:
 - Permitem escalar o layout definido para viewports de dimensão superior
 - Pelo contrário, para viewports de dimensão inferior (mobile), o layout 2 é assumido.

Breakpoints		
X-small	-	0-576px
Small	sm	≥ 576px
Medium	md	≥ 768px
Large	lg	≥ 992px
Extra Large	xl	≥ 1200px
Extra Extra large	xxl	≥ 1400px

Layout 2 (não permite layout 1) até uma dimensão máxima de 576px

Bootstrap: Grid System

■ Breakpoints

- <https://getbootstrap.com/docs/5.3/layout/breakpoints/#core-concepts>

```
// X-Small devices (portrait phones, less than 576px)
// No media query for `xs` since this is the default in Bootstrap

// Small devices (landscape phones, 576px and up)
@media (min-width: 576px) { ... }

// Medium devices (tablets, 768px and up)
@media (min-width: 768px) { ... }

// Large devices (desktops, 992px and up)
@media (min-width: 992px) { ... }

// X-Large devices (large desktops, 1200px and up)
@media (min-width: 1200px) { ... }

// XX-Large devices (larger desktops, 1400px and up)
@media (min-width: 1400px) { ... }
```


Bootstrap: Grid System

■ Configurações Comuns:

■ 2 colunas

- As 2 colunas **iniciam-se $\geq 768\text{px}$ e escalam** para viewports de dimensão superior



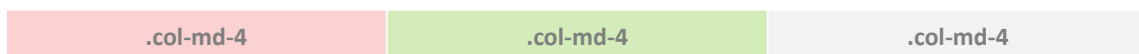
```
<div class="container">
  <div class="row">
    <div class="col-md-8">.col-md-8</div>
    <div class="col-md-4">.col-md-4</div>
  </div>
</div>
```

- para viewports inferiores a 768px, as colunas irão assumir 100% da largura disponível (layout 2)

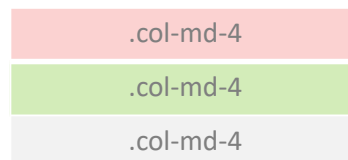


Bootstrap: Grid System

■ 3 colunas



- $vp \leq 768\text{px}$



```
<div class="container">
  <div class="row">
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
    <div class="col-md-4">.col-md-4</div>
  </div>
</div>
```

Bootstrap: Grid System

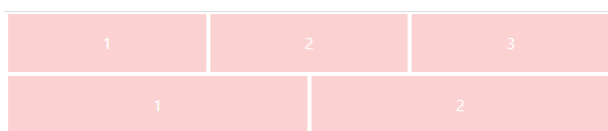
- Caso sejam utilizadas classes não baseadas nos *break points* , **altera-se o comportamento responsive:**

- Em *viewports* reduzidos mantém-se o número de colunas original

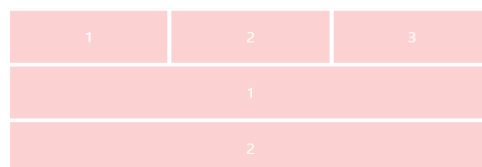
```
<div class="container-fluid">
  <div class="row">
    <div class="col">1</div>
    <div class="col">2</div>
    <div class="col">3</div>
  </div>
  <div class="row">
    <div class="col-sm">1</div>
    <div class="col-sm">2</div>
  </div>
</div>
```

class **col** mantém a definição original

class **col-sm** passam a estar sobrepostas (layout2)



Redução do viewport



- tipicamente utilizam-se classes baseadas nos *breakpoints* e com dimensão definida
- exemplo: **.col-md-4**; **.col-md-8**; **.col-sm-6**, ...

Bootstrap: Grid System

■ Containers

- Existem diversas classes pré-definidas para estabelecer o comportamento dos containers

C

- .container → 100% width ≤ 540px
- .container-sm → 100% width ≤ 540px
- .container-md → 100% width ≤ 720px
- .container-lg → 100% width ≤ 960px
- .container-xl → 100% width ≤ 1140px
- .container-xxl → 100% width ≤ 1320px
- .container-fluid → 100% width

```
<div class="container">.container</div>
<div class="container-sm">.container-sm</div>
<div class="container-md">.container-md</div>
<div class="container-lg">.container-lg</div>
<div class="container-xl">.container-xl</div>
<div class="container-xxl">.container-xxl</div>
<div class="container-fluid">.container-fluid</div>
```

Bootstrap: Grid System

Containers

- Responsive para os diversos breakpoints

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
<code>.container</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-sm</code>	100%	540px	720px	960px	1140px	1320px
<code>.container-md</code>	100%	100%	720px	960px	1140px	1320px
<code>.container-lg</code>	100%	100%	100%	960px	1140px	1320px
<code>.container-xl</code>	100%	100%	100%	100%	1140px	1320px
<code>.container-xxl</code>	100%	100%	100%	100%	100%	1320px
<code>.container-fluid</code>	100%	100%	100%	100%	100%	100%

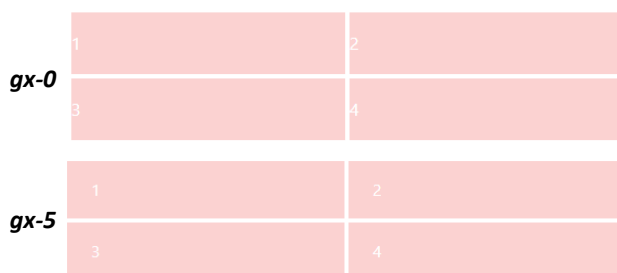
<https://getbootstrap.com/docs/5.0/layout/containers/>

Bootstrap: Grid System

gutter

- gx** define o padding

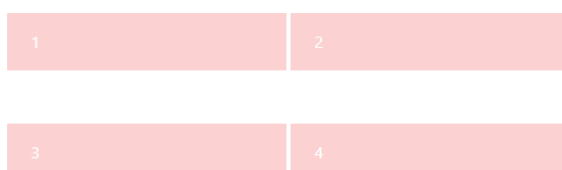
- gx-0 a gx-5



```
<div class="container">
  <div class="row gx-5">
    <div class="col-6">1</div>
    <div class="col-6">2</div>
    <div class="col-6">3</div>
    <div class="col-6">4</div>
  </div>
</div>
```

- gy** define margin vertical

- gy-0 a gy-5



- g-5** é equivalente a **gx-5; gy-5**

```
<div class="container">
  <div class="row gx-5 gy-5">
    <div class="col-6">1</div>
    <div class="col-6">2</div>
    <div class="col-6">3</div>
    <div class="col-6">4</div>
  </div>
</div>
```

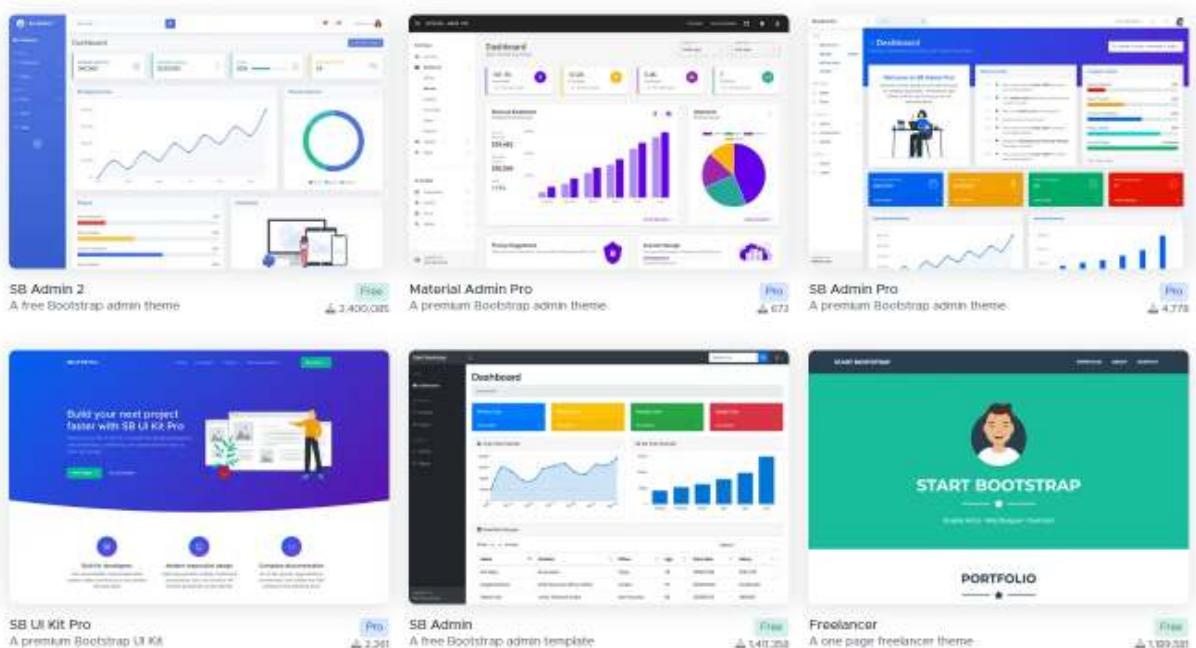
Templates

Bootstrap5

Bootstrap: Templates

- Existe um conjunto alargado de templates disponíveis para download

■ <https://startbootstrap.com/>



Components

Bootstrap5

Bootstrap: Components

- Elementos com formatação/funcionalidade definidas (reutilização)
- Alguns componentes permitem configurações adicionais

■ <https://getbootstrap.com/docs/5.0/components/accordion/>

✓ Components

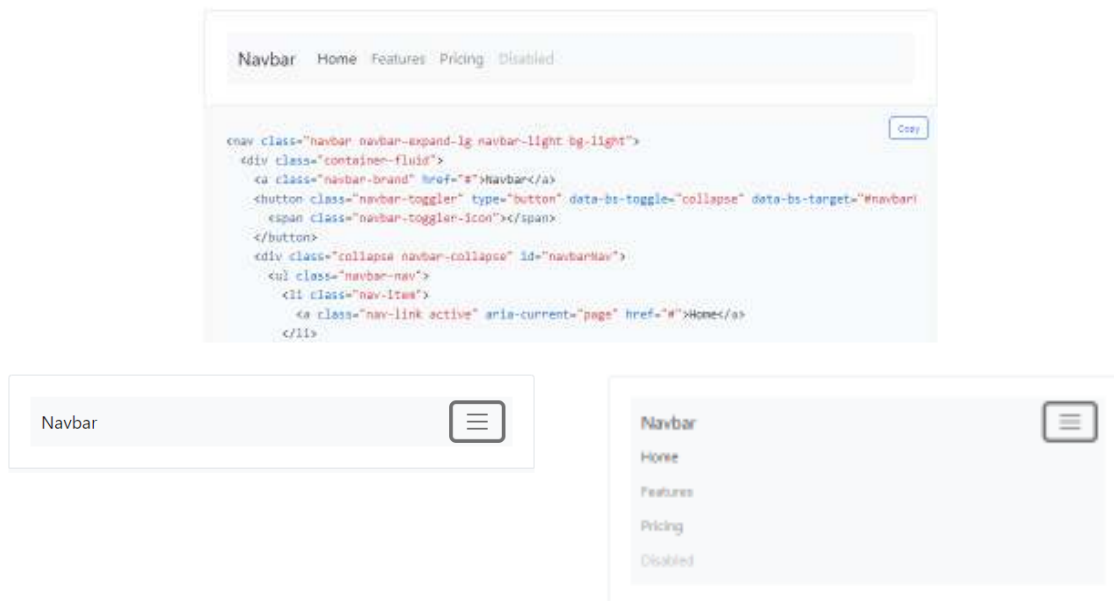
Accordion
Alerts
Badge
Breadcrumb
Buttons
Button group
Card
Carousel
Close button
Collapse
Dropdowns
List group
Modal
Navs & tabs
Navbar
Offcanvas
Pagination
Popovers
Progress
Scrollspy
Spinners



Bootstrap: Components

■ *Navbar*

- Barra de navegação, totalmente configurável, responsive em que nos *viewports* de menor dimensão assume um funcionamento em *sandwich button*



Bootstrap: Components

■ *Buttons*

- É disponibilizado um conjunto diversificado de botões, os quais podem ser diretamente utilizados
- <https://getbootstrap.com/docs/4.0/components/buttons/>

Buttons



```
<div class="container" >
  <div class="row g-5">
    <h1>Buttons</h1>
    <div>
      <button type="button" class="btn btn-primary">Primary</button>
      <button type="button" class="btn btn-secondary">Secondary</button>
      ...
    </div>
  </div>
</div>
```

Bootstrap: Components

Buttons

- São disponibilizadas várias classes que permitem definir cor e dimensão dos botões

Buttons

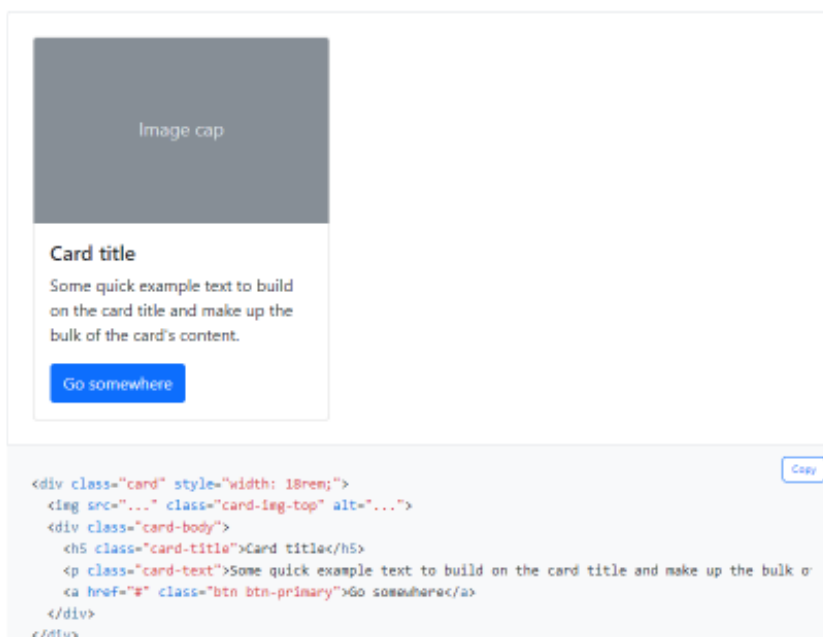


```
<div class="container" >
  <div class="row g-5">
    <h1>Buttons</h1>
    <div>
      <button type="button" class="btn btn-primary btn-lg">Primary</button>
      <button type="button" class="btn btn-secondary btn-sm">Secondary</button>
      <button type="button" class="btn btn-success">Success</button>
    </div>
  </div>
</div>
```

Bootstrap: Components

Cards

- Componente que permite associar de forma direta imagem / título / conteúdo textual



Bootstrap: Components

■ Carousel

- A inserção de um slide show é imediata e pode ser configurada de diversas formas

■ <https://getbootstrap.com/docs/5.1/components/carousel/>

```
<div id="carouselExampleControls" class="carousel slide" >
  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    ....
    <a class="carousel-control-prev" href="#carouselExampleControls" role="button" data-slide="prev">
      <span class="carousel-control-prev-icon" aria-hidden="true"></span>
      <span class="sr-only">Previous</span>
    </a>
    <a class="carousel-control-next" href="#carouselExampleControls" role="button" data-slide="next">
      <span class="carousel-control-next-icon" aria-hidden="true"></span>
      <span class="sr-only">Next</span>
    </a>
  </div>
```



Bootstrap: Components

■ Navs & Tabs

```
<ul class="nav">
  <li class="nav-item">
    <a class="nav-link active" aria-current="page" href="#">Active</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled" href="#" tabindex="-1" aria-disabled="true">Disabled</a>
  </li>
</ul>
```

Active Link Link Disabled

```
<ul class="nav nav-tabs">
  <li class="nav-item">
    <a class="nav-link active" aria-current="page" href="#">Active</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
  </li>
  <li class="nav-item">
    <a class="nav-link disabled" href="#" tabindex="-1" aria-disabled="true">Disabled</a>
  </li>
</ul>
```

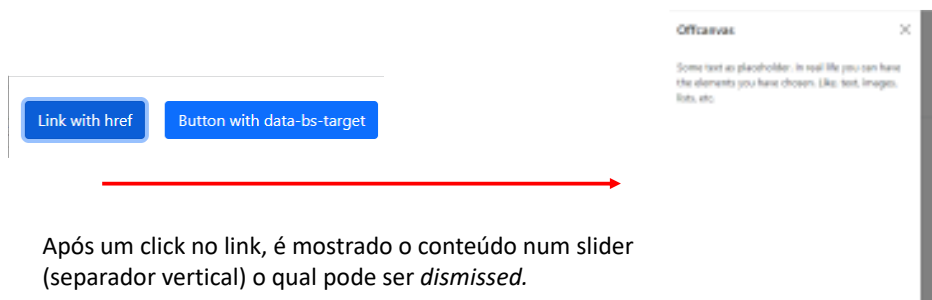
Active Link Link Disabled

Bootstrap: Components

Offcanvas

- Novo componente no Bootstrap5, permite a visualização de conteúdo (definido offcanvas) através de um *slider*

```
<a class="btn btn-primary" data-bs-toggle="offcanvas" href="#offcanvasExample" role="button">Link with href </a>
<button class="btn btn-primary" type="button" data-bs-toggle="offcanvas" data-bs-target="#offcanvasExample">
  Button with data-bs-target
</button>
<div class="offcanvas offcanvas-start" tabindex="-1" id="offcanvasExample" aria-labelledby="offcanvasExampleLabel">
  <div class="offcanvas-header">
    <h3 class="offcanvas-title" id="offcanvasExampleLabel">Offcanvas</h3>
    <button type="button" class="btn-close text-reset" data-bs-dismiss="offcanvas" aria-label="Close"></button>
  </div>
  <div class="offcanvas-body">
    <div> Some text as placeholder. In real life you can have the elements you have chosen. Like, text, images, lists, etc.</div>
  </div>
</div>
```



Após um click no link, é mostrado o conteúdo num slider (separador vertical) o qual pode ser *dismissed*.

Bootstrap: Components

Accordion

- Muito útil para a implementação de FAQ's

Accordion Item #1	▼
Accordion Item #2	▼
Accordion Item #3	▼

Accordion Item #1	▲
This is the first item's accordion body. It is shown by default, until the collapse plugin adds the appropriate classes that we use to style each element. These classes control the overall appearance, as well as the showing and hiding via CSS transitions. You can modify any of this with custom CSS or overriding our default variables. It's also worth noting that just about any HTML can go within the <code>.accordion-body</code> , though the transition does limit overflow.	
Accordion Item #2	▼
Accordion Item #3	▼

```
<div class="accordion" id="accordionExample">
  <div class="accordion-item">
    <h2 class="accordion-header" id="headingOne">
      <button class="accordion-button" type="button" data-bs-toggle="collapse" data-bs-target="#collapseOne" aria-expanded="true"
        aria-controls="collapseOne"> Accordion Item #1 </button></h2>
    <div id="collapseOne" class="accordion-collapse collapse show" aria-labelledby="headingOne" data-bs-parent="#accordionExample">
      <div class="accordion-body">
        <strong>This is the first item's accordion body.</strong> It is shown by default, ....
      </div>
    </div>
  </div>
  ....
</div>
```

Alerts

- Reforçar a interação com o utilizador.

```
<div class="alert alert-primary" role="alert">
  A simple primary alert—check it out!
</div>
```



- Eliminar o *alert*

```
<div class="alert alert-warning alert-dismissible fade show">
  Dismiss!!
  <button type="button" class="btn-close" data-bs-dismiss="alert" ></button>
</div>
```



Utilities

Bootstrap5

Bootstrap: Utilities

- *Utilities* agrega um conjunto de classes específicas para definir texto, dimensões, cores, cores de fundo, espaçamentos,

- <https://getbootstrap.com/docs/5.0/utilities/>

▼ Utilities

API

Background

Borders

Colors

Display

Flex

Float

Interactions

Overflow

Position

Shadows

Sizing

Spacing

Text

Vertical align

Visibility

.bg-primary

.bg-secondary

.bg-success

.bg-danger

.bg-warning

.bg-info

.bg-light

.bg-dark

Bootstrap: Utilities

■ *Utilities: Spacing*

■ Notação:

- {property}{sides}-{size} para *breakpoint* xs
- {property}{sides}-{breakpoint}-{size} para *breakpoints* sm, md, lg, xl e xxl.

■ *property*

- *margin: m*
- *padding: p*

■ *sides*

t - for classes that set *margin-top* or *padding-top*
b - for classes that set *margin-bottom* or *padding-bottom*
s - (start) for classes that set *margin-left* or *padding-left*
e - (end) for classes that set *margin-right* or *padding-right*
x - for classes that set both **-left* and **-right*
y - for classes that set both **-top* and **-bottom*
blank - for classes that set a *margin* or *padding* on all 4 sides

size

0 - for classes that eliminate the *margin* or *padding* by setting it to 0
1 - (by default) for classes that set the *margin* or *padding* to \$spacer * .25
2 - (by default) for classes that set the *margin* or *padding* to \$spacer * .5
3 - (by default) for classes that set the *margin* or *padding* to \$spacer
4 - (by default) for classes that set the *margin* or *padding* to \$spacer * 1.5
5 - (by default) for classes that set the *margin* or *padding* to \$spacer * 3
auto - for classes that set the *margin* to auto

<https://getbootstrap.com/docs/5.0/utilities/spacing/>

Bootstrap: Utilities

■ Utilities: Spacing

■ Padding:

```
<div>  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

■ horizontal/vertical

```
<div class="px-5 py-3" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

■ Todas as direções

```
<div class="p-5" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

Bootstrap: Utilities

■ Utilities: Spacing

■ margin:

```
<div class="p-1" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

■ exemplo: (*mt* → *margin top*; *m* → *margin*)

```
<div class="p-1" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
<p class="mt-3">  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde incidunt nobis doloremque exercitationem nihil accusantium voluptatibus? Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta temporibus voluptates suscipit ab.

■ Utilities: Alinhamento de texto

```
<div class="p-2 text-center" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde
incidunt nobis doloremque exercitationem nihil accusantium voluptatibus?
Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta
temporibus voluptates suscipit ab.

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Voluptatum unde
incidunt nobis doloremque exercitationem nihil accusantium voluptatibus?
Doloribus optio doloremque nihil veritatis, magni, blanditiis modi dicta
temporibus voluptates suscipit ab.

```
<div class="w-25 p-3 text-center mx-auto my-4 border" >  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
<p>  
  Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipit ab.</p>  
</div>
```

Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Voluptatum unde incidunt nobis doloremque exercitationem
nihil accusantium voluptatibus? Doloribus optio doloremque
nihil veritatis, magni, blanditiis modi dicta temporibus
voluptates suscipit ab.

Lorem ipsum dolor sit amet, consectetur adipisicing elit.
Voluptatum unde incidunt nobis doloremque exercitationem
nihil accusantium voluptatibus? Doloribus optio doloremque
nihil veritatis, magni, blanditiis modi dicta temporibus
voluptates suscipit ab.

- **w-25** (width 25%) {25, 50, 75, 100}
- **p-3** (padding nível 3) {0,1,2,3,4,5}
- **text-center** (texto com alinhamento centrado)
- **mx-auto** (margens horizontais automaticas, elemento centrado)
- **my-4** (margens verticais de nível 4) {0,1,2,3,4,5}
- **border**

Images

Bootstrap5

■ Imagens Fluídas

- O Bootstrap possui uma class especifica para tornar uma imagem flexível

- imagem sem formatação associada

```
<div class="w-25 p-3 text-center mx-auto my-4 border" >  
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipi ab.</p>  
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipi ab.</p>  
    
</div>
```



- imagem flexível com capacidade de ajustar automaticamente ao espaço disponível

```
<div class="w-25 p-3 text-center mx-auto my-4 border" >  
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipi ab.</p>  
  <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. ... suscipi ab.</p>  
    
</div>
```



Tables

Bootstrap5

Bootstrap: Tables

■ Tabelas

- Estão disponíveis um conjunto de classes associadas às tabelas que permitem uma formatação completa deste elemento
- A estrutura HTML permanece inalterada

```
<div class="mx-auto w-50">
  <table class="table">
    <thead>
      <tr>
        <th
          scope="col">#</th>
        <th
          scope="col">First</th>
        <th
          scope="col">Last</th>
        <th
          scope="col">Handle</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        ...
      </tr>
      ...
    </tbody>
  </table>
</div>
```

#	First	Last	Handle
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry the Bird		@twitter

Bootstrap: Tables

■ Class específicas para tabelas

```
<div class="mx-auto w-50">
  <table class="table table-light table-striped table-hover">
    ...
  </table>
</div>
```

■ *table-light*

- Cor da tabela (tema)

■ *table-striped*

- Linhas alternadas

■ *table-hover*

- Efeito de sobreposição do cursor do rato



Class	Heading	Heading
Default	Cell	Cell
Primary	Cell	Cell
Secondary	Cell	Cell
Success	Cell	Cell
Danger	Cell	Cell
Warning	Cell	Cell
Info	Cell	Cell
Light	Cell	Cell
Dark	Cell	Cell

#	First	Last	Handle
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry the Bird		@twitter

Forms

Bootstrap5

Bootstrap: Forms

- São disponibilizados diversos campos/elementos de formulários.
- Todos os elementos podem ser configuráveis

First name: Simao
Last name: Paredes
Username: @
City:
State: Choose...
Zip:
☐ Agree to terms and conditions
Submit form

- Podem ser definidas validações locais, as quais também podem ser configuráveis

First name: Simao ✓ Look good!
Last name: Paredes ✓ Look good!
Username: @ ✓
City:
State: Choose...
Zip:
☐ Agree to terms and conditions
Submit form

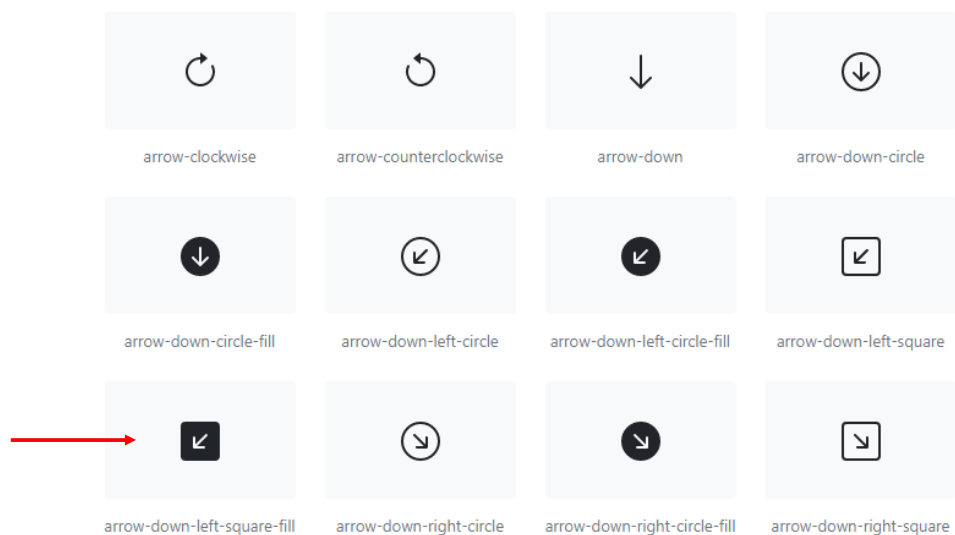
Validação do formulário é efetuada tendo por base duas pseudo-classes **:invalid** e **:valid**, as quais são aplicadas a todas as <tags> de inserção de dados.

Icons

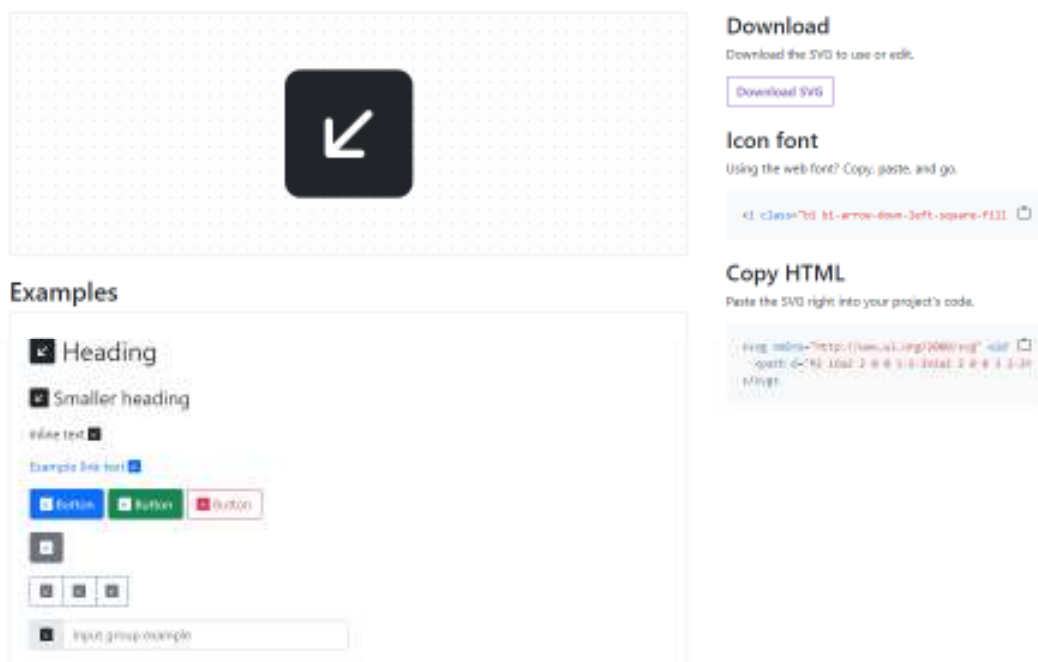
Bootstrap5

Bootstrap: Icons

- São disponibilizadas centenas de icons implementados em SVG
 - <https://icons.getbootstrap.com/>
 - A inserção destes icons é direta, apenas é necessária a sua seleção e cópia do respetivo código SVG



- Após a seleção do *icon* pretendido, é possível editar diretamente o respetivo código SVG



Customize

Bootstrap5

Bootstrap: Customize

- As formatações do bootstrap podem ser alteradas de forma a corresponder a necessidades específicas de uma aplicação.
 - este processo de customization tem várias opções disponíveis, sempre suportados pelos ficheiros Sass (*.scss) originais.
 - <https://getbootstrap.com/docs/5.0/customize/overview/>

▼ Customize

Overview

Sass

Options

Color

Components

CSS variables

Optimize

Sass

[View on GitHub](#)

Utilize our source Sass files to take advantage of variables, maps, mixins, and functions to help you build faster and customize your project.

Bootstrap: Customize

- Exemplo:
 - Modificação do valor base das variáveis Sass, após o que são gerados os *.css com as alterações produzidas.

Variable	Values	Description
<code>\$spacer</code>	<code>1rem</code> (default), or any value <code>> 0</code>	Specifies the default spacer value to programmatically generate our spacer utilities .
<code>\$enable-rounded</code>	<code>true</code> (default) or <code>false</code>	Enables predefined <code>border-radius</code> styles on various components.
<code>\$enable-shadows</code>	<code>true</code> or <code>false</code> (default)	Enables predefined decorative <code>box-shadow</code> styles on various components. Does not affect <code>box-shadows</code> used for focus states.
<code>\$enable-gradients</code>	<code>true</code> or <code>false</code> (default)	Enables predefined gradients via <code>background-image</code> styles on various components.
<code>\$enable-transitions</code>	<code>true</code> (default) or <code>false</code>	Enables predefined <code>transitions</code> on various components.
<code>\$enable-reduced-motion</code>	<code>true</code> (default) or <code>false</code>	Enables the prefers-reduced-motion media query , which suppresses certain animations/transitions based on the users' browser/operating system preferences.
<code>\$enable-grid-classes</code>	<code>true</code> (default) or <code>false</code>	Enables the generation of CSS classes for the grid system (e.g. <code>.row</code> , <code>.col-md-1</code> , etc.).

FIM!